Algoritmos e Complexidade

A essência da Computação

Molt BBG

22 de setembro de 2022

The harmony of the world is made manifest in Form and Number, and the heart and soul and all the poetry of Natural Philosophy are embodied in the concept of mathematical beauty.

– D'Arcy Wentworth Thompson

Sumário

Sumário iii

Complexidade 1

1 O que Arthur Não sabe resolver eficientemente 2
1.1 Rei Arthur e o casamento real . . . 2
1.1.1 O que nós estudaremos . . . 3
1.2 A Importância de ser polinomial . . . 3
1.3 Classes de Complexidade . . . 4
1.3.1 Verificar versus Encontrar . . . 5
1.4 Pesquisa, Existência e Não Determinismo . . . 7
1.4.1 O que é o N em NP . . . 8
1.5 Transforme e Conquiste (Redução) . . . 9
1.6 O Mais Difícil de Todos . . . 11
1.6.1 NP-Completude . . . 13
1.6.2 Circuitos e Fórmulas . . . 16
1.6.3 De Circuitos para Fórmulas . . . 17
1.6.4 Projetando Reduções . . . 20
1.6.5 Problemas $NP-$completos . . . 21
1.7 Parecidos porém diferentes . . . 41
1.8 Classe coNP . . . 43
1.9 Memória . . . 46
1.9.1 Mostre-me o Caminho . . . 49
1.9.2 Jogos . . . 53
1.9.3 Jogo de Damas . . . 56
1.9.4 GO . . . 58
1.10 Questão Fundamental: P x NP . . . 59
1.10.1 A Extinção da Magia . . . 64
1.10.2 Limites Superiores são Fáceis, Limites Inferiores são Difíceis . . . 66
1.10.3 Diagonalização e a Hierarquia de Tempo . . . 67
1.11 Provas Interativas . . . 68
1.12 Exercícios . . . 72

Apêndices 82

Bibliografia 83

Alphabetical Index 85

Lista de Figuras

1.1 Donzelas . . . 3
1.2 Arvore-computação . . . 8
1.3 A redução de *Matching Bipartido Perfeito* para *Fluxo Máximo* . . . 10
1.4 Konigsberg . . . 12
1.5 Grafo Konigsberg . . . 12
1.6 Circuito . . . 16
1.7 Grafo Implicação . . . 19
1.8 Relação entre Conjunto Independente, Cobertura por Vértice, e Clique . . . 21
1.9 Redução 3−Sat para Conjunto Independente . . . 23
1.10 Fortaleza estrela . . . 23
1.11 Fortaleza Winterfell . . . 23
1.12 Exemplo redução de 3Sat à Clique . . . 26
1.13 Grafo de Petersen com uma 3-coloração apropriada . . . 28
1.14 Tranqueira escolha - 3−Coloração . . . 29
1.15 Tranqueira restrição - 3-Coloração . . . 29
1.16 Tranqueira restrição - três literais falsos . . . 29
1.17 Tranqueira restrição - um literal verdadeiro . . . 29
1.18 Teste . . . 30
1.19 Instância do quebra-cabeça *TipOver* e sua solução . . . 31
1.20 Tranqueira Or - TipOver . . . 32
1.21 Fio de mão única no TipOver . . . 33
1.22 Tranqueira Or com entrada protegida . . . 33
1.23 Tranqueira And . . . 33
1.24 Passos para utilizar a tranqueira And . . . 33
1.25 Tranqueira escolha . . . 34
1.26 Redução da fórmula $\phi = (x \vee \neg x \vee y) \wedge (\neg y \vee z \vee w) \wedge (\neg w)$ a uma instância de *TipOver* . . . 34
1.27 Tranqueira escolha . . . 35
1.28 Amarrando as variáveis juntas . . . 36
1.29 Um exemplo redução . . . 37
1.30 *Exemplo de redução* $\neq 3 - ESatpara$ Corte Máximo . . . 42
1.31 Autômato celular - Regra 30 . . . 47
1.32 Execução do autômato celular . . . 48
1.33 Geografia . . . 53
1.34 Tranqueira escolha - Geografia . . . 54
1.35 Tranqueira restrição - Geografia . . . 54
1.36 Exemplo redução Jogo Sat à Geografia . . . 55
1.37 Exemplo de execução do Jogo Geografia no grafo de uma instância de Jogo Sat. . . . 56
1.38 Tranqueiras para a redução de Jogo Sat à *Damas Sempre Capturando* . . . 58
1.39 Ladder - Jogo Go . . . 59
1.40 Hierarquia Polinomial . . . 63
1.41 Interação entre Merlin e o rei Arthur no protocolo *IP* . . . 68

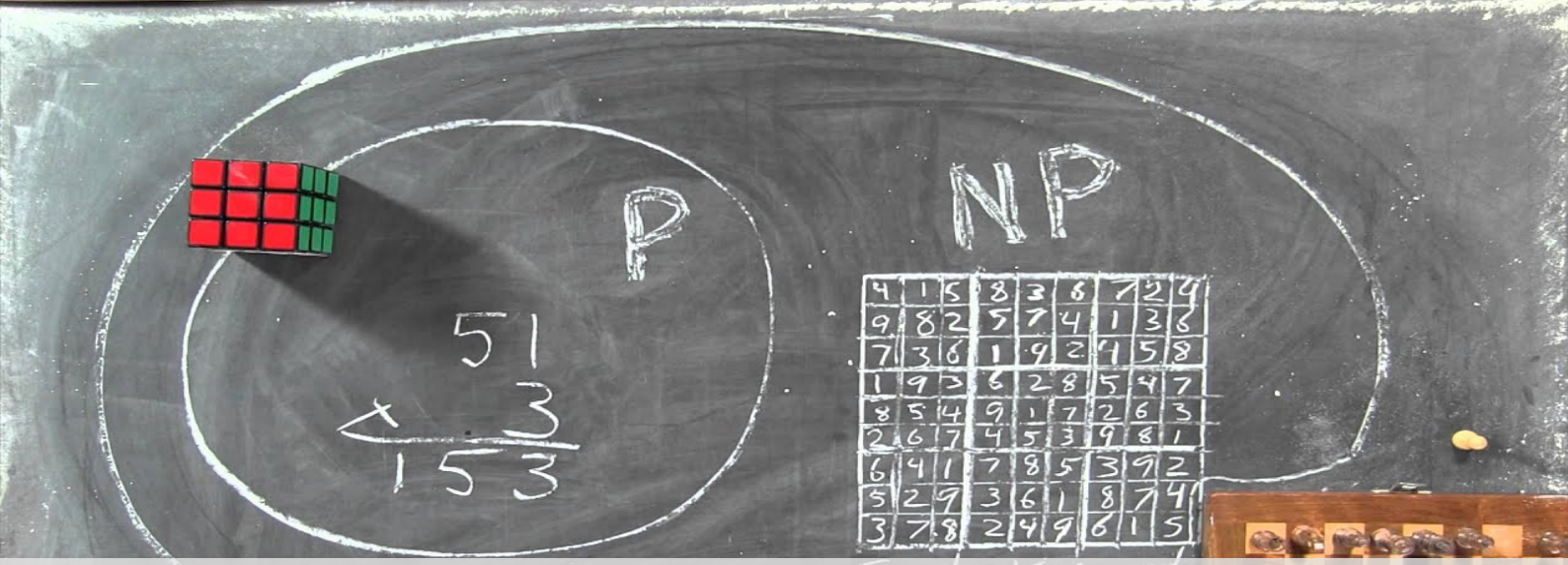
Lista de Tabelas

1.1 Instância Soma Subconjunto para ϕ 39

List of Algorithms

1 Polinomial para 2-Sat 19
2 Palíndromo 47
3 Verificando Certificado 50
4 Savitch 53
5 Geografia 54

Complexidade



1 O que Arthur Não sabe resolver eficientemente

1.1 Rei Arthur e o casamento real

Um dia, o Rei Arthur, em ótimo estado de espírito, tem a ideia: Tenho na corte 150 cavaleiros e 150 donzelas. Por que não casá-los e ter 150 casais?

Sendo o Rei alguém de bom senso, ele pede que cada donzela elabore uma lista de "parceiros aceitáveis". O Rei então se compromete a achar 150 casamentos que respeitem as preferências das donzelas.

Nenhuma donzela terá como esposo um cavaleiro que não seja do seu agrado.

Vendo as 150 listas, o Rei Arthur sente imediatamente uma enorme dor de cabeça - problema de achar 150 casamentos parece muito complicado.

O Rei é inteligente, mas também muito ocupado. Ele não tem tempo de verificar todas as $150! \approx 5,7 \times 10^{262}$ possibilidades ele mesmo. Verificando uma bijeção por segundo, ele levaria $1,8 \times 10^{255}$ anos. A idade do universo é estimada ser da ordem de 10^{22} anos, e o número total de prótons no universo visível é da ordem de 10^{79}.

Nesse instante Arthur se lembra das aulas de Teoria dos Grafos, que ele assistiu quando estava se preparando para assumir o trono. Ele percebe que pode transformar a instância do problema do casamento em uma instância do problema de Fluxo Máximo, de modo que uma solução deste último pode ser convertida, de forma fácil, em uma solução do primeiro problema (Seção 1.5). Ele também se lembra que o problema do Fluxo Máximo pode ser resolvido de forma eficiente, por vários algoritmos, dentre os quais o algoritmo de Ford-Fulkerson. Como o rei foi um aluno muito dedicado do curso de teoria dos grafos, e ele se lembra perfeitamente de como o algoritmo de Ford-Fulkerson funciona. O rei então executa, em um pedaço de papel, o algoritmo e encontra os 150 casamentos. O rei fica exultante com a resposta.

1.1 Rei Arthur e o casamento real 2
1.1.1 O que nós estudaremos . 3
1.2 A Importância de ser polinomial 3
1.3 Classes de Complexidade 4
1.3.1 Verificar versus Encontrar 5
1.4 Pesquisa, Existência e Não Determinismo . . . 7
1.4.1 O que é o N em NP 8
1.5 Transforme e Conquiste (Redução) 9
1.6 O Mais Difícil de Todos 11
1.6.1 NP-Completude 13
1.6.2 Circuitos e Fórmulas . . . 16
1.6.3 De Circuitos para Fórmulas 17
1.6.4 Projetando Reduções . . . 20
1.6.5 Problemas $NP-$completos 21
1.7 Parecidos porém diferentes 41
1.8 Classe coNP 43
1.9 Memória 46
1.9.1 Mostre-me o Caminho . . 49
1.9.2 Jogos 53
1.9.3 Jogo de Damas 56
1.9.4 GO 58
1.10 Questão Fundamental: P x NP 59
1.10.1 A Extinção da Magia . . . 64
1.10.2 Limites Superiores são Fáceis, Limites Inferiores são Difíceis 66
1.10.3 Diagonalização e a Hierarquia de Tempo 67
1.11 Provas Interativas 68
1.12 Exercícios 72

1.1.1 O que nós estudaremos

- Por que alguns problemas permitem que gênios como Ford e Fulkerson tenham "estalos" que os resolvem de forma eficiente, e outros parecem demandar uma busca exaustiva? O que torna esses problemas diferentes?
- Uma gama enorme de problemas encontrar caminhos Hamiltonianos, colorir grafos, satisfazer fórmulas são todos igualmente difíceis. Se nós pudéssemos resolver qualquer um deles eficientemente; nós poderíamos resolver todos de forma eficiente. O que estes problemas tem em comum? Como nós podemos transformar um deles no outro?
- Se nós pudéssemos encontrar caminhos Hamiltonianos eficientemente, nós poderíamos também facilmente encontrar provas curtas, se elas existirem, para os grandes problemas não resolvidos da matemática, tais como a hipótese de Riemann. Nós acreditamos que fazer matemática é mais difícil que isso, e que esta requer toda a nossa criatividade e engenho. Nós estamos certos?
- Uma determinada linguagem de programação, ou um tipo de computador pode resolver problemas que outra não pode? Ou todos os computadores universais são equivalentes?
- Existem problemas que nenhum computador pode resolver, não importando quanto tempo este tenha para resolvê-lo? Existem verdades matemáticas que nenhum sistema axiomático pode provar?
- Se soluções exatas são difíceis de encontrar, nós podemos então encontrar soluções aproximadas? Existem problemas nos quais mesmo soluções aproximadas são difíceis de encontrar? Existem problemas nos quais encontrar soluções ótimos são difíceis, mas nos quais nós podemos encontrar soluções tão próximas do ótimo quanto nós desejarmos?
- O que ocorre se nós focamos no montante de memória que uma computação necessita, ao invés do tempo?
- Se nós nos comprometemos com uma estratégia de resolução de problema, um adversário esperto pode nos confrontar com a instâncias mais difícil para a nossa estratégia. Será que nós podemos derrotá-lo atuando de uma forma imprevisível, por exemplo, arremessando moedas para decidir o que fazer?
- Suponha que Merlin tenha potência computacional ilimitada, mas O Rei Arthur é um mero mortal. Se Merlin sabe que as brancas têm uma estratégia vencedora no xadrez, ele poderá convencer Arthur desse fato, sem jogar uma única partida? Quanto o Rei Arthur pode aprender perguntando à Merlin questões aleatórias?
- Quão raro são instâncias difíceis de uma problema difícil? Se nós construímos instâncias aleatórias de um problema difícil, estas instâncias são fáceis ou difíceis? Quando nós adicionamos mais restrições a um problema, de uma forma aleatória, esse problema fará uma transição de fácil para difícil ou vice-versa?

Figura 1.1: Listas das Donzelas.

1.2 A Importância de ser polinomial

Para propósitos práticos a diferença entre ordem polinomial e ordem exponencial é frequentemente mais importante

que a diferença entre finito e não finito.
Jack Edmonds, 1965

Encontrar um algoritmo de multiplica dois inteiros de n dígitos em tempo $O(n^{1,585})$, ao invés de $O(n^2)$, revela algo importante a respeito da complexidade da multiplicação. E é de interesse prático. Se $n = 10^6$, por exemplo, este algoritmo melhora o tempo de execução por um fator de 300 se as constantes omitidas na notação big O são as mesmas.

No entanto, a distinção mais básica que importará na complexidade computacional é entre funções polinomiais de ne funções exponenciais.

Uma forma de ilustrar a diferença entre polinomiais e exponenciais é pensar sobre como o tamanho dos problemas podem aumentar quando nossa tecnologia melhora. A chamada Lei de Moore é a observação empírica que as medidas básicas da tecnologia computacional, tal como a densidade de transistores em um chip, melhoram exponencialmente com o tempo.

Uma forma comum de enunciar a Lei de Moore é dizer que a velocidade de processamento duplica a cada dois anos. Se o tempo de execução de um algoritmo é $\Theta(n)$, duplicando a velocidade nós também duplicamos o tamanho dos problemas que nós podemos resolver, por exemplo, em uma semana. Mas se o tempo de execução do algoritmo cresce como $O(2^n)$,a duplicação na velocidade aumenta n para somente $n + 1$.

Assim se o tempo de execução é polinomial ou exponencial faz uma enorme diferença no tamanho dos problemas que nós podemos resolver agora e num futuro previsível.

1.3 Classes de Complexidade

Uma das questões mais básicas que nós podemos perguntar a respeito de um problema é se ele pode ser resolvido em tempo polinomial. O conjunto de todos os problemas com a propriedade de serem resolvidos em tempo polinomial formam a classe de complexidade P

Definição 1.3.1 *P é a classe dos problemas para os quais existem algoritmos que resolvem instâncias de tamanho n em tempo $O(n^c)$ para alguma constante c. Por outras palavras, P é a classe dos problemas que o Rei Arthur pode resolver sozinho, sem a ajuda de Merlin.*

Por outro lado, um problema não pertence a classe P se e somente se *não* existir um algoritmo que resolve alguma instância desse problema de tamanho n em tempo polinomial, por exemplo o algoritmo mais eficiente demanda um tempo exponencial $O(2^{\epsilon n})$ para algum $\epsilon > 0$, no pior caso.

Definição 1.3.2 *$Tempo(f(n))$ é a classe de problemas para os quais existem algoritmos que resolvem instâncias de tamanho n em tempo $O(f(n))$.*

Em particular, P contem $Tempo(n)$, $Tempo(n^2)$, e assim sucessivamente. Formalmente,

$$P = \bigcup_{c>0} Tempo(n^c)$$

Da mesma forma

$$Exp = \bigcup_{c>0} Tempo(2^{O(n^c)})$$

Note que a definição da classe P é extremamente robusta com relação à qual modelo de computação nós utilizamos. Por exemplo, mudando de uma máquina de Turing para um modelo RAM, ou mesmo um computador paralelo - desde que este tenha um número polinomial de processadores - economiza no máximo tempo polinomial. Computadores quânticos são outra história.

Mas isso não significa que nós podemos fazer o que nós quisermos.

Considere, por exemplo, a situação na qual nós temos um algoritmo A para o problema da Fatoração. O algoritmo espera receber como entrada um número natural a representado em unário, isto é uma sequência de uns, e executa em tempo linear com relação ao tamanho da entrada. Isto significa que o problema da Fatoração pode ser resolvido em tempo polinomial? A resposta é não, porque nós estamos representando o inteiro em unário ao invés de binário, e a representação em unário é exponencialmente mais longa que a representação em binário, em binário o tamanho da entrada é $\lg n$ ao invés de n. Ou seja nós codificamos a entrada de uma forma ineficiente tornando o problema "mais fácil" de uma forma artificial.

Finalmente, P é robusta com respeito a muitos detalhes de como nós implementamos o nosso algoritmo. Utilizando estruturas de dados mais elaboradas, tal como uma árvore de pesquisa binária ao invés de um vetor, tipicamente reduz o tempo de execução do nosso algoritmo. Mas esta mudança apenas diminui a ordem do polinômio.

O fato de que a classe P permanece inalterada mesmo de nós mudamos os detalhes de nosso computador, nosso formato de entrada ou como nós implementamos o nosso algoritmo, sugere que pertencer à classe P é uma propriedade fundamental de um problema, ao invés de uma propriedade de como nós o resolvemos. Por outras palavras, a questão da pertinência de *Caminho Hamiltoniano* à classe P é uma questão matemática, não uma questão subjetiva a respeito de nossas habilidades computacionais.

1.3.1 Verificar versus Encontrar

O problema de dizer se um ciclo Hamiltoniano existe tem a propriedade de que encontrar uma solução é difícil mas verificá-las é fácil. Outro exemplo desse fenômeno é fatoração de inteiros. Até onde nós sabemos não existe nenhuma forma eficiente de fatorar um inteiro grande n em seus divisores - pelo menos sem um computador quântico - por outro lado , dado dois números p e q é fácil multiplicá-los, e verificar se $n = pq$.

Este fato foi ilustrado de forma belíssima em um encontro da Sociedade Matemática Americana em 1903. O matemático Frank Nelson Cole deu uma palestra sem palavras ele simplesmente executou a multiplicação

$193707721 \times 761838257287 = 147573952589676412927$, no quadro negro. O número do lado direito é $2^{67} - 1$, o qual o matemático francês Marin Mersenne conjecturou que fosse primo. Em 1876, Edward Lucas provou que este é composto, mas não apresentou nenhuma indicação de quais números o dividiam. A audiência sabendo quão difícil é fatorar um número com 21 dígitos ovacionou a apresentação de Cole de pé.

Que outros problemas tem essas características, e como eles se relacionam? A classe de tais problemas formam uma classe de complexidade, a chamada classe NP. Existem várias definições de NP, todas equivalentes, algumas definições falam de provas, outras de testemunhas, outras de adivinhações sortudas, etc. Nós veremos que NP inclui uma grande variedade de problemas fundamentais: colorir mapas, satisfazer sistemas de restrições, etc.

Definição 1.3.3 *Um problema de decisão está em NP se quando a resposta para uma instância particular é "sim", existe uma prova simples deste fato. Em termos da relação entre o Rei Arthur e Merlin, um problema pertence a classe NP se quando a resposta é "sim" existe uma prova simples que Merlin pode entregar ao Rei Arthur que o convencerá que ele não está mentindo. É essencial que o Rei possa verificar a prova sozinho sem a ajuda de Merlin.*

Por prova simples nós queremos dizer uma prova que pode ser verificada em tempo polinomial.

É importante perceber que NP é uma noção profundamente assimétrica, Se eu conheço um caminho é fácil provar para você que um determinado grafo é Hamiltoniano, mas como eu posso provar que um grafo não é Hamiltoniano? Ou seja Merlin pode soterrar o Rei Arthur com uma lista enorme de caminhos, e afirmar que uma pesquisa exaustiva nesta lista revelará que nenhum deles é o caminho procurado. Mas isso não é uma prova simples, uma vez que o Rei não tem tempo de verificá-la, afinal levará uma eternidade checar a prova.

Se um problema está em P, então este tem um algoritmo polinomial que é garantido dar a resposta correta. Se a entrada é uma instância "sim", a sequência de passos executados por esse algoritmo, o qual termina retornando "sim", é uma prova deste fato. Por outras palavras, se um problema está em P o rei Arthur não precisa do certificado entregue por Merlin, já que ele tem um algoritmo que pode executado e assim verificar se Merlin não está mentindo. Logo

$$P \subseteq NP \tag{1.1}$$

Essencialmente , P é o subconjunto de NP consistindo dos problemas para os quais existe alguma ideia matemática que nos permite evitar a pesquisa exaustiva. Por exemplo:

- quebrar o problema em partes menores (Transformada de Fourier),
- construir a solução a partir de soluções para subproblemas mais simples (Caminho mais curto),

- ampliar uma solução gulosa (Árvore geradora mínima),
- melhorar uma solução até encontrar um ótimo (Fluxo máximo),
- reduzir a um problema que nós já sabemos como resolver (Matching perfeito).

Na ausência de tal ideia, parece difícil evitar uma pesquisa através de todo espaço de soluções possíveis, e isto demanda um tempo exponencial se o número de soluções possíveis é exponencial.

Por outro lado, é muito difícil saber se nós já encontramos o melhor algoritmo possível. e é muitíssimo difícil provar que nenhum algoritmo polinomial existe. Dessa forma, enquanto existem muitos problemas em NP, tal como Caminho Hamiltoniano, ninguém foi capaz, até agora, de encontrar um algoritmo polinomial ou provar que tal algoritmo não existe.

1.4 Pesquisa, Existência e Não Determinismo

Até este ponto nós apresentamos uma definição informal da classe NP, e alguns exemplos que ilustram esta classe. Nós agora discutiremos em mais detalhes a classe NP.

Definição 1.4.1 *NP é a classe dos problemas tais que se x é uma instância "sim" de A então existe um w tal que $B(x, w)$ é uma instância "sim" de B, onde B é um problema de decisão pertencente à classe P, e o tamanho de w é polinomial no tamanho de x.*

Nós chamamos w um certificado do fato de que x é uma instância "sim" de A. w também é chamado de uma testemunha.

Esta definição está apenas afirmando a presença de um prova (w) que pode ser descrita por meio de um número polinomial de bits. Por exemplo, se x é um grafo com n vértices, o número de bits necessários para representar um caminho Hamiltoniano w é $|w| = n\,lg\,n = poli(|x|)$. Uma vez que a entrada (x, w) de B tem tamanho total $|x| + |w|$ e B está em P, nós temos que o tempo total de execução de B é $O((|x| + |w|)^k)$, ou seja polinomial no tamanho de x.

Quanto tempo é necessário para resolver este problema deterministicamente? Uma vez que$|w| = O(n^k)$, então existem $2^{O(n^k)}$ certificados possíveis. Nós podemos verificar todos executando-se o algoritmo B sobre cada um deles, assim

$$NP \subseteq Exp. \tag{1.2}$$

Aqui $Exp = Tempo(2^{O(n^k)})$, é a classe dos problemas que podem ser resolvidos em tempo exponencial. Muitos acreditam que esta é a melhor inclusão possível de NP em uma classe determinística, por outras palavras, que o único método geral para resolver problemas em NP é uma busca exaustiva.

Para estabelecer isso de uma forma clara, muitos acreditam que existe uma constante $\alpha > 0$ tal que, para qualquer $g(n) = o(2^{n^\alpha})$, nós temos que

$$NP \not\subseteq Tempo(g(n)). \tag{1.3}$$

Dessa forma resolver problemas em NP geralmente demandam tempo exponencial, onde "exponencial" significa 2^{n^α} para algum α positivo. Poderia ser possível que $NP \subseteq Tempo(g(n))$ para algum $g(n)$ que é superpolinomial mas ainda subexponencial, tal como $n^{lg\, n}$. Isso **não** implicaria que $P = NP$, mas mostraria que nós podemos fazer melhor que uma busca exaustiva.

1.4.1 O que é o N em NP

Em muitos livros textos, NP é definido não em termos de um certificado, mas em termos de um tipo curioso de computador. O N em NP significa "não determinístico", e um computador não determinístico é capaz de fazer várias escolhas possíveis em cada passo da computação. Imagine uma linguagem de programação na qual nós temos a seguinte instrução mágica:

goto ambas linha 1, linha 2

Esta instrução divide a computação em dois processos paralelos, um continuando na linha 1 e outro continuando na linha 2. Quando nós encontramos mais e mais dessas instruções, o programa ramifica-se em uma árvore de caminhos de computação. Nós dizemos que o tempo de computação é polinomial se todo caminho termina em um número polinomial de passos. Por último, nós definimos a saída de um computador não determinístico da seguinte forma: esta é "sim" se algum desses caminhos conduz ao "sim" e é "não" se todos os caminhos conduzem ao "não".

Uma vez que nós podemos ramificar em todo passo de um algoritmo, o número de caminhos da computação pode crescer de forma exponencial como uma função do tempo de execução. Isto confere aos computadores não determinísticos uma capacidade não realista de pesquisar um espaço exponencialmente grande em tempo polinomial.

Computadores não determinísticos nos permitem ter a seguinte definição alternativa da classe NP.

Definição 1.4.2 *NP é a classe dos problemas para os quais existem programas não determinísticos que executam em tempo polinomial em n tal que a entrada é uma instância "sim" se e somente se existir um computação que retorna "sim".*

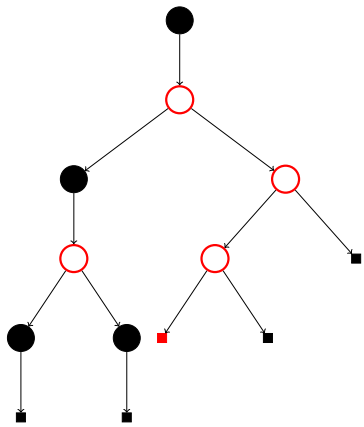


Figura 1.2: A árvore de computação de um programa não determinístico.

De uma forma similar, para qualquer função $f(n)$ nós podemos definir $NTempo(f(n))$ como:

Definição 1.4.3 *$NTempo(f(n))$ é a classe dos problemas para os quais existem programas não determinísticos que executam em tempo $O(f(n))$, tal que a entrada é uma instância "sim" se e somente se existir um caminho de computação que retorna "sim".*

Note que computadores não determinísticos podem executar programas determinísticos. Isso fornece outra prova que $P \subseteq NP$ e mais geralmente $Tempo(f(n)) \subseteq NTempo(f(fn))$.

Nós também podemos simular um programa não determinístico A com um programa determinístico B, dando a B uma string adicional de bits que diz qual caminho seguir em cada instrução "goto ambas". Esta string é o certificado w, e seu tamanho é no máximo o tempo de execução de A. Por outro lado, nós podemos simular tanto o Provador(Merlin) quanto o Verificador (Rei Arthur) com um programa não determinístico A que começa adivinhando o certificado, com $|w|$ igual ao número de instruções "goto ambas", e então procede a verificação executando o programa do Rei Arthur.

Nós devemos enfatizar que computação "não determinística" não tem nenhuma relação com computação aleatória ou probabilística.

1.5 Transforme e Conquiste (Redução)

> "Esse é o segredo da vida. . . Substituir uma preocupação por outra." (Charlie Brown)

Suponha, por um instante, que o Rei Arthur além de aulas de cavalaria também fez um curso de teoria dos grafos em Hogwards. No curso foi ensinado uma série de problemas e algoritmos em grafos. Um dos problemas foi o problema do *Matching Bipartido Perfeito*:

Matching Bipartido Perfeito
Entrada: Um grafo bipartido G.
Pergunta: G possui um matching perfeito?

Um *matching* é um subconjunto $M \subseteq E$ consistindo de arestas disjuntas, que cobre tantos vértices quanto possíveis. Se nós somos afortunados, o subconjunto M será *perfeito*, ou seja M cobrirá todo vértice de G.

Ao se lembrar das suas aulas de teoria dos grafos o Rei Arthur percebe que ele pode transformar qualquer instância do problema do *Casamento Real* em um instância do problema *Matching Bipartido Perfeito*. Cada donzela e cada cavaleiro será representado por um vértice. E existirá uma aresta da donzela A para o cavaleiro a se e somente se a estiver na lista de cavaleiros aceitáveis da donzela A. Note que o grafo resultante da transformação será bipartido.

Isso parece não ajudar muito, afinal existe um número exponencial de matchings possíveis, e não é óbvio como encontrar o matching perfeito em tempo polinomial. A boa notícia, para as donzelas, é que o Rei foi um aluno aplicado do curso de grafos, e ele já sabe como fazer isto. Ele precisa apenas traduzir este problema para outro que ele já resolveu antes.

A figura abaixo mostra como nós podemos transformar uma instância de *Matching Bipartido Perfeito* em uma instância de *Máximo Fluxo*. Nós transformamos cada aresta (U, v) em uma aresta dirigida $U \rightarrow v$, apontando do vértice que representa a donzela para o vértice que

representa o cavaleiro. Nós então adicionamos dois vértices s, t com arestas de s para cada vértice que representa uma donzela, e arestas de cada vértice que representa um cavaleiro para u. Finalmente, nós damos a toda aresta nesta rede a capacidade de 1.

Nós afirmamos que se o fluxo máximo nessa rede for igual ao número de donzelas então o grafo G possui um matching perfeito, e o Rei Arthur pode casar todas as donzelas. Mostre que existe um matching perfeito se e somente se a rede possui um fluxo máximo de valor m, onde m é o número de donzelas e cavaleiros.

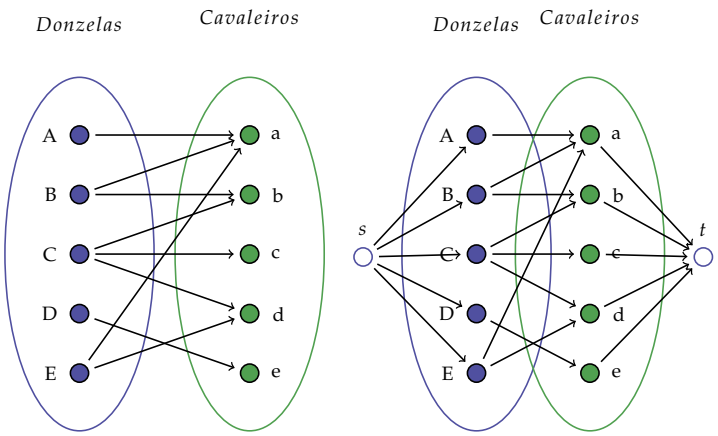


Figura 1.3: A redução de *Matching Bipartido Perfeito* para *Fluxo Máximo*

Assim o Rei Arthur pode resolver o problema Matching Bipartido Perfeito executando essa transformação e então resolvendo a instância de Fluxo Máximo. Uma vez que Fluxo Máximo está em P, e desde que a transformação é fácil, ou seja pode ser executada em tempo polinomial, isto prova de Matching Bipartido Perfeito está em P.

Este tipo de transformação é normalmente chamada de *redução* em ciência da computação. Em um nível, este diz que nós podemos resolver Matching Bipartido Perfeito chamando um algoritmo para Fluxo Máximo como uma sub-rotina. Mas em outro nível este diz algo ainda mais profundo - Matching Bipartido Perfeito não é mais difícil que Fluxo Máximo. Nós podemos escrever isto da seguinte forma:

$$Matching\,Bipartido\,Perfeito \leq_K Fluxo\,Máximo. \tag{1.4}$$

Considere o fato que, antes que o Rei aprendesse o algoritmo de tempo polinomial para *Fluxo Máximo*, não erá nada óbvio que ele considerasse *Matching Bipartido Perfeito* pertencesse à classe P. Mas assim que ele aprendeu um algoritmo de tempo polinomial para *Fluxo Máximo* ele também aprendeu um algoritmo para *Matching Bipartido Perfeito*.

Reduções tem uma outra aplicação in complexidade computacional. Uma $A \leq_K B$ mostra que A é no máximo tão difícil quanto B, esta também mostra que B é pelo menos tão difícil quanto A. Assim nós obtemos um limite inferior condicional à respeito da complexidade de B bem como limite superior condicional sobre a complexidade de A. Mais especificamente, quando $B \in P$ implica que $A \in P$, é igualmente verdade que $A \notin P$ implica $B \notin P$.

Quando nós dizemos que A pode ser reduzido à B parece que B é menor ou mais simples que A. Contudo, isto significa exatamente o oposto A pode ser entendido como um caso especial de B,dessa forma B é mais geral, e portanto mais difícil que A. Colocando de outra forma, B é *expressivo o bastante* para descrever as metas e restrições de A.

Reduções desempenham três papeis. Por um lado se $A \leq_K B$ e B pode ser resolvido em tempo polinomial então A também pode ser resolvido em tempo polinomial. Dessa forma reduções são uma das técnicas mais úteis no projeto de algoritmos; na verdade não seria muito ousadia dizer que reduções são a forma mais comum de projeto de algoritmos.

Por outro lado, se $A \leq_K B$ e A não pode ser resolvido em tempo polinomial então B também não pode ser resolvido em tempo polinomial. Portanto, reduções são extremamente úteis para provar resultados de intratabilidade.

Por último, se $A \leq_K B$ e $B \leq_K A$, neste caso nós utilizamos a notação $A \equiv B$. Isto significa que um deles pode ser resolvido em tempo polinomial se e somente se o outro também pode.

Resumindo:

Reduções classificam os problemas de acordo com a sua dificuldade relativa.

1.6 O Mais Difícil de Todos

Esta seção começa com uma adaptação do diálogo entre Aquiles e a Tartaruga que você pode encontrar no maravilhoso livro" Gödel, Escher, Bach: an Eternal Golden Braid" de Douglas Hofstadter. Aquiles (guerreiro grego, o mais rápido dos mortais) e uma velha tartaruga estão dando um passeio dominical pelo bosque. Eles resolveram subir uma colina em cujo topo diz-se haver uma maravilhosa casa de chá.

Aquiles: Magnífico Caranguejo ...

Tartaruga: Magnífico Caranguejo?

Aquiles: Eu ia dizer: caranguejo mais inteligente que nosso amigo, o Caranguejo, eu nunca vi. Ele deve ser pelo menos duas vezes mais esperto que qualquer outro caranguejo. Ou talvez três vezes mais esperto que qualquer outro caranguejo. Ou talvez ...

Tartaruga: Alma minha! Como magnificas o Caranguejo!

Aquiles: Bom, é que sou um admirador dele ...

Tartaruga: Não precisa se desculpar. Eu também o admiro. Falando em admiração, eu lhe contei que o Caranguejo possui um dom especial para a interpretação dos sonhos?

Aquiles: Que interessante! Talvez ele possa me ajudar a entender um sonho que tem povoado as minhas noites insones.

Caranguejo: O senhor Tartaruga está exagerando!! Na verdade eu sou apenas um discípulo entusiasmado do trabalho do Dr. Sigmund Freud.

Permita me então ouvir o seu sonho prezado Aquiles, quem sabe eu possa aplicar alguns dos ensinamentos do Dr Freud e ajudá-lo.

Aquiles: Nesse sonho eu estou no portão de entrada da cidade de Konigsberg, quando um ancião aproxima-se e me convida para conhecer a cidade. Ele me desafia a percorrer a cidade atravessando cada ponte exatamente uma vez. Mas sempre que eu estou quase completando esta tarefa o meu calcanhar dói de uma forma terrível e eu tenho que parar às margens do rio que banha a cidade. Eu sempre acordo nesse ponto.

Tartaruga: Que interessante, eu tenho tido um sonho muito parecido. Eu estou na entrada da cidade de Veneza e também encontro um velho. Ele me desafia a visitar cada ilha da cidade exatamente uma vez, não importando o número de vezes que eu atravesso cada ponte. E toda vez que eu estou quase completando a tarefa o meu casco dói e eu tenho que parar às margens de um canal. Eu também sempre acordo nesse ponto.

Aquiles. Que estranho!! Talvez o nosso amigo Magnífico Caranguejo possa interpretá-los. Na verdade creio que uma interpretação será suficiente, afinal de contas os dois sonhos são idênticos, a diferença entre eles é mínima, eu diria mesmo irrelevante.

Caranguejo: O Aquiles como eu invejo a sua sabedoria!! Espero que um dia os deuses me honrem com uma capacidade de compreensão tão grande quanto a sua. Nesse momento tudo que eu posso dizer é que em sabedoria você é capaz de reconhecer quando uma tarefa não pode ser realizada. Afinal o ciclope Euler mostrou que não é possível atravessar cada ponte de Konigsberg exatamente uma vez.

Aquiles: Impossível!! O ciclope deve ter esquecido de olhar alguma alternativa, afinal existem 7 pontes e portanto existem, em tese, 128 respostas possíveis. E checá-las levaria um tempo considerável, com certeza ele deve ter esquecido de considerar alguma resposta.

Figura 1.4: Konigsberg.

Tartaruga: Ah! Eu acho que eu li algo a respeito no diário Prussiano de Matemática. Bem como era mesmo ... A sim!! Euler mostrou que, independente do número de pontes da cidade, para atravessar cada ponte exatamente uma vez, toda vez que nós chegamos a um pedaço de terra situado de um lado da ponte, nós temos que partir dessa pedaço de terra por uma ponte diferente. Se eu me lembro bem, ele chamou cada pedaço de terra de vértice e cada ponte de aresta. Portanto, o números de arestas que tocam cada vértice deve ser par. Isto deve ser verdade para todos os vértices exceto os vértices de começo e fim, que devem ter grau ímpar a menos que eles coincidam e o caminho é um ciclo.

Aquiles: Felizmente eu trouxe comigo um mapa da cidade de Konigsberg.Vamos ver

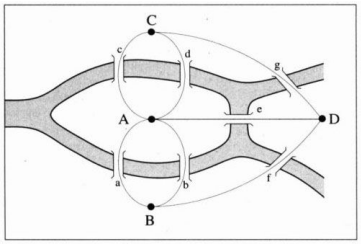


Figura 1.5: Grafo Konigsberg.

Tartaruga: Pelo que eu entendi essa cidade pode ser representada pelo seguinte grafo:

Note caro Aquiles que o grau de todos os vértices é ímpar. Logo você não pode atravessar cada ponte exatamente uma vez;

Caranguejo: Como eu havia lhe dito Aquiles, tudo que eu posso fazer é analisá-lo segundo os princípios de meu mestre Freud. Parece me que você, Aquiles, sentia-se preso em uma espécie de mito de Sísifo, destinado

a uma insatisfação perpétua. Felizmente, graças ao grande Euler nós podemos libertá-lo deste destino cruel.

Aquiles: Ufa!! Escapamos de uma boa, meu amigo Tartaruga.

Caranguejo: Infelizmente meu caro Aquiles eu não posso dizer o mesmo do senhor Tartaruga.

Aquiles: Mas como? Os dois sonhos são muito parecidos. A única diferença é o tamanho dos grafos gerados pelas cidades.

Caranguejo: Existe uma diferença grande entre os dois. O teorema de Euler.

Tartaruga: É verdade Aquiles. Até onde nós sabemos ninguém ainda encontrou um teorema similar para o meu problema. Suponha que eu queira saber se um determinado grafo possui um caminho de Aquiles, ou seja um caminho que atravessa toda aresta do grafo exatamente uma vez, tudo que nós necessitamos fazer é verificar se o grau de cada vértice é exatamente par, com a exceção de dois vértices que devem ser ímpar. Por outro para saber se um grafo possui um caminho da Tartaruga, um caminho que percorre cada vértice exatamente uma vez, ainda demanda, até onde nós sabemos, verificar todos os caminhos possíveis.

Aquiles: Pelo menos o nosso amigo Tartaruga tem apenas 150 anos, e como nós sabemos as tartarugas vivem mais de 300 anos. Então o nosso amigo terá tempo bastante para encontrar uma solução para o seu problema.

Caranguejo: Ah Aquiles! Se o senhor Tartaruga tivesse sonhado com outra cidade talvez houvesse esperança, mas Veneza tem 128 ilhas e 400 pontes; logo o número de soluções possíveis é 128!. Se ele for capaz de verificar uma solução por segundo ele levará aproximadamente $1.22 * 10^{208}$ anos!!!

Tartaruga: É melhor eu acelerar o passo e chegar logo a casa de chá. Talvez lá eu encontre um bom chá calmante para tranquilizar o meu sono.

Por que alguns problemas permitem que gênios como Euler tenham lampejos de criatividade que os resolvem de forma eficiente, e outros parecem demandar uma busca exaustiva? O que torna esses problemas diferentes?
Você pode pensar que existe um método muito mais eficiente para determinar se um grafo possui um caminho da Tartaruga[a], e que nós simplesmente não fomos, ainda, espertos o bastante para encontrá-lo. Mas como nós veremos neste curso, existem boas razões para se acreditar que tal método não existe. Em um última instância, a dificuldade do problema da Tartaruga está relacionada com as nossas crenças mais profundas a respeito de criatividade científica e matemática.

[a] Este problema é normalmente chamado de problema do caminho Hamiltoniano, mas eu preferi chamá-lo, por razões óbvias, de problema do caminho da Tartaruga.

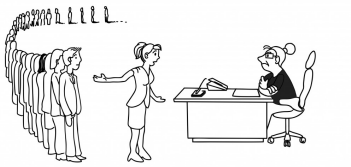
1.6.1 NP-Completude

Existem problemas que iludem todas as estratégias algorítmicas conhecidas, tal como 3-ESat, Caminho Hamiltoniano, Jantar Real, etc. Estes problemas necessariamente demandam uma pesquisa exponencial? Ou

nós simplesmente não temos sido espertos o bastante para encontrar algoritmos eficientes para eles?

Nossa inabilidade em resolver estes problemas não é acidental. Em certo sentido eles são os problemas mais difíceis da classe NP. Nomeadamente, cada um desses problemas tem a incrível capacidade de

expressar todos os outros problemas da classe NP, Esta propriedade é chamada de $NP-$Completude, e implica que se qualquer um desses problemas necessariamente demandar tempo exponencial, então todos demandam.



"I can't find an efficient algorithm, but neither can all these famous people."

Definição 1.6.1 *$NP-$completude*

Um problema B é $NP-$completo se, para qualquer problema A em NP, existir uma redução de tempo polinomial de A para B, e se B também pertencer à classe NP.

Por outras palavras, um problema $B \in NP$ é $NP-$completo se e somente se $A \leq_K B$ para todo $A \in NP$.

Como nós podemos provar que um problema é $NP-$completo? Deve ser possível traduzir instâncias de qualquer problema $A \in NP$ em instâncias de B, de modo que se nós pudéssemos resolver B, nós poderíamos resolver A também. Colocando de outra forma, B deve ser tão geral, capaz de expressar tipos tão diferentes de variáveis e restrições, que este contem a estrutura de todo outro problema da classe NP.

Note que definir alguma coisa não implica que esta coisa existe. Nós podemos definir muitas entidades que simplesmente não existem, por exemplo, mulas sem cabeça, Curumim, etc. Ou seja, não é óbvio que existam problemas $NP-$completos. Por que não poderia existir dois problemas incomparáveis A e B, de modo que não exista nenhum $C \in NP$ com a propriedade que $A \leq_K C$ e $B \leq_K C$? Por que não poderia existir uma sequência infinita de problemas $X_1, X_2, X_3, \ldots$ em NP, onde cada problema é estritamente mais difícil que o anterior? Para provar que um problema é $NP-$completo, nós devemos mostrar como este poderia codificar *qualquer* problema em NP. Isto está longe de ser uma tarefa fácil.

Felizmente no início da década de 1970 dois pesquisadores, de forma independente, lograram revelar a existência de problemas $NP-$completos.

Teorema 1.6.1 *[6, 13]* SAT *é $NP-$completo.*

[6]: Cook (1971), *The Complexity of Theorem-Proving Procedures*
[13]: Levin (1973), *Universal Sequential Search Problems*

Nós não apresentaremos a prova original de Cook aqui porque, apesar dessa não ser especial difícil, nós acreditamos que existe uma forma mais simples e direta de entender o seu resultado. A ideia consiste em considerar o seguinte problema:

Problema Existência de Certificado
Entrada: Um programa $\Pi(x, w)$, uma entrada xe um inteiro t escrito em unário.
Pergunta: Existe um w de tamanho $|w| \leq t$, tal que $\Pi(x, w)$ retorna "sim" depois de t ou menos passos?

Este problema é $NP-$completo porque este repete a definição da classe NP. Por exemplo, se nós queremos reduzir Caminho Hamiltoniano à Existência de Certificado, nós fazemos Π ser o programa verificando certificado que toma como argumento um grafo x e um caminho w e retorna "sim" se w é um caminho Hamiltoniano em x.

Por que nós expressamos t em unário? Isto assegura que t é menor ou igual ao tamanho total n da entrada do Problema Existência de Certificado, assim tanto o tempo de execução de Π e o tamanho do certificado w são polinomiais como uma função de n. Assim nós podemos verificar w tempo polinomial em n executando Π por t passos, portanto, Problema Existência de Certificado está em NP.

É interessante saber que pelo menos um problema $NP-$completo existe. No entanto, Problema Existência de Certificado é um problema abstrato, que parece não guardar nenhuma relação com os problemas que nós encontramos no dia a dia. O fato surpreendente é que muitos problemas interessantes são $NP-$completos. Cada um desses problemas são pelo menos tão difíceis quanto qualquer outro problema da classe NP, e tentar encontrar um algoritmo polinomial para qualquer um deles é equivalente a tentar encontrar um algoritmo polinomial para a classe inteira. Antes, quando você se defrontava com um problema específico, como por exemplo Jantar Real ou Sat, você pode ter imaginado que uma ideia esperta específica para àquele problema lhe permitiria resolvê-lo em tempo polinomial. Mas a $NP-$completude ergue uma muralha. Se qualquer um destes problemas possui um algoritmo de tempo polinomial, então todos também têm:

Se qualquer problema $NP-$completo está em P, então $P = NP$.

Muitas pessoas se alegrariam se isto fosse verdade, uma vez que existem muitos problemas práticos que nós desejaríamos que nós pudéssemos resolver de forma mais eficiente. Para teórico da computação, por outro lado, isto seria devastador, uma vez que implicaria em uma completa reformulação da nossa visão de mundo. Seria como descobrir que a realidade é o sonho de um ser demiurgo, ou que nossa imaginação não desempenha nenhuma função nas descobertas matemáticas. As consequências de $P = NP$ seriam tão grandes que nós consideramos $NP-$completude como essencialmente uma "prova" que um problema não pode ser resolvido em tempo polinomial, que não existe nenhuma forma de evitar uma pesquisa exponencial pelo espaço das soluções possíveis.

Nós devemos realçar que quando nós discutimos $NP-$completude nós nos restringimos as reduções que transformam uma instância de um problema A em uma instância de um problema B, este tipo de redução é

chamada de redução de Karp. Nós também demandamos que a nossa redução mapeie instâncias "sim" para instâncias "sim", e instâncias "não" para instâncias "não". Então se x é uma instância "sim" de um problema A, existe um certificado para a correspondente instância "sim" $f(x)$ de B. Tais reduções preservam a natureza assimétrica de NP. Em particular, se B está em NP e $A \leq_K B$, então A também está em NP. Dessa forma:

Definição 1.6.2 *Uma redução de tempo polinomial de A para B é uma função f, computável em tempo polinomial, tal que se x é uma instância de A, então $f(x)$ é uma instância de B. E $f(x)$ é uma instância "sim" de B se e somente se x é uma instância "sim" de A.*

Note que a composição de duas reduções de tempo polinomial é uma redução de tempo polinomial. Assim a relação de redução é transitiva:

Se $A \leq_K B$ e $B \leq_K C$, então $A \leq_K C$.

Portanto, para provar que um determinado problema A é $NP-$completo, é suficiente reduzir um problema que nós já sabemos ser $NP-$completo a ele. Com a condição que o problema A também pertença a classe NP. Estas reduções criam uma árvore de problemas $NP-$completos. Desde que o conceito de $NP-$completude foi descoberto (inventado) a árvore tem crescido para incluir problemas em teoria dos grafos, álgebra, planejamento, otimização, física, biologia, quebra-cabeças, e muitos outros campos.

1.6.2 Circuitos e Fórmulas

Um *circuito Booleano* é um grafo dirigido com um conjunto de nós fonte chamados de entradas, e um ou mais nós destinos chamados de saídas. Cada nó interno, ou "porta", é rotulado de And, Or, ou Not, e produz a correspondente função de sua entrada. Este grafo é acíclico, significando que não existem laços - a informação flui da entrada para a saída.

Nós podemos dividir um circuito Booleano em uma série de camadas, onde cada camada recebe sua entrada das portas da camada acima, A profundidade de um circuito é o caminho mais longo da entrada para a saída. Sua largura é o maior número de portas em uma camada, e seu tamanho é o número total de portas.

Nós nos concentraremos apenas nos circuitos Booleanos com uma única saída. Tal como com fórmulas Booleanas, nós dizemos que um circuito é satisfatível se existe uma atribuição de valores lógicos falso e verdadeiro as suas entradas que faz a saída verdadeira. Por exemplo o circuito mostrado na Figura 1.6 é satisfatível, uma vez que a saída é verdadeira se C = verdadeiro.

Agora considere o seguinte problema:

> Problema Circuito Sat
> *Entrada: Um circuito Booleano C.*
> *Pergunta: C é satisfatível?*

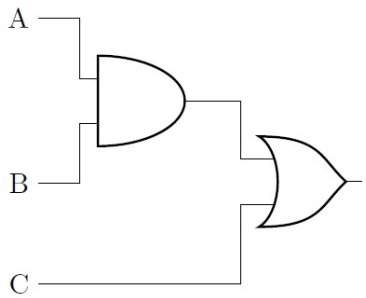


Figura 1.6: Circuito Booleano

Nós afirmamos que Existência de Certificado $\leq_K$ Circuito Sat, e que portanto Circuito *Sat* é $NP-$completo. Para ser mais preciso, nós descreveremos como converter uma instância (Π, x, t) de *Existência de Certificado* em um circuito Booleano C de tamanho polinomial, tal que C é satisfatível se e somente se existir um certificado w tal que $\Pi(x, w)$ retorna "sim" em no máximo t passos.

A redução consiste em compilar o programa $\Pi(x, w)$ em um circuito Booleano C. Nós podemos fazer isso uma vez que circuitos Booleanos são poderosos o bastante para expressar qualquer algoritmo. Se você duvida dessa afirmação considere o fato de que no mundo real nós compilamos programas em sequências de instruções em linguagem de máquina, que manipulam poucos bits de cada vez. Nós executamos estas instruções em chips feitos de portas lógicas, cada uma delas construída de um conjunto de transistores.

Cada camada de C corresponde a um único passo de $\Pi(., .)$. Os fios indo de uma camada para a próxima transportam, em forma binária, os valores das variáveis que Π utiliza. Se Π contem laços "while" ou "for", que executam a mesma instrução muitas vezes, nós construímos a mesma instrução muitas vezes, nós construímos C "desenrolando" estes laços e repetindo as camadas correspondentes do circuito.

Uma vez que o tempo de execução de Π é no máximo t passos, o circuito C possui $O(t)$ camadas, Além disso, Π tem tempo de acessar $O(t)$ variáveis diferentes ou localizações da memória do computador assim o número de bits que nós necessitamos transportar de uma camada para a próxima é $O(t)$. Portanto a largura e a profundidade de C são ambos $O(t)$, e seu tamanho total é $O(t^2)$.

Qualquer programa que verifica uma solução pode ser compilado em um circuito Booleano.

A saída do circuito é "verdadeira" se e somente se existe um conjunto de valores lógicos para as entradas que faz com que a saída tenha o valor lógico verdadeiro. Resumindo, Circuito Sat é $NP-$completo porque circuitos Booleanos são poderosos o bastante para executar qualquer computação finita.

1.6.3 De Circuitos para Fórmulas

Um problema central em computação é o problema Sat.

Assuma que $\mathcal{A}$ é o alfabeto de uma linguagem proposicional $\mathcal{L}$. Uma fórmula proposicional ϕ é construída indutivamente utilizando-se as letras proposicionais existentes em $\mathcal{A}$ e os conectivos lógicos $\wedge, \vee$. Em lógica clássica, variáveis proposicionais $p_1, \ldots, p_n$ são "verdadeiras" ou "falsas". Um interpretação é uma atribuição de "verdadeiro" ou "falso" a toda variável proposicional. Um literal é uma variável proposicional ou a negação de uma variável proposicional $\neg p_i$. Uma fórmula Booleana ϕ está em forma normal conjuntiva se ela é uma conjunção de cláusulas, onde cada cláusula é uma disjunção de literais, por exemplo:

$$\phi(p, t, r) = (p \vee \neg t) \wedge (\neg p \vee \neg r) \wedge (t \vee r) \wedge (p \wedge t) \wedge (\neg t \vee r) \quad (1.5)$$

O problema da Satisfabilidade é:

> *Problema* SAT
> *Entrada: Uma fórmula Booleana $\phi(x_1, \ldots, x_n)$, escrita em FNC.*
> *Pergunta: ϕ é satisfatível?*

3SAT é um problema SAT em que toda cláusula possui três literais.

Agora nós mostraremos que *3SAT* é $NP-$completo. Nós faremos isso mostrando que

$$\text{CIRCUITO SAT} \leq_K \text{3SAT}.$$

Para reduzir *CIRCUITO SAT* à *3SAT*, nós necessitamos mostrar como transformar um circuito Booleano C para uma fórmula FNC ϕ *3SAT* que é satisfatível se e somente se C também é. Considere as seguintes construções:

$$y = x_1 \wedge x_2 \quad \Leftrightarrow \quad (x_1 \vee \overline{y}) \wedge (x_2 \vee \overline{y}) \wedge (\overline{x}_1 \vee \overline{x}_2 \vee y) \tag{1.6}$$
$$y = x_1 \vee x_2 \quad \Leftrightarrow \quad (\overline{x}_1 \vee y) \wedge (\overline{x}_2 \vee y) \wedge (x_1 \vee x_2 \vee \overline{y}) \tag{1.7}$$
$$y = \overline{x} \quad \Leftrightarrow \quad (x \vee y) \wedge (\overline{x} \vee \overline{y}) \tag{1.8}$$

Finalmente, a exigência que a saída seja "verdadeira" é apenas uma cláusula com uma única variável. Por exemplo, a fórmula Booleana da Figura 1.6 torna-se a fórmula:

$$\begin{aligned}\phi =&(x_1 \vee \overline{y}_1) \wedge (x_1 \vee \overline{y}_1) \wedge (\overline{x}_1 \vee \overline{x}_2 \vee y_1)\\ &\wedge (\overline{x}_1 \vee y_2) \wedge (\overline{x}_2 \vee y_2) \wedge (x_1 \vee x_2 \vee \overline{y}_2)\\ &\wedge (y_1 \vee y_3) \wedge (\overline{y}_1 \vee \overline{y}_3)\\ &\wedge (y_2 \vee \overline{z}) \wedge (y_3 \vee \overline{z}) \wedge (\overline{y}_2 \vee \overline{y}_3 \vee z)\\ &\wedge z.\end{aligned}$$

Isto converte qualquer circuito C com n portas lógicas para uma fórmula ϕ com $O(n)$ cláusulas, a qual é satisfatível se e somente se C também é. Esta fórmula tem uma mistura de cláusulas de tamanho 1, 2 e 3, mas, como nós mostramos agora, esta pode ser transformada em uma fórmula *3-Sat* adicionando-se variáveis artificiais. Se c é uma cláusula possui um literal, nós podemos convertê-la em um conjunto de cláusulas com três literais da seguinte forma:

$$x \quad \Leftrightarrow \quad (x \vee z_1 \vee z_2) \wedge (x \vee z_1 \vee \overline{z}_2) \wedge (x \vee \overline{z}_1 \vee z_2) \wedge (x \vee \overline{z}_1 \vee \overline{z}_2) \tag{1.9}$$

Se uma cláusula c possui dois literais nós podemos convertê-la no seguinte conjunto de cláusulas com três literais:

$$x \vee y \quad \Leftrightarrow \quad (x \vee y \vee z_1) \wedge (x \vee y \vee \overline{z}_1). \tag{1.10}$$

Isto mostra que *Circuito Sat* pode ser reduzido à *3Sat*, e portanto *3Sat* é $NP-$completo.

Recorde-se que o problema *Sat* consiste em determinar que uma fórmula CN ϕ é satisfatível. Nós acabamos de mostrar que 3-*Sat* é $NP-$completo, mas 3-*Sat* é um caso especial de Sat, logo *Sat* também é $NP-$completo. Por outro lado, 1Sat, que é o problema *Sat* onde cada cláusula tem tamanho 1, pertence à classe P. Será que 2*Sat* pertence à P ou é $NP-$completo? Nós mostraremos agora que 2*Sat* pertence à P, nós faremos isso apresentando um algoritmo polinomial para este problema.

A ideia chave é representar uma fórmula 2*Sat* como um grafo dirigido. Primeiro note que se um literal de uma cláusula 2*Sat* é falso, o outro literal deve ser verdadeiro. Portanto, uma cláusula 2*Sat* é equivalente a um par de implicações:

$$(l_1 \vee l_2) \quad \Leftrightarrow \quad (\bar{l}_1 \rightarrow l_2) \wedge (\bar{l}_2 \rightarrow l_1). \tag{1.11}$$

Suponha que ϕ é a seguinte fórmula:

$$\phi(p,q,r) \quad = \quad (p \vee \overline{q}) \wedge (\overline{p} \vee \overline{q}) \wedge (\overline{p} \vee \overline{r}) \wedge (q \vee r) \wedge (p \vee q). \tag{1.12}$$

Esta fórmula pode ser representada pelo seguinte grafo 1.7:

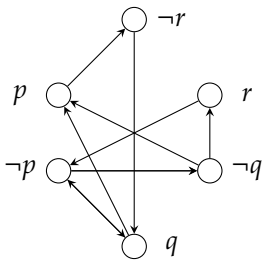


Figura 1.7: Grafo Implicação

Teorema 1.6.2 *Uma fórmula 2-Sat ϕ é satisfatível se e somente se não existir nenhuma variável x com um caminho $x \rightsquigarrow \overline{x}$ e $\overline{x} \rightsquigarrow x$ no grafo implicação de ϕ.*

Utilizando o teorema nós podemos implementar diretamente o algoritmo 1 para o problema 2*Sat* (existem algoritmos de tempo linear para o problema, [4, 7]).

[4]: Aspvall et al. (1979), *A Linear-Time Algorithm for Testing the Truth of Certain Quantified Boolean Formulas*
[7]: Even et al. (1976), *On the Complexity of Timetable and Multicommodity Flow Problems*

Algorithm 1 Polinomial para 2-Sat

```
2Sat(φ)
for ∀x ∈ φ
    if (x ⇝ x̄ ∧ x̄ ⇝ x)
        return FALSE
return TRUE
```

Considere a seguinte fórmula FNC $\phi(p,q,r) = (p \vee \overline{q}) \wedge (\overline{p} \vee \overline{q}) \wedge (\overline{p} \vee \rightsquigarrow \overline{r}) \wedge (q \vee r) \wedge (p \vee q)$. Qual é o resultado de executar o algoritmo 2Sat sobre esta fórmula? O teorema anterior prova que nós podemos reduzir

$$2Sat \leq_K \textit{Alcançabilidade}.$$

Mais precisamente 2-Sat pode ser resolvido resolvendo-se $2n$ instâncias do problema da Alcançabilidade. Uma vez que o problema da Alcançabilidadee pertence à classe P, nós temos que 2-Sat também pertence à P.

Lições aprendidas: Subtipos de uma problema X podem pertencer à classe P enquanto que o problema geral é $NP-$completo.

1.6.4 Projetando Reduções

Nós mostraremos agora que outros problemas são $NP-$completos porque eles são poderosos o bastante para expressar a ideia que variáveis Booleanas são verdadeiras ou falsas, e impor restrições sobre estas variáveis tal como 3-*Sat* faz, e que outros problemas são suficientemente poderosos para expressar a estrutura desses problemas. Muitos desses problemas foram mostrados ser $NP-$completos pelo trabalho seminal de Richard Karp [11].

[11]: Karp (1972), *Reducibility Among Combinatorial Problems*

Quando nós projetamos estas reduções, é importante recordar que uma redução é um mapeamento de instâncias para instâncias. De modo a provar que uma redução é correta, nós também devemos considerar como mapear soluções para soluções. No entanto, estes mapeamentos são parte da nossa análise da redução, não fazendo parte da redução em si. Esta tradução tem de ser confiável, ou seja, uma resposta "sim" ou "não" tem de permanecer a mesma, mas a responsabilidade da redução é transformar a pergunta, não a resposta.

Nós mostraremos, a seguir, que vários problemas diferentes são $NP-$completos, e dessa forma todos são variações de um mesmo tema. Uma das propriedades mais úteis para provar que um problema é o fato de que reduções serem relações transitivas. Por outras palavras, dado três problemas diferentes A, B e C, se $A \leq_K B$, e $B \leq_K C$, então $A \leq_K C$. Por exemplo,

$$\textit{3-Sat} \leq_K \textit{Conjunto Independente} \leq_K \textit{Cobertura por Vértice},$$

nós podemos concluir que *3-Sat* pode ser reduzido em tempo polinomial ao problema *Cobertura por Vértice*.

*Como você podes provar que um determinado problema X é $NP-$*completo*?*

.Algumas provas são simples outras por sua vez demandam um lote de engenho e criatividade. Dessa forma nós não devemos acalentar a ilusão de que existem mecanismos que aplicam de forma indiscriminada a todas as situações. Tudo o que nós oferecer são guias gerais, as quais; por si só, não são suficientes para fornecer uma resposta completa ao problema. Contudo; que estas regras, suplementadas com a prática; podem fornecer uma via segura através da arte de provar que um determinado problema é $NP-$completo. Mas antes de prosseguirmos nós devemos; mais uma vez; realçar a importância de se provar que um problema é $NP-$completo. Como nós já dizemos anteriormente é praticamente consenso que $P \neq NP$; então a descoberta que um determinado problema é $NP-$completo deve ser vista como uma indicação que este problema não pode ser resolvido; de uma forma geral; em tempo polinomial.

A algoritmo básico para mostrar que um determinado problema X é $NP-$completo é a seguinte:

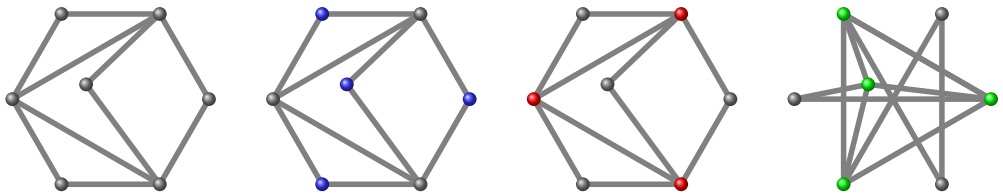
1. Prove que X pertence à classe NP. Este passo normalmente é direto; mas nem sempre.

2. Escolha um problema Y que é conhecido ser $NP-$completo. Esta escolha muitas vezes passeia-se na similaridade entre os dois problemas mas outros aspectos devem também serem considerados; por exemplo; a familiaridade com o problema.
3. Prove que $Y \leq_K X$. Aqui nós devemos dizer uma palavra de cautela; cuidado com a direção; é muito comum se errar a direção da redução; nós queremos provar que X é difícil então nós temos que transformar um problema difícil em X e não o contrário.

Muitas reduções transformam uma instância de Y em uma única instância de X com a mesma resposta. Ou seja nós estamos utilizando; neste curso; apenas reduções de Karp. Dessa forma quando nós estamos tentando provar que a redução funciona corretamente nós temos que provar que instâncias "sim" de Y são mapeadas para instâncias "sim" de X; e que instâncias "sim" de X tem que ser mapeadas para instâncias "sim" de Y; isto corresponde exatamente ao "se e somente se."

1.6.5 Problemas $NP-$completos

Nós agora apresentaremos alguns exemplos de problemas $NP-$completos. Nós iniciamos com alguns problemas de uma área que tem sido extremamente fecunda em problemas $NP-$completos, a Teoria dos Grafos. Nós tentaremos mostrar como esses problemas poderiam ter sido encontrados pelo rei.



(a) Grafo Original (b) Conjunto Independente (c) Cobertura por Vértice (d) Clique

Figura 1.8: Relação entre Conjunto Independente, Cobertura por Vértice, e Clique

A Festa Real

O rei Arthur está planejando dar uma baile para a corte. No entanto, existem algumas rivalidades entre os membros da corte, ou seja alguns pares de membros se odeiam por alguma razão que não interessa. O rei gostaria de convidar tantas pessoas quanto possível, mas ele deseja garantir que ele não convidou ambos os membros de qualquer par de desafetos.

Formalmente, este problema poderia ser modelado da seguinte forma: dado um grafo $G = (V, E)$, será dito que um subconjunto $S \subseteq V$ é independente se qualquer par de vértices em S não é adjacente. Se os vértices representam membros da corte e as arestas as rivalidades, o Rei deseja encontrar o maior conjunto independente possível em grafo das relações sociais da corte.k

Tal como estabelecido este é um problema de otimização. De modo a formulá-lo como um problema de decisão, nós devemos estabelecer um tamanho alvo k como parte da entrada, e perguntar se é possível encontrar um conjunto independente de tamanho pelo menos k.

Problema Conjunto Independente
Entrada: Dado um grafo $G = (V, E)$, e um inteiro k.
Pergunta: G possui um conjunto independente de tamanho k ou maior?

Exemplo 1.6.1 Considere o grafo da figura **??**. Este possui um conjunto independente de tamanho 2 formado pelos vértices d e a ou pelos vértices d e b. Mas não possui um conjunto independente de tamanho 3.

Teorema 1.6.3 *Conjunto Independente é $NP-$completo [11].*

[11]: Karp (1972), *Reducibility Among Combinatorial Problems*

Demonstração. Nós podemos ver que Conjunto Independente pertence à classe NP, uma vez que Merlin pode facilmente convencer o Rei Arthur que um determinado grafo G possui um conjunto independente de tamanho k ou maior, tudo que ele precisa fazer é mostrar ao rei um conjunto independente de tamanho k. O rei pode verificar em tempo polinomial, no número de vértices, se o conjunto é um certificado válido para a instância do problema.

Como nós podemos projetar uma redução de um problema de satisfação de restrições para um problema como *Conjunto Independente*, no qual nós queremos saber se um objeto de determinado tamanho existe? Note que nós podemos ajustar o tamanho do objeto para qualquer valor que seja conveniente. Por exemplo, suponha que nós queremos provar que Conjunto Independente é $NP-$completo reduzindo *3-Sat* a este. Então nós necessitamos mostrar como transformar uma fórmula 3 FNC ϕ em um par (G_ϕ, k) tal que G_ϕ possui um conjunto independente de tamanho k se e somente se ϕ é satisfatível.

Para executar esta redução, nós temos que responder a duas perguntas:

i) Quais são as escolhas que nós temos quando nós tentamos resolver uma instância *3-Sat*?

ii) Quais as restrições que cada instância 3-*Sat* impõe?

Na primeira pergunta você tem que escolher qual literal será ajustado para o valor lógico verdadeiro em cada cláusula. Vamos assumir por um instante que apenas um literal pode ser ajustado para verdadeiro em cada cláusula. Como cada cláusula tem tamanho três é natural pensar em nossa tranqueira escolha como sendo um grafo completo com 3 vértices, ou seja um triângulo. Em um triângulo existem 3 conjuntos independentes diferentes, cada um consistindo de um único vértice. O conjunto independente inclui pelo menos um vértice de cada uma destas tranqueiras, mas nenhum conjunto independente pode incluir mais que

um vértice de cada tranqueira. Então ao traduzirmos uma instância ϕ de *3-Sat* para uma instância G de Conjunto Independente nós devemos formar um triângulo para cada cláusula. Cada nó do triângulo é rotulado com índice da cláusula e o nome da variável, por exemplo se a cláusula c_k tem uma variável x_i então existirá um nó rotulado v_{ki} no triângulo correspondente à clausula c_k.

Mas um problema permanece, como nós podemos garantir que a mesma variável não assume valores contraditórios, em cláusulas diferentes, ou seja como nós podemos garantir que uma variável proposicional x não é ajustada para verdadeiro e falso ao mesmo tempo? O que precisa ser feito é ligar os vértices correspondente aos literais complementares, ou seja se existe um literal x em uma cláusula e um literal $\neg x$ em outra cláusula os dois nós correspondentes são ligados por uma aresta. Isto garante que o conjunto independente não pode incluir os dois vértices.

Agora tudo o que nos resta fazer é ajustar k para o número de cláusulas na fórmula ϕ.

Nós afirmamos que ϕ é satisfatível se e somente se existir um conjunto independente S de tamanho k no grafo G_ϕ.

Assuma que uma atribuição satisfatória existe para ϕ. Escolha um nó em cada triângulo o qual foi ajustado para o valor "verdadeiro" nesta atribuição. O conjunto de nós escolhidos deve ser um conjunto independente. Assim nós temos um conjunto independente de tamanho $k = m$. Na outra direção vamos supor que nós temos um conjunto independente S em G_ϕ tal que $|S| = m$. Se nós ajustarmos o literal correspondente ao vértice escolhido de cada triângulo para o valor lógico "verdadeiro", então, uma vez que nenhum par de literais complementares é escolhido, a atribuição é consistente. Como um literal de cada cláusula é ajustado para "verdadeiro" e assim todas as cláusulas são satisfeitas. □

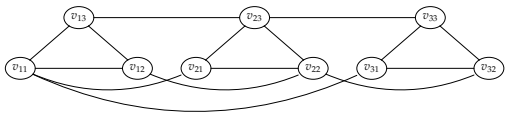


Figura 1.9: Redução 3−Sat para Conjunto Independente

Exemplo 1.6.2 Suponha que a instância *3-Sat* seja

$$\phi = \overbrace{(x_1 \vee x_2 \vee x_3)}^{c_1} \wedge \overbrace{(\neg x_1 \vee \neg x_2 \vee \neg x_3)}^{c_2} \wedge \overbrace{(\neg x_1 \vee x_2 \vee x_3)}^{c_3},$$

então o grafo G_ϕ resultante da redução é mostrado na figura 1.12.

Protegendo a Fortaleza

O rei Arthur construiu uma fortaleza na fronteira mais ao norte do seu reino. Infelizmente existem poucos soldados para guardar a fortaleza. O rei deseja que cada trecho da muralha seja vigiado por pelo menos um soldado. Um soldado posicionado na junção de vários trechos pode vigiar todos os trechos. Por exemplo, um soldado posicionado em a_7 conseguirá vigiar todos os segmentos da muralha da fortaleza mostrado na figura 1.10.

Figura 1.10: Fortaleza estrela

O rei obteve então um mapa do castelo de Winterfell. Mostrado na figura 1.11. Note que as junções são representadas pelos vértices. O rei

Figura 1.11: Fortaleza Winterfell

chama Merlin para ajudá-lo na tarefa de projetar a estratégia de defesa da fortaleza.

Nós podemos generalizar o problema da seguinte forma:

Problema Protegendo a Fortaleza[a]
Entrada: Dada uma fortaleza, representada como um grafo G, e um certo número k de soldados.
Pergunta: É possível proteger a fortaleza com k soldados?

[a] *Este problema é conhecido, na literatura, como problema da Cobertura por Vértice.*

Teorema 1.6.4 *O problema Protegendo a Fortaleza é $NP-$completo [11] (Karp mostrou que Cobertura por Vértice é $NP-$completo.)*

[11]: Karp (1972), *Reducibility Among Combinatorial Problems*

Demonstração. Claramente *Protegendo a Fortaleza* está em NP. O certificado que Merlin entrega ao rei nada mais é que um conjunto S de k localizações onde os soldados devem ser posicionados. O rei deve verificar que cada trecho da muralha é coberta por pelo menos um soldado, em termos de grafos isto significa dizer que cada aresta tem pelo menos uma das suas extremidades no conjunto S. O rei pode verificar isto em tempo polinomial.

A seguir nós devemos reduzir um problema que nós já sabemos ser $NP-$completo ao problema *Protegendo a Fortaleza*. Uma escolha natural é o problema do Conjunto Independente, ou seja nós reduziremos *Conjunto Independente* à *Protegendo a Fortaleza*.

Assuma que $G = ((V, E), k)$ é uma instância do problema *Conjunto Independente*. A correspondente instância do problema *Protegendo a Fortaleza* é $G = ((V, E), |V| - k)$. A observação crucial é que o complemento de um conjunto independente é uma cobertura por vértice e vice-versa.

Se S é um conjunto independente de tamanho k, então $V - S$ é de tamanho $n - k$. Considere agora uma aresta arbitrária $(u, v) \in E$. Se S é um conjunto independente isso implica que $u \notin S$ ou $v \notin S$, ou ambos não pertencem. Assim, $V - S$ é uma cobertura por vértice.

Na direção contrária, se $V-S$ é uma cobertura por vértice de tamanho $n-k$, então S é de tamanho k. Considere uma aresta arbitrária $(u, v) \in E$. Uma vez que $V - S$ é uma cobertura por vértice nós temos que $v \in (V - S)$, ou $u \in (V - S)$, ou ambos os vértices pertencem ao conjunto $V - S$. Assim, S um conjunto independente. Os dois dois problemas: *Conjunto Independente* e *Cobertura por Vértices* são equivalentes, o que nós representamos como *Conjunto Independente* $\equiv$ *Cobertura por Vértices*. Portanto, *Cobertura por Vértice* é $NP-$completo. □

Clique

O rei Arthur ficou muito tão impressionado com o filme "300"que ele resolveu formar uma legião chamada "300 de Camelot". Como o filme realçou, uma parcela considerável do êxito dos espartanos em combate devia-se ao fato de que cada espartano confiava em todos os seus outros

companheiros. Dessa forma o rei deseja que formar uma legião onde todos confiem um nos outros. Note que relações de confiança são simétricas, porém não transitivas. Por outras palavras, o rei deseja saber se o grafo das relações existe um clique de tamanho 300.

Um clique C em um grafo $G = (V, E)$ é um subgrafo completo, ou seja todo vértice em C é adjacente a todos os outros vértices de C.

Problema CONJUNTO INDEPENDENTE
Entrada: Dado um grafo $G = (V, E)$, e um inteiro k.
Pergunta: G possui um clique de tamanho k?

Teorema 1.6.5 *Clique é $NP-$completo.*

Demonstração. Clique claramente está em NP. Nós podemos reduzir CONJUNTO INDEPENDENTE à CLIQUE. Assuma que $(G = (V, E), k)$ denota uma instância do problema conjunto independente. A correspondente instância do problema Clique é $(G^c = (V, E^c), k)$. Onde G^c é o grafo complementar de G, ou seja G^cpossui os mesmos vértices, contudo uma aresta pertence à E^c se e somente se ela não pertence à E. A observação crucial é que qualquer conjunto independente em G corresponde a um clique do mesmo tamanho em G^c e vice-versa. Como Conjunto Independente é $NP-$completo então Clique é $NP-$hard. E nós podemos concluir que Clique é $NP-$completo. □

Uma outra forma de provar que Clique é $NP-$hard é reduzir um outro problema $NP-$completo a ele. Um dos problemas mais utilizados em

Teorema 1.6.6 *3-SAT $\leq_P$ CLIQUE.*

Demonstração. Nós reduziremos 3Sat à Clique. Assuma que $\phi = C_1 \wedge C_2 \wedge \ldots \wedge C_m$ é uma instância de 3Sat. Nós construiremos uma instância (G_ϕ, k) do problema clique que possui um clique de tamanho k se e somente se ϕ é satisfatível. A construção é a seguinte:

Faça $k = m$, e o grafo $G_\phi = (V, E)$, onde

$$V = \{v_{ik} \mid 1 \leq i \leq 3, 1 \leq k \leq m\},$$

isto é, existe um vértice para cada ocorrência de um literal em ϕ;

$$E = \{(v_{ij}, v_{rs}) \mid \text{para todo} v_{ij}, v_{rs} \in V \text{ tal que } i \neq r \text{ e } l_{ij} \neq \neg l_{rs}\},$$

isto é, as arestas conectam vértices correspondentes a literais em cláusulas diferentes e que não são complementares.

Note que a construção de G_ϕ pode ser executa em tempo polinomial com relação ao tamanho da fórmula ϕ.

A figura 1.12 mostra um exemplo de construção do grafo G_ϕ correspondente à fórmula $\phi = (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \neg x_2 \vee \neg x_3) \wedge (x_1 \vee x_2 \vee x_3)$.

Nós agora temos que mostrar que redução é correta, ou seja ϕ é satisfatível se e somente se G_ϕ é uma instância "sim". reduções deste tipo é o problema 3Sat.

Vamos provar agora que se G_ϕ possui um clique de tamanho k então a fórmula é satisfatível. A única forma de possuir G_ϕ um clique de tamanho k é se um vértice v_{ij} correspondente a cada uma das cláusulas fizer parte do clique, uma vez que não existem arestas entre os vértices correspondentes aos literais de uma mesma cláusula. Agora nós podemos ajustar o valor lógico do literal correspondente a cada um dos vértices v_{ij}, que fazem parte do clique, para o valor lógico verdadeiro. A construção também garante que nenhuma variável proposicional possui, ao mesmo tempo, o valor verdadeiro e falso. Portanto, toda cláusula possui pelo menos um literal cujo valor lógico é verdadeiro, e dessa forma toda cláusula é satisfeita.

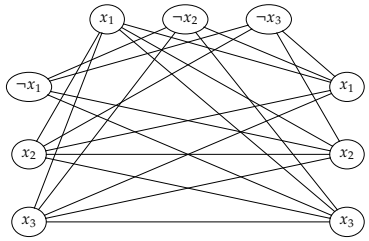


Figura 1.12: Exemplo redução de 3Sat à Clique

Na outra direção nós vamos assumir que a fórmula ϕ é satisfatível. Assuma que $\Gamma : \{x_1, \ldots, x_n\} \rightarrow \{verdadeiro, falso\}$ é uma atribuição lógica que satisfaz ϕ. Dessa forma, pelo menos um literal de cada cláusula possui o valor Booleano "verdadeiro", e uma variável proposicional não pode ser falsa e verdadeira ao mesmo tempo. Escolha um literal verdadeiro de cada cláusula, e inclua o vértice correspondente ao literal em um conjunto C. Note que C tem tamanho k. E existe uma aresta entre qualquer par de vértices de C, afinal de contas eles correspondem a literais em cláusulas diferentes que não são complementares. Portanto, C é um clique de tamanho k. □

Cobertura por Conjunto

Problema Cobertura por Conjunto
Entrada: Um conjunto U de elementos, uma coleção S de subconjuntos de U, e um inteiro positivo k.
Pergunta: Existe um subconjunto S' de S, tal que $|S'| \leq k$, e cuja união é igual ao conjunto U?

Exemplo 1.6.3 Nós temos m procedimentos, e um conjunto U de capacidades que nós gostaríamos que nosso sistema possuísse. O $i-$ésimo procedimento fornece um conjunto $S_i \subseteq U$ de capacidades. O objetivo é alcançar todas as n capacidades utilizando no máximo

$k = 2$ procedimentos.

$$\begin{aligned} U &= \{1,2,3,4,5,6,7\} \\ S &= \{\overbrace{\{3,7\}}^{S_a},\overbrace{\{2,4\}}^{S_b},\overbrace{\{3,4,5,6\}}^{S_c},\overbrace{\{5\}}^{S_d},\overbrace{\{1\}}^{S_e},\overbrace{\{1,2,6,7\}}^{S_f}\} \end{aligned} \quad (1.13)$$

Nós temos que o conjunto $S' = \{S_c, S_f\}$ é uma cobertura por conjunto de tamanho 2.

Teorema 1.6.7 *O problema* Cobertura por Conjunto *é $NP-$completo.*

Demonstração. Claramente Cobertura por Conjunto pertence à classe NP, o certificado que Merlin entrega ao rei é apenas conjunto de subconjuntos. Arthur, por sua vez, pode verificar a veracidade do certificado em tempo polinomial. O que nós precisamos provar é que Cobertura por Cobertura por Conjunto é $NP-$hard. Isto será feito reduzindo-se um problema que nós já sabemos ser $NP-$completo à Cobertura por Conjunto. Um candidato natural é o problema *Cobertura por Vértice*, na verdade Cobertura por Conjunto é uma generalização do problema Cobertura por Vértice.

Dado uma instância $\{G = (V, E), k\}$ do problema Cobertura por Vértice, nós construímos uma instância de Cobertura por Conjunto (U, S, k), o qual possui uma cobertura por conjunto de tamanho k se e somente se G possui uma cobertura por vértice de tamanho k.

A construção é a seguinte: o universo de elementos a serem cobertos U é igual ao conjunto de arestas E. A coleção de subconjuntos incluirá um subconjunto para cada nó $v \in V$, de modo que S_v é o conjunto de arestas que são incidentes no vértice v, ou seja $S_v = \{l \in E \mid$ a aresta l é incidente em $v\}$. Por exemplo, o grafo é transformado na instância do problema Cobertura por Conjunto mostrado em 1.13.

Agora nós precisamos mostrar que a redução é correta. Suponha que $X \subseteq V$ é uma cobertura por vértice de tamanho k em G; então $Y = \{S_v \mid v \in V\}$ é uma por conjunto de tamanho k. Dessa forma instâncias "sim" de Cobertura por Vértice são resolvidas corretamente.

Na outra direção, se $Y \subseteq S$ é uma cobertura por conjunto de tamanho k em (U, S, k); então $X = \{v \mid S_v \in Y\}$ é, claramente, uma cobertura por vértice de tamanho k em G. □

Pintando e Colorindo

O rei Arthur comprou um mapa para colorir, mostrado na Figura **??**, que ele deu de de presente para o seu filho. Então, ele pediu que o jovem príncipe pintasse cada região *vizinha* com uma cor diferente. O príncipe deseja saber quantos lápis de cor ele também ganhará. Lápis de cor são muito caros, na verdade cada um vale uma pequena fortuna, dessa forma o rei gostaria de comprar o menor número possível de lápis de cor.

O rei tem agora o seguinte problema:

Problema $k-$Coloração de Mapas
Entrada: Um mapa para colorir M.
Pergunta: É possível pintar o mapa M com k cores, de modo que regiões vizinhas tenham cores diferentes?

Este problema é equivalente a um famoso problema em grafos. Dado um grafo G uma $k-$coloração é uma atribuição de uma das k cores diferentes para cada vértice, e uma coloração é apropriada se nenhum par de vértices adjacentes tem a mesma cor, como mostrado na figura 1.13.

Problema $k-$Coloração de Grafo
Entrada: Um grafo G.
Pergunta: Existe uma $k-$coloração apropriada de G?

O problema $k-$Coloração de Grafo está em NP uma vez que é fácil para o Rei Arthur verificar que uma coloração é apropriada. Nós mostraremos agora que $k-$Coloração de Grafo é $NP-$completo para $k \geq 3$.

Projetar reduções é em certos aspectos semelhante a construir um computador. Computadores são construídos de componentes físicos que transportam as variáveis de um lugar para outro, e que processa essas variáveis impondo-lhes restrições quando elas se encontram. Similarmente, uma redução típica consiste em construir dois tipos de tranqueiras (gadgets): a tranqueira "escolha" que representa o ajuste de uma variável para um dos seus valores possíveis, e a tranqueira "restrição" que força duas ou mais variáveis a obedecer uma determinada restrição.

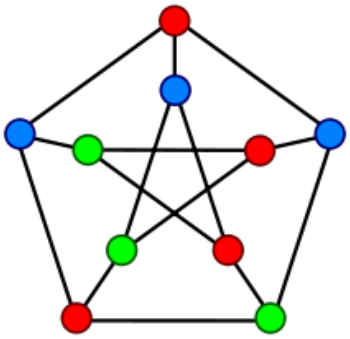


Figura 1.13: Grafo de Petersen com uma 3-coloração apropriada

Vamos agora tentar reduzir 3-*Sat* à $k-$*Coloração de Grafo*. Neste caso, nossa tranqueira escolha necessita representar o ajuste de uma variável Booleana para verdadeiro ou falso. Considere, por exemplo, uma variável proposicional x, nós temos que *escolher* se x será ajustado para o valor verdadeiro ou para o valor falso. Vamos assumir, de forma arbitrária, que a cor azul representa o valor lógico verdadeiro, e a cor vermelha o valor lógico falso. Nós temos que garantir que x será pintado de azul ou vermelho; nós também temos que garantir que se x é pintado de azul então o seu complemento, $\neg x$, não pode ser pintado de azul, uma vez que uma variável deve ser falsa ou verdadeira. Como nós podemos fazer isso? A ideia é ter, para cada variável x, uma tranqueira (gadget) consistindo de um par de vértices, os quais estão conectados um ao ao outro e a um terceiro vértice, como mostrado na figura 1.14, para duas variáveis x_1 e x_2. Note que este terceiro vértice pode ser compartilhado entre todas as tranqueiras. Em qualquer coloração, o vértice central terá alguma cor, e por simetria nós podemos assumir que está é verde. Então em cada tranqueira, um vértice deve ser branco e o outro preto.

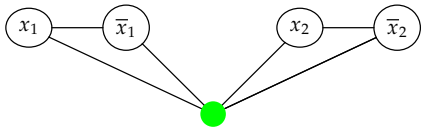


Figura 1.14: Tranqueira escolha - 3−Coloração

Nossa meta é definir uma tranqueira restrição. Esta deveria ser um grafo pequeno ao qual nós podemos conectar as três tranqueiras escolhas, o qual só pode ser pintado com as três cores se e somente se pelo menos um dos literais na correspondente cláusula é ajustado para verdadeiro. Se nós conectamos a tranqueira cláusula aos vértices da tranqueira escolha correspondendo à x ou $\overline{x}$, dependendo se x está negado na cláusula, nós gostaríamos que a tranqueira só poderia ser colorida de forma apropriada se e somente se pelo menos um dos seus vértices deve ser preto. Como poderia ser esta tranqueira restrição? Nesse ponto nós temos que apenar para a criatividade, como nós dissemos projetar reduções é muito semelhante a projetar algoritmos, e os dois são exemplos de arte. A tranqueira restrição para a cláusula $(x_i \vee \neg x_j \vee x_k)$ é mostrada na figura 1.15.

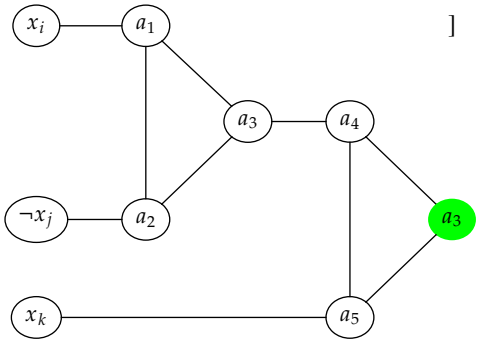


Figura 1.15: Tranqueira restrição - 3-Coloração

Se os literais x_i, $\neg x_j$ e x_k têm os valor Booleanos "verdadeiro" ou "falso", então a tranqueira restrição 1.15 pode ser pintada com três cores se e somente se pelo menos um dos literais x_i, $\neg x_j$ e x_k tem o valor verdadeiro, ou seja pintado de azul. Vamos considerar o caso que todos os três vértices são pintados de vermelho, então nós somos forçados ao esquema de cores mostrado na

Figura 1.16. Mas agora nós precisamos colorir os vértices a_1, a_2 e a_3 de cores diferentes, mas nós não conseguimos fazer isto.

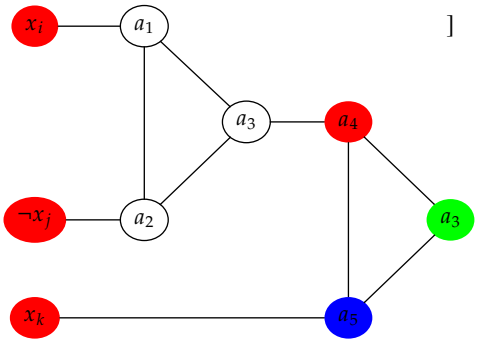


Figura 1.16: Tranqueira restrição - três literais falsos

Vamos considerar a situação em que x_i é ajustado para o valor verdadeiro e as dois outros literais para falso, ou seja x_i é pintado de azul e x_j e x_k são pintados de vermelho. Nesta situação a tranqueira pode ser apropriadamente colorida. O restante da prova consiste em mostrar que as outras 6 atribuições de formas de pintar os vértices também permitem que todos vértices sejam apropriadamente pintados.

Gerando um grafo com uma tranqueira escolha para cada variável e uma tranqueira restrição para cada cláusula como descrito anteriormente nós podemos converter qualquer fórmula *NAE-3-Sat* ϕ com n varáveis e m cláusulas em um grafo G com $2n + 3m + 1$ vértices, que pode ser colorido com três cores se e somente se ϕ é satisfatível. Esta transformação pode ser executada em tempo polinomial. Isto completa a prova que

$$\textit{3-Sat} \leq_K \textit{3-Coloração de Grafo},$$

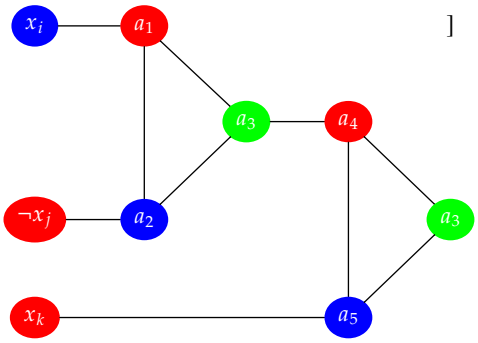


Figura 1.17: Tranqueira restrição - um literal verdadeiro

e que portanto *3-Coloração de Grafo* é $NP-$completo.

Note que a redução não colore o grafo, ou satisfaz a fórmula. Esta simplesmente converte fórmulas para grafos, transformando uma pergunta em satisfabilidade lógica em uma a respeito da coloração de grafo.

Exemplo 1.6.4 Vamos mostrar um exemplo da redução de uma instância de 3-Sat a uma instância do problema de 3−Coloração de grafos. Assuma que $\phi = (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \neg x_2 \vee \neg x_3) \wedge (x_1 \vee x_2 \vee x_3)$, então a correspondente instância de *3-Coloração de Grafo* é mostrada na Figura 1.18.

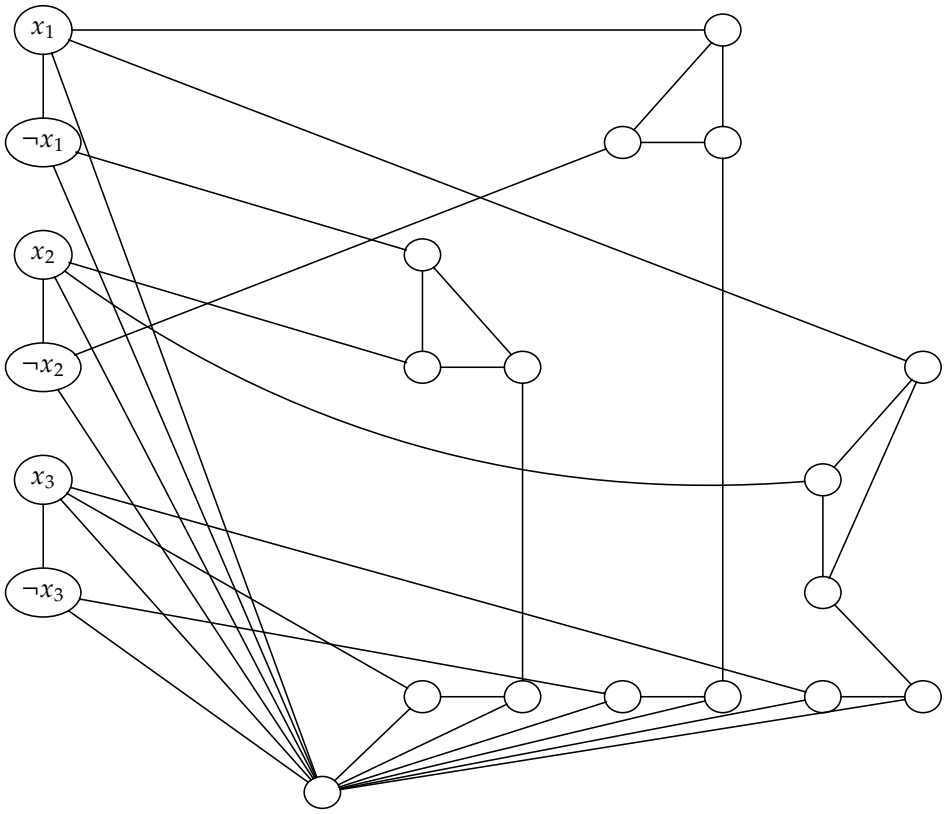


Figura 1.18: A redução de 3Sat para 3-Coloração

O que isto nós diz a respeito do problema $k-$Coloração de Grafos para outros valores de k? Prove que $k-$Coloração de Grafos $\leq$ $(k+1)-$*Coloração de Grafos* para qualquer k. Observação: adicione um vértice. O exercício anterior mostra que $k-$*Coloração de Grafos* é $NP-$completo para qualquer $k \geq 3$. O Teorema das Quatro Cores[1, 2] implica que $k-$*Coloração de Grafos Planares* está em P para $k \geq 4$.

[1]: Appel et al. (1977), *Every planar map is four colorable. Part I: Discharging*
[2]: Appel et al. (1977), *Every planar map is four colorable. Part II: Reducibility*

O Quebra-cabeça Real

O Rei Arthur está entediado, não existe nenhuma guerra que ele precise lutar, nenhum casamento para celebrar, nenhuma festa para organizar,

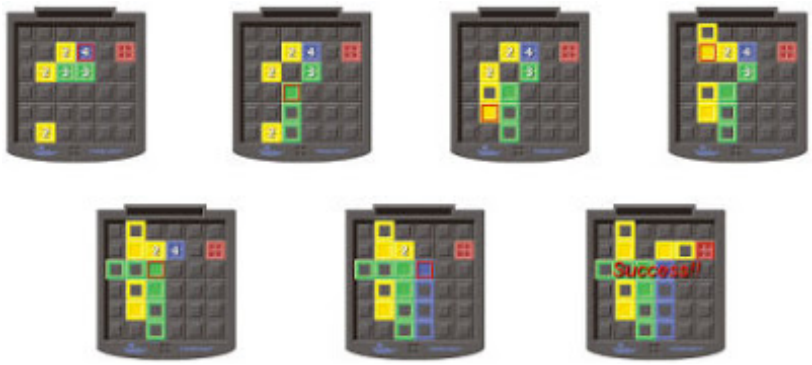


Figura 1.19: Instância do quebra-cabeça *TipOver* e sua solução

nada. Sabendo disso Merlin projetou um quebra-cabeça que ele acredita possa entreter o Rei.Merlin resolveu chamar o seu quebra-cabeça de *TipOver*.

O quebra-cabeça começa em uma configuração com vários engradados verticais de diversas alturas ($1\times1\times h$) arranjados em uma malha, conforme mostrado na figura 1.19, o desafio consiste em mover um saltador para o engradado vermelho. O saltador pode derrubar qualquer engradado vertical sobre o qual ele está, em qualquer uma das quatro direções, desde que exista espaço vazio bastante para àquele engradado cair sem ser obstruído. O saltador é ágil o bastante para pousar de forma segura sobre o engradado recém derrubado. O saltador pode também caminhar ou escalar o topo de qualquer engradado, mesmo quando eles têm alturas diferentes. No entanto, o saltador não pode pular para espaços vazios de modo a alcançar outro engradado, o saltador também não pode pular diagonalmente - os engradados devem se tocar.

O Rei não parece muito impressionado, afinal de contas o quebra-cabeça parece muito fácil. Merlin pede ao Rei para tentar resolver uma configuração. Depois de algumas horas de tentativas frustradas. Merlin mostra ao Rei como resolver a instância do quebra-cabeça *TipOver*.

Neste ponto nós nos despedimos, momentaneamente, do Rei e do mago Merlin, e voltamos ao assunto do nosso curso. A primeira coisa a ser observada é que o número de movimentos feito pelo saltador nunca pode ser maior que o número de engradados, uma vez que um engradado derrubado permanecerá deitado para sempre. **??**. Portanto, o tamanho da solução é polinomial no tamanho da instância do quebra-cabeça. Merlin pode facilmente convencer o Rei Arthur de que uma determinada instância do problema é uma instância "sim". Considere, por exemplo, a instância mostrada mais à direita na figura **??**. Merlin pode mostrar a sequência de movimentos da figura. A localização do saltador é indicado por um contorno vermelho, e a altura de cada engradado é representado por um número. Em cada passo sucessivo, exatamente um engradado foi derrubado.

O fato de o rei ter levado tanto tempo para resolver uma instância do quebra-cabeça pode ter feito você depreciar a inteligência de Arthur. Contudo, antes de ser tão rigoroso no seu julgamento, permita-me dizer que Robet A. Hearn mostrou que *TipOver* é $NP-$completo[10]*. Ele fez isto mostrando como *Sat* pode ser reduzido à *TipOver*, ou seja

[10]: Hearn (2006), *Games, puzzles, and computation*

* Nós seguimos em grande parte a apresentação de Hearn.

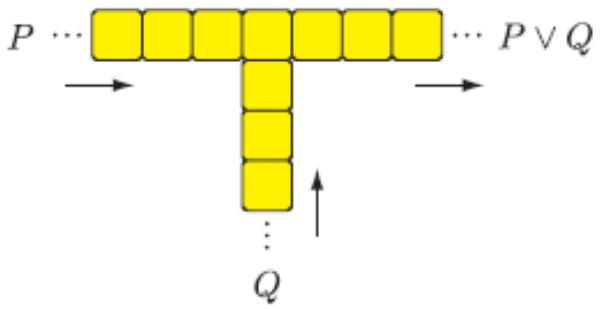


Figura 1.20: Tranqueira Or - TipOver

$$Sat \leq_K TipOver.$$

A redução consiste em várias tranqueiras mostrando como variáveis, conectivos lógicos And e Or podem ser expressos no quebra-cabeça. É importante mostrar também como propagar os sinais e conectar as várias tranqueiras juntas na estrutura do quebra-cabeça. Finalmente, Hear também mostrou um forma de cruzar os fios, os quais conduzem o valor lógico das variáveis, de modo conectar variáveis as portas lógicas. !!!!!!!!!!!! Talvez vala a pena restringir o problema a Planar Sat. Pensar

Os sinais fluindo do uma tranqueira para a próxima são representados no quebra-cabeça pela alcançabilidade: a ideia é que o saltador é capaz de alcançar certas regiões somente quando a correspondente subexpressão da fórmula é verdadeira. Para resolver o quebra-cabeça, o saltador deve utilizar uma atribuição que satisfaz todas as cláusulas da fórmula, permitindo-lhe alcançar uma determinada posição alvo.

A primeira preocupação é como propagar o valor de uma variável lógica? Como Hearn diz a solução é simples: utilize uma sucessão de engradados, colocados em linha um adjacente ao outro de modo que o saltador possa caminhar ao longo do topo deles. Engradados de altura dois serão suficientes para este objetivo. Na verdade, engradados de altura dois serão os únicos tipos de engradados utilizados. Uma vez que se o quebra-cabeça TipOver for $NP-$completo para este tipo particular de engradado ele também será $NP-$completo para o quebra-cabeça geral.

A tranqueira que representa um conectivo Or deve ser tal que o saltador pode sair da tranqueira se ele entrar em qualquer uma de duas direções. O que faz a tranqueira mostrada na figura 1.20? Funciona como um conectivo lógico Or, uma vez que o saltador poderá prosseguir para a saída se ele chegar pelo fio Q ou se ele chegar pelo fio P. Existe, contudo, um problema. O saltador poderá caminhar da saída para a entrada, o que nós não queremos. A solução encontrada Hear foi criar uma via de mão única, a qual só poderá ser percorrida, a primeira vez, se o saltador vier no sentido correto. A figura 1.21 mostra um fio de mão única.

Agora nós podemos construir uma tranqueira Or protegida por fios de mão única, de modo que o saltador terá que percorrê-la no sentido correto. A figura 1.22 mostra esta construção.

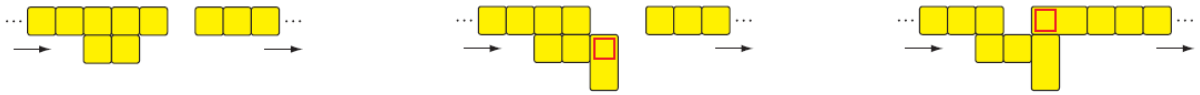


Figura 1.21: Fio de mão única no TipOver

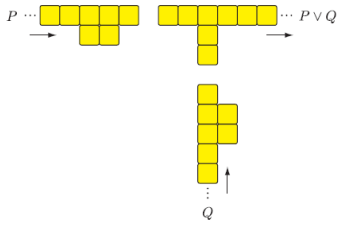


Figura 1.22: Tranqueira Or com entrada protegida

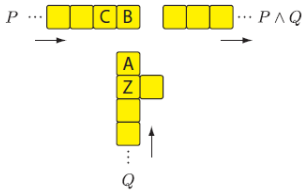


Figura 1.23: Tranqueira And

Agora nós devemos mostrar como construir a tranqueira And. A solução de Hear é mostrada na figura 1.23. Note que o saltador deve conseguir sair pela direita se e somente se ele pode independentemente entrar pela esquerda e por baixo. A primeira coisa a ser observada que existe um único engradado que permitirá ao saltador alcançar o lado direito, o engradado rotulado C, nenhum outro permitirá ao saltador chegar ao lado direito. Note que o saltador deve ser capaz de livremente transitar, mesmo precisando caminhar muito, entre os lados P e Q. Suponha que ele chegou a tranqueira pelo caminho rotulado Q, ele pode derrubar o engradado A para a direita, então ele deve retroceder e dar a volta e alcançar a entrada rotula P, agora ele derruba o engradado B, ele novamente dá a volta atinge a entrada P e, voilà, derruba o engradado C no sentido da saída, permitindo-lhe chegar a saída. A maneira correta de utiliza a tranqueira And é mostrada na figura 1.24.

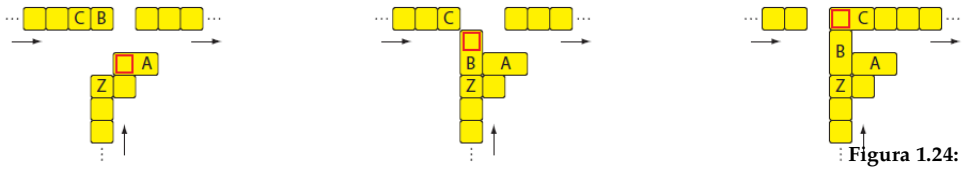


Figura 1.24: Passos para utilizar a tranqueira And

Como o resolvedor do quebra-cabeça decide ajustar uma determinada variável x para o valor lógico verdadeiro ou falso? É claro, dado a natureza do quebra-cabeça, que isto significa fazer o saltador ir em uma direção ou em outra. A tranqueira mostrada na figura 1.25 representa a construção que permite esta escolha. Note que ao derrubar o engradado A em um sentido ou no outro o usuário está escolhendo o valor da variável x. A parte de baixo permite que o saltador retorne para explorar outras partes do quebra-cabeça, mas não permite que o saltador altere novamente o valor lógico da variável, uma vez que o engradado A já foi derrubado em uma direção, e engradados derrubados nunca mais voltam a ficar de pé.

Suponha que nós tenhamos a seguinte instância $\phi = (x \vee \neg x \vee y) \wedge (\neg y \vee z \vee w) \wedge (\neg w)$ do problema *Sat* a figura 1.26 mostra como esta instância pode ser reduzida a uma instância do quebra-cabeça *TipOver*.

Dessa forma qualquer instância de Sat pode ser reduzida em tempo polinomial a uma instância do quebra-cabeça. Portanto, *TipOver* é $NP-$completo.

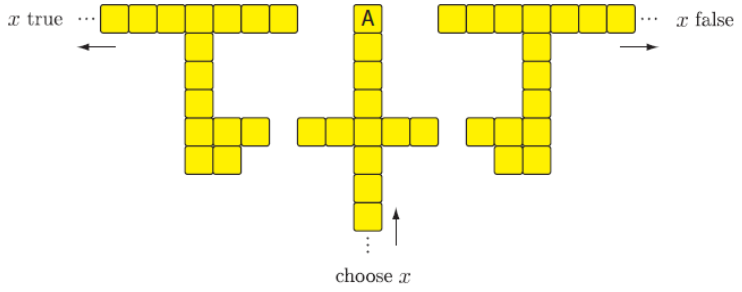


Figura 1.25: Tranqueira escolha

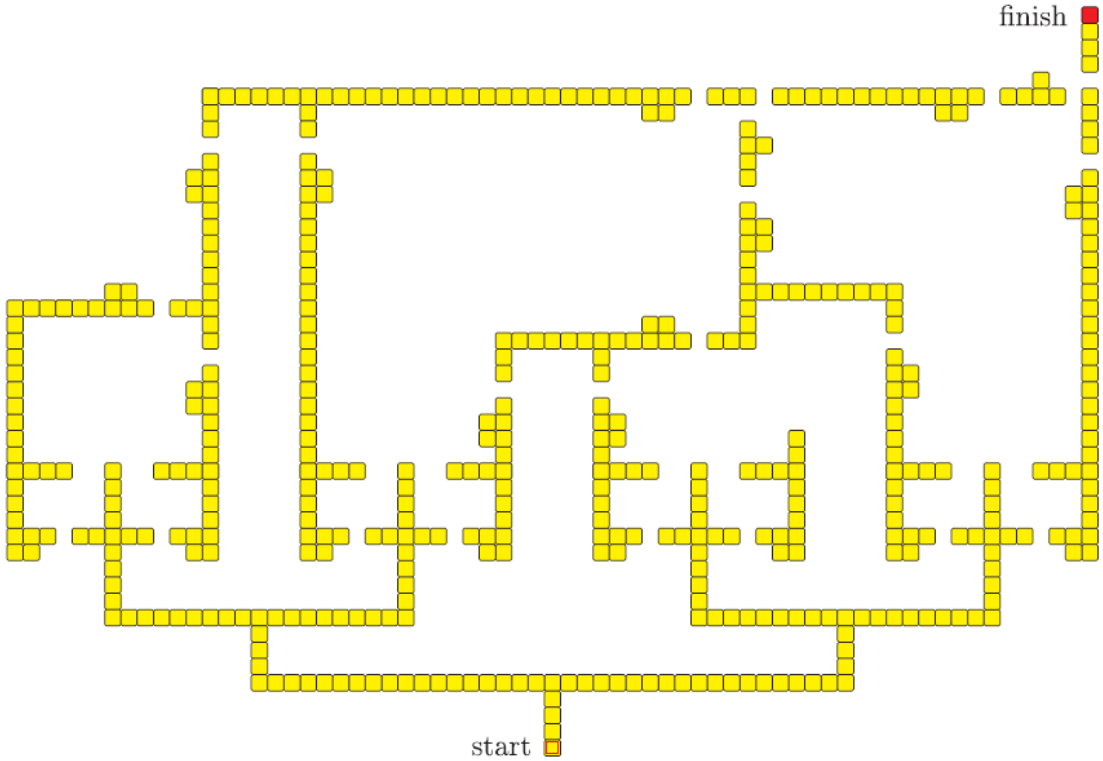


Figura 1.26: Redução da fórmula $\phi = (x \vee \neg x \vee y) \wedge (\neg y \vee z \vee w) \wedge (\neg w)$ a uma instância de *TipOver*

Percorrendo o Reino

Um belo dia Rei Arthur resolveu visitar todos os ducados do seu reino. Sabedor, contudo, que os senhores feudais são muito ciumentos e que se ele visitasse um ducado mais do que outro isto poderia levar a um conflito entre os feudos, o Rei resolveu visitar cada ducado exatamente uma vez. Ele pediu que seu bobo da corte lhe trouxesse um mapa do reino. Ao ver o mapa, mais uma vez, Arthur percebeu a situação delicada em que havia se metido.

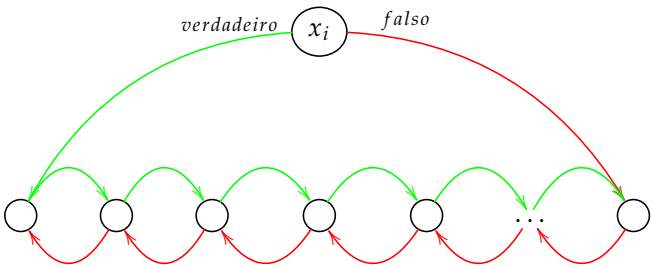


Figura 1.27: Tranqueira escolha

Problema Percorrendo o Reino[a]
Entrada: Entrada um grafo dirigido $G = \{V, E\}$.
Pergunta: Existe um caminho em G que passa por todo vértice (ducado) exatamente uma vez ?

[a] *Na literatura este problema é chamado de problema do caminho Hamiltoniano.*

Tal caminho, se este existe, será chamado de Caminho Hamiltoniano Dirigido. Nós mostraremos que o problema *Percorrendo o Reino* é $NP-$completo. Nós faremos isto reduzindo o problema *Sat* ao problema *Percorrendo o Reino*.

Inicialmente nós temos que mostrar que *Percorrendo o Reino* pertence à classe NP. Como nós dizemos anteriormente um problema pertence à classe NP se quando a instância é uma instância "sim" existe um certificado curto que Merlin pode entregar ao Rei e que este possa verificar em tempo polinomial. O certificado é apenas uma permutação dos ducados.

Ao projetar a redução nós temos que mostrar:

- Como as escolhas do valor lógico de cada variável proposicional podem ser expressas na instância do problema do caminha Hamiltoniano.
- Como expressar a restrição de que toda cláusula deve ter pelo menos um literal ajustado para o valor lógico que a satisfaz.

A escolha que nós temos no problema do caminho Hamiltoniano é como nós percorreremos o grafo. Ao percorrermos uma parte do grafo de uma determinada forma nós estamos fixando o valor lógico de uma variável proposicional para um dos dois valores possíveis. Para cada variável x_i, nós temos uma tranqueira escolha, mostrada na Figura 1.27, consistindo de uma cadeia de t vértices, onde t é pelo menos igual ao número de cláusulas em que x aparece. Nós podemos atravessar esta cadeia da esquerda para à direita ou no sentido contrário, e a escolha em qual sentido a cadeia será percorrida corresponde a ajustar x_i para verdadeiro ou x_i para falso.

Suponha que uma fórmula proposicional ϕ é construída a partir de três variáveis proposicionais x_1, x_2 e x_3. Nós precisamos amarrar as três tranqueiras escolhas juntas, isto pode ser feito como mostrado na Figura 1.28, onde as arestas azuis conectam as três tranqueiras escolhas.

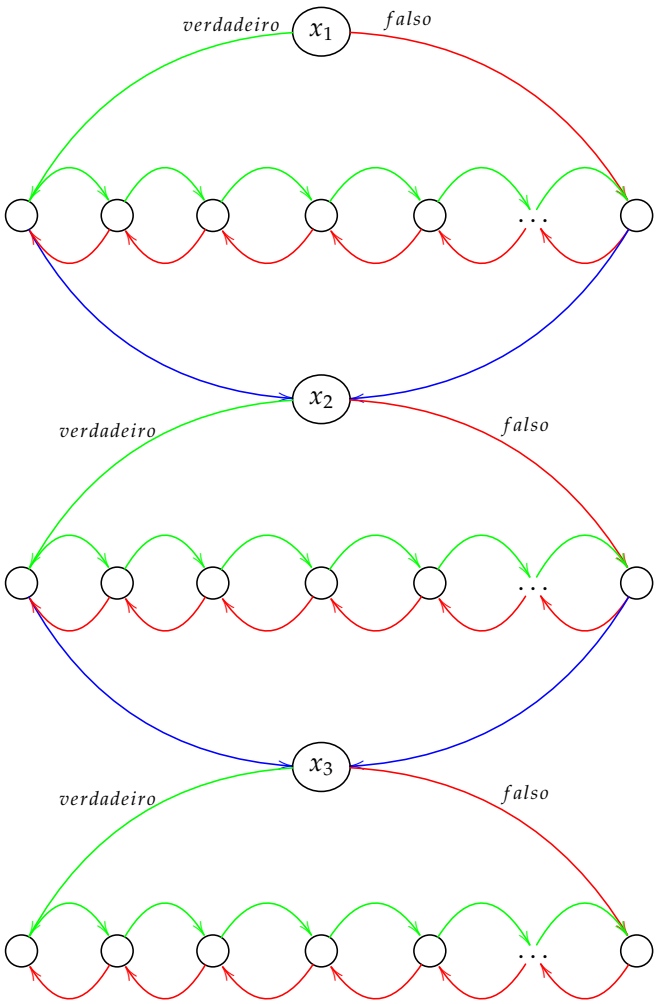


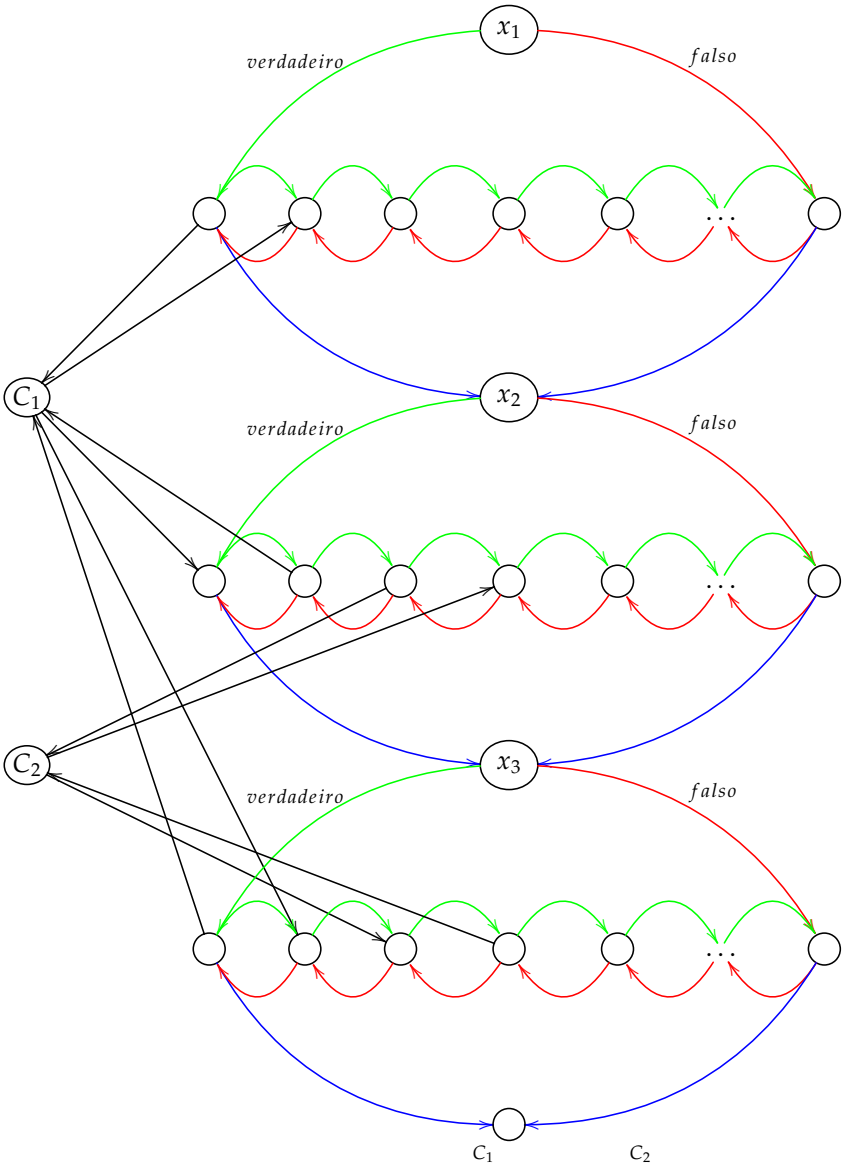
Figura 1.28: Amarrando as variáveis juntas

Para cada cláusula, a tranqueira restrição consiste de um vértice único. Para visitar este vértice nós acrescentamos uma aresta da tranqueira escolha para este vértice e uma aresta no sentido contrário, estas arestas são orientadas de modo a que nós possamos percorrer os vértices, sem repeti-los, se nós caminhos em um sentido que satisfaz à cláusula.

Uma vez que todos os vértices cláusulas tem que ser visitados em algum momento, pelo menos uma dos seus literais tem que ter o valor correto. Assim existe um caminho Hamiltoniano que se move do vértice do topo para o vértice no fundo visitando cada vértice do grafo exatamente uma vez, se e somente se a fórmula *SAT* é satisfatível. A Figura 1.29 mostra a redução para a fórmula $\phi = \overbrace{(x_1 \vee \neg x_2 \vee x_3)}^{C_1} \wedge \overbrace{(x_2 \vee \neg x_3)}^{C_2}$.

Isto mostra que o problema *Percorrendo o Reino* é $NP-$completo. Adicionando-se uma aresta do vértice do fundo para vértice do topo transforma o caminho em um ciclo, provando que o problema do *Ciclo Hamiltoniano* é $NP-$completo.

Finalmente, nós consideramos o problema pelo qual todos os alunos



O grafo resultante da redução de $\phi = \overbrace{(x_1 \vee \neg x_2 \vee x_3)}^{C_1} \wedge \overbrace{(x_2 \vee \neg x_3)}^{C_2}$

Figura 1.29: Um exemplo redução

estavam esperando o *Problema do Caixeiro Viajante*. A versão de otimização do problema pergunta por um caminho através de um conjunto de cidades que minimizando a distância total. Existem várias variantes do problema do Caixeiro Viajante, dependendo das propriedades dos grafos, da função de distância e do tipo de caminho desejado. Nós consideraremos aqui a versão de Garey e Johnson [8].

[8]: Garey et al. (1979), *Computers and Intractability: A Guide to the Theory of NP-Completeness (Series of Books in the Mathematical Sciences)*

Problema do Caixeiro Viajante
Entrada: Um grafo dirigido $G = \{V, E\}$ com arestas com pesos não negativos, e um número $k \geq 0$.
Pergunta: Existe um ciclo em G que passa por todo vértice exatamente uma vez e tem peso total de no máximo k?

O problema do *Caixeiro Viajante* está em NP uma vez que os ciclos são curtos e eles podem ser verificados em tempo polinomial. A verificação consiste basicamente em saber se o tamanho do ciclo é n, e se existe um aresta entre um vértice v no ciclo e o vértice seguinte u, onde u ainda não foi visitado. O peso total do ciclo é simplesmente é soma dos pesos das arestas do ciclo.

Caixeiro Viajante é NP-hard, uma vez que existe uma redução direta de *Ciclo Hamiltoniano* para ele: dê a todas as arestas peso unitário é pergunte por um ciclo de peso n.

Quem sabe o que aparecerá aqui.

Soma Subconjunto

Agora nós iremos considerar um problema pertence ao domínio da aritmética.

Problema Soma Subconjunto
Entrada: Uma lista $S = \{a_1, a_2, \ldots, a_n\}$ de números naturais e um número natural k.
Pergunta: Existe um subconjunto S' de S, tal que $\sum_{a_i \in S'} a_i = k$?

Teorema 1.6.8 *Soma de Subconjunto é $NP-$completo.*

Demonstração. Claramente Soma de Subconjunto pertence à classe NP. Então o que resta mostrar é que Soma de Subconjunto é $NP-$hard. Nós faremos isto reduzindo 3-Sat ao problema Soma de Subconjunto.

Dado uma instância $\phi = C_1 \wedge C_2 \wedge \ldots \wedge C_m$de m cláusulas sobre n variáveis, nós construiremos uma instância I de *Soma de Subconjunto*, a qual possui $2(m+n)$ números, cada um tendo $(m+n)$ dígitos. Para cada variável proposicional x_i, existem dois números T_i e F_i. E correspondendo a cada cláusula C_i, existem duas fileiras S_{i_1}e S_{i_2}. Finalmente, nós temos um valor alvo k que tem 1 nos primeiros n dígitos e 4 nos m dígitos finais.

Por exemplo, assuma que $\phi = \overbrace{(x_1 \vee \neg x_3 \vee \neg x_4)}^{C_1} \wedge \overbrace{(\neg x_1 \vee x_2 \vee \neg x_4)}^{C_2}$, então a correspondente instância de *Soma Subconjunto* é:

Assuma que ϕ é satisfatível. Escolha todas as linhas que correspondem aos literais verdadeiros. Uma vez que a atribuição é consistente, os primeiros n bits de k combinam com estes n literais. Por outro lado, como cada cláusula C_i é satisfeita, pelo menos uma linha em que $C_i = 1$ é escolhido. Dependendo se a cláusula é satisfeita por um literal, dois literais ou pelos três literais, nós escolhemos S_{i_1}ou S_{i_2}ou ambos, respectivamente. Dessa forma todos os m bits de k são combinados. E a instância do problema Soma Subconjunto é uma instância "sim."

	x_1	x_2	x_3	x_4	C_1	C_2
T_1	1	0	0	0	1	0
F_1	1	0	0	0	0	1
T_2	0	1	0	0	0	1
F_2	0	1	0	0	0	0
T_3	0	0	1	0	0	0
F_3	0	0	1	0	1	0
T_4	0	0	0	1	0	0
F_4	0	0	0	1	1	1
S_{1_1}	0	0	0	0	1	0
S_{1_2}	0	0	0	0	2	0
S_{2_1}	0	0	0	0	0	1
S_{2_2}	0	0	0	0	0	2
k	1	1	1	1	4	4

Tabela 1.1: Instância Soma Subconjunto para ϕ

No sentido contrário, assuma que o alvo k foi alcançado por algum subconjunto dos números $\{T_1, F_1, \ldots, T_n, F_n, S_{1_1}, S_{1_2}, \ldots, S_{m_1}, S_{m_2}\}$. Para que isto ocorra nós devemos ter escolhido um de T_i e F_i para cada i. Se T_i é escolhido, ajuste x_i para verdadeiro; caso contrário, ajuste-a para falso. Assim existe uma atribuição consistente, e como os m bits finais de k são combinados, nós não podemos ter a situação onde todos os literais de uma cláusula são ajustadas para falso. Portanto, a fórmula é satisfatível. □

Programação Linear Inteira

Considere o seguinte problema otimização

Programação Linear Inteira (PLI)
Entrada: Uma matriz $A \in \mathbb{Z}^{m \times d}$ e vetores $\mathbf{b} \in \mathbb{Z}^m$ e $\mathbf{c} \in \mathbb{Z}^d$.
Pergunta: Existe um vetor de inteiros $\mathbf{x}$ tal queque maximiza $\mathbf{c}^T\mathbf{x}$; sujeito às restrições

$$\begin{aligned} \mathbf{Ax} &\leq \mathbf{b} \\ \mathbf{x} &\in \mathbb{Z}^d \end{aligned} \tag{1.14}$$

O correspondente problema de decisão consiste em determinar se existe um vetor x que satisfaz todas as restrições; em termos geométricos isto significa perguntar se existe um ponto com coordenadas inteiras na região definida pela intersecção dos semiespaços definidos por cada restrição.

Na programação linear; as variáveis x's não estão restritas a valores inteiros; podendo assumir valores reais. O problema de programação linear foi mostrado ser resolvido em tempo polinomial em 1980 por Khachian [12] utilizando um método que ficou conhecido como o método elipsoide. Em 1984, um algoritmo de tempo polinomial mais eficiente foi

[12]: Khachiyan (1980), *Polynomial algorithms in linear programming*

inventado por Karmarkar, seu método tornou-se conhecido como método do ponto interior. Desde então; vários refinamentos tem sido propostos. O método mais velho, conhecido como método SIMPLEX, proposto por Dantzig, ainda é o método mais utilizado para resolver problemas de programação linear; apesar de ser exponencial no pior caso.

Teorema 1.6.9 *A versão de decisão do problema de* PROGRAMAÇÃO LINEAR INTEIRA *é $NP-$completo.*

Demonstração. Não é difícil ver que versão de decisão de *PROGRAMAÇÃO LINEAR INTEIRA* pertence à classe NP. O certificado é apenas uma atribuição de valores inteiros ao x's; a qual pode ser verificada em tempo polinomial.

Para provar que *PROGRAMAÇÃO LINEAR INTEIRA* é NP-hard nós reduziremos o problema SAT a ele.

Note que em *PROGRAMAÇÃO LINEAR INTEIRA;* todas as restrições devem ser satisfeitas simultaneamente; ou seja em conjunção por qualquer solução viável. Então é portanto; natural representar cada cláusula; de uma fórmula ϕ;como uma restrição e as variáveis proposicionais são representadas por variáveis numéricas que podem assumir apenas os valores 0 ou 1; significando falso e verdadeiro; respectivamente. Considere; por exemplo; a cláusula

$$\alpha \vee \neg\beta \vee \gamma.$$

A satisfabilidade dessa cláusula pode ser capturada pela seguinte restrição

$$x_\alpha + (1 - x_\beta) + x_\gamma \geq 1. \tag{1.15}$$

Onde o operador lógico binário $\vee$é representado pela soma; de modo que $\alpha \vee \beta$ é $x_\alpha + x_\beta$; e a negação de uma variável $\neg\beta$ é $(1 - x_\beta)$.

A satisfabilidade da fórmula

$$\phi = (a) \wedge (b \vee \neg c \vee d) \wedge (\neg a \vee \neg d) \wedge (\neg b \vee c \vee e) \tag{1.16}$$

é equivalente a existência de uma solução para o seguinte sistema de desigualdades lineares:

$$\begin{aligned} x_a &\geq 1 \\ x_b + (1 - x_b) + x_d &\geq 1 \\ (1 - x_a) + (1 - x_d) &\geq 1 \\ (1 - x_b) + x_c + x_e &\geq 1 \\ x_a, x_b, x_c, x_d, x_e &\in \{0; 1\} \end{aligned}$$

Note que se uma fórmula lógica ϕ é satisfatível então pelo menos um literal deve ser ajustado para o valor verdadeiro em cada cláusula; isto significa que pelo menos uma variável foi ajustada para 1 se o literal é positivo ou ajustado para 0 se o literal é negativo; de qualquer forma

a soma do lado da mão direita é maior que 1 em cada desigualdade. Satisfazendo; portanto; o sistema de restrições lineares.

Na outra direção a prova prossegue de forma análoga. □

1.7 Parecidos porém diferentes

Nós vimos que o problema SAT é $NP-$completo, por outro lado 2SAT pertence à classe P. Um problema que está intimamente relacionado ao problema 2SAT, e que está no centro de muitos problemas de otimização, é o problema MAX-2SAT:

Problema MAX-2SAT
Entrada: Uma fórmula lógica ϕ em FNC, com cláusulas disjuntivas $C_1, C_2, \ldots, C_d$, cada uma contendo no máximo dois literais , e um inteiro positivo k.
Pergunta: Existe uma atribuição de valores lógicos as variáveis que que satisfaça k ou mais cláusulas?

A primeira vista nós poderíamos esperar que MAX-2SAT também pertencesse a classe P, uma vez que neste problema a restrição de satisfazer todas as cláusulas, como em 2SAT, foi relaxada de modo que agora nós "só" precisamos satisfazer k ou mais cláusulas. Infelizmente, contudo, MAX-2SAT é $NP-$completo.

Teorema 1.7.1 *[9] 3ESAT $\leq_P$ MAX-2SAT.*

[9]: Garey et al. (1976), *Some Simplified NP-Complete Graph Problems*

Demonstração. Suponha que nós temos uma instância do problemas *3ESat*, ou seja nós temos uma fórmula ϕ em CN em que cada cláusula contem exatamente três literais, dessa forma $\phi = (a_1 \vee b_1 \vee c_1) \wedge \ldots \wedge (a_m \vee b_m \vee c_m)$, onde cada a_i,b_i e c_i representa um literal. Para cada cláusula $C_i = (a_i \vee b_i \vee c_i)$ pertencente à ϕ nós criamos o seguinte fórmula lógica

$$\begin{aligned} S_i \quad = \quad & (a_i) \wedge (b_i) \wedge (c_i) \wedge (d_i) \wedge (\neg a_i \vee \neg b_i) \wedge (\neg a_i \vee \neg c_i) \wedge (\neg b_i \vee \neg c_i) \wedge \\ & (a_i \wedge \neg d_i) \wedge (b_i \vee d_i) \wedge (c_i \vee \neg d_i), \end{aligned}$$

onde d_i é uma variável proposicional que não parece em nenhum outro lugar. O valor de k é ajustado para $7m$, ou seja sete vezes o número de cláusulas de ϕ. Note que qualquer atribuição que satisfaz ϕ deve fazer um, dois ou três literais de cada cláusula $(a_i \vee b_i \vee c_i)a, b_i$ e c_i de cada cláusula assumir o valor lógico verdadeiro. Em qualquer um desses casos existe uma atribuição de valor lógico a variável d_i que faz com que exatamente 7 cláusulas de S_i podem ser satisfeitas. Adicionalmente nenhuma atribuição a d_i permitirá que mais que 7 cláusulas em 10 sejam satisfeitas, e no máximo seis cláusulas de S_i podem ser satisfeitas se a_i, b_i e c_i têm o valor falso. □

Dessa forma nós temos que a versão de decisão do problema Max-2Sat é $NP-$completo. O que contrária a nossa expectativa original.

Um outro problema que nós vimos pertencer à classe P é o problema Corte Mínimo, dessa forma seria natural esperar que o seu dual o problema do Corte Máximo também pertencesse à classe P, mas eis que para a nossa surpresa Corte Máximo é $NP-$completo.

Problema Corte Máximo
Entrada: Um grafo ponderado G. e um inteiro k.
Pergunta: Existe um corte em G de peso maior ou igual à k?

Nós mostraremos que Corte Máximo é $NP-$Completo provando que o problema $\neq$ 3-ESat pode ser reduzido em tempo polinomial à Corte Máximo.

Teorema 1.7.2 *Corte Máximo não Ponderado é $NP-$Completo.*

Demonstração. Mais especificamente nós reduziremos $\neq$ 3-ESat à Corte Máximo não Ponderado, onde toda aresta tem peso 1 e o peso de um corte é simplesmente o número de arestas que cruzam o corte. Uma outra forma de ver o problema de Corte Máximo não Ponderado é como se este estivesse perguntando se é possível pintar os vértices do grafo com duas cores de modo que k ou mais arestas tenham extremidades pintadas de cores diferentes. Uma vez que nós sabemos que a versão não ponderada de Corte Máximo é $NP-$Completo, nós também sabemos que a versão ponderada é $NP-$Completo.

A Figura 1.30 mostra a redução de $\neq$ 3-ESat para Corte Máximo. Nós utilizamos a fórmula $\phi = (x_1, x_2, x_3) \wedge (\neg x_1, x_2, \neg x_3)$, e ilustramos esta com a atribuição lógica $x_1 = verdadeiro$, $x_2 = verdadeiro$ e $x_3 = falso$. Toda aresta tem peso 1. Note que nesta atribuição todas as arestas, com exceção das tracejadas, atravessam o corte, o qual tem peso $n + 5m = 13$.

Nossa tranqueira escolha possui um par de vértices para cada variável x com uma aresta entre eles. A intenção é forçar qualquer solução para Corte Máximo a escolher um valor lógico para x colocando estes vértices em lados opostos, assim incluindo todas as $n$arestas entre estes vértices no corte.

A tranqueira restrição são triângulos. Incluindo as arestas conectando os vértices de cada triângulo as tranqueiras escolhas, cada triângulo possui 6 arestas. Se seus vizinhos não estão todos do mesmo lado do corte - isto é, se os literais na correspondente cláusula $\neq 3-$ESat não tem o mesmo valor lógico - então nós podemos pintar os vértices da tranqueira restrição de modo que 5 das seis arestas estejam no corte. Mas se seus vizinhos estão todos do mesmo lado, não mais que 4 destas arestas podem estar no corte. Assim nós podemos alcançar um corte de peso $n+5m$ se e somente se toda cláusula pode ser satisfeita, assim nós ajustamos k para $n + 5m$. Claramente nós podemos fazer esta redução em tempo polinomial. □

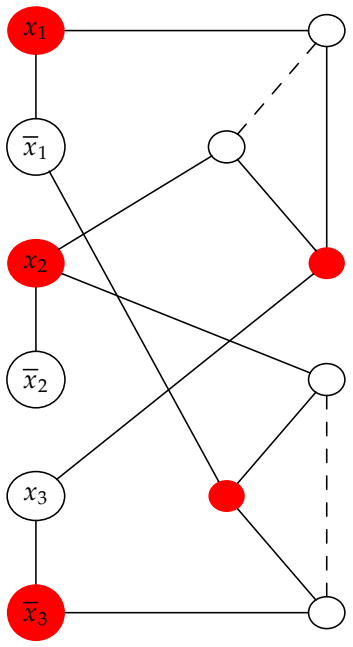


Figura 1.30: *Exemplo de redução $\neq 3-ESat$para* Corte Máximo

Qual é a lição? O fato de um problema ser fácil, como é o caso de Corte Mínimo, não significa que um primo próximo ou até mesmo um irmão próximo pertence a mesma classe de complexidade. Dois problemas parecidos podem ter complexidade diametralmente diferentes.

Nós vimos situações em que a variante de um determinado problema está na classe P, enquanto que outra variante, do mesmo problema, pertence à classe $NP-$completo. Qual é a origem da diferença? Por que alguns problemas são fáceis enquanto que outros, muito parecidos, são difíceis? Algumas razões são relativamente diretas e envolvem a quantidade de escolhas que nós temos quando resolvendo um problema e a forma pela qual as variáveis interagem umas com as outras.

Por exemplo, o problema de colorir um grafo com duas cores está em P, já o problema de determinar se um grafo pode ser pintado com três cores é $NP-$completo. Parece existir uma barreira que nós cruzamos quando nós compramos mais um lápis de cor. Se você só tem alternativas de cores para pintar um grafo, então ao escolher acor de um vértice você automaticamente também se comprometeu com a cor de todos os vértices adjacentes a este. O processo de colorir propaga-se de forma automática para todos os componentes conectados do grafo.

Por outro lado, se nós temos três alternativas de cores para pintar o grafo, então ao decidir pintar um vértice com uma determinada cor nós ainda podemos pintar cada um dos seus vizinhos de duas cores diferentes. Dessa forma se um vértice possui n vizinhos então existem 2^n formas diferentes de pintá-los; e assim sucessivamente. Dessa forma um algoritmo de força bruta poderá demandar um tempo da ordem de $O(2^n)$ para explorar todas as alternativas, o que é exponencial.

O mesmo tipo de raciocínio aplica-se ao problema 2Sat. Se nós decidimos fixar uma variável lógica x para o valor "verdadeiro" então todas as cláusulas que possuem x estão satisfeitas, já as cláusulas que possuem o complemento de x, ou seja $(\neg x \vee y)$, nos forçam a ajustar y para o valor "verdadeiro", ou seja nós temos.

1.8 Classe coNP

O que ocorre se nós tomamos um problema como Caminho Hamiltoniano, e trocamos instâncias sim e instâncias não?

Problema Nenhum Caminho Hamiltoniano
Entrada: Um grafo G.
Pergunta: G não possui nenhum caminho Hamiltoniano?

Este é um problema muito diferente. Agora existe uma prova simples que Merlin pode entregar para o Rei Arthur que o convença que a resposta é "não", mas parece ser difícil, para Merlin, provar que a resposta é "sim" sem uma pesquisa exaustiva. Problemas como este tem uma classe de complexidade própria, eles são chamados de $coNP$, a classe

de complexidade na qual, se a entrada é uma instância "não", então existe um certificado curto que Merlin pode entregara para o Rei que o convencerá desse fato.

Definição 1.8.1 *Um problema A pertence à classe $coNP$ se e somente se $\overline{A}$ pertence à classe NP.*

Note que se um problema está em P então seu complemento também está em P.

Considere o seguinte problema:

Problema TAUTOLOGIA
Entrada: Uma fórmula Booleana $\phi(x_1, \ldots, x_n)$.
Pergunta: ϕ é satisfatível por toda atribuição lógica?

Teorema 1.8.1 *TAUTOLOGIA é $coNP$*

Demonstração. Se ϕ não é uma tautologia então tudo o que Merlin precisa entregar ao rei para convencê-lo disso é uma atribuição que não satisfaz a fórmula. O rei pode, em tempo polinomial, verificar que o certificado é um certificado válido. □

Um amigo seu está meio confuso com o conceito de NP e $coNP$. Ele acredita que $coNP$ é o complemento de NP. Prove que ele está errado. A classe $NP \cap coNP$ é classe dos problemas onde, para cada instância x, existe uma prova simples que x é uma instância "sim" ou uma prova simples que x não é uma instância "sim". Prove que $P \subseteq (NP \cap coNP)$. Mostre que se $P = NP$ então $NP = coNP$. O reverso é necessariamente verdade? Considere o seguinte problema de decisão

Problema Fator Menor
Entrada: Dois inteiros q e r.
Pergunta: q possui um fator menor que r?

Mostre que esse problema pertence à classe $NP \cap coNP$. Prove que se existir um problema A tal que $A \in (NP \cap coNP)$ e A é $NP-$completo então $NP = coNP$. Esta é uma das razões pelas quais muitos acreditam que Fatoração não é $NP-$completo.

Fatoração

O problema da fatoração é:

> *Problema* FATORAÇÃO
> *Entrada: Um número positivo n.*
> *Saída: A fatoração de n em números primos, ou seja, $n = p_1^{i_1} p_2^{i_2} \cdots p_k^{i_k}$*

Note que o problema Fatoração *não* é um problema de decisão. Contudo, nós podemos criar uma versão de decisão do problema da fatoração.

> *Problema* FATORAÇÃO VERSÃO DECISÃO
> *Entrada: Um número positivo n, e um número b.*
> *Saída: n tem um fator não trivial $\leq b$?*

Observação 1.8.1 n tem um fator $\leq b$ se e somente se este tem um fator primo $\leq b$.

Nós podemos fatorar n com polinomialmente muitas chamadas a um algoritmo para o problema de decisão. Utilize pesquisa binária para encontrar o menor fator não trivial de n (este será primo), divida n pelo fator, e repita.

A primeira coisa a ser observada é que o problema FATOR MENOR pertence à classe NP. A prova é simples, adivinhe um fator $p > 1$ e $p \leq b$, e verifique se p divide n.

FATOR MENOR $\in coNP$. O certificado é a fatoração prima de $n = p_1^{i_1} p_2^{i_2} \cdots p_k^{i_k}$, verifique a fatoração e a primalidade de p_1 (nós podemos utilizar o algoritmo de primalidade em P ou algoritmo que prova que primalidade pertence à NP). Verifique que nenhum p_i é menor ou igual à b.

Portanto, FATOR MENOR $\in NP \cap coNP$. Se $NP \cap coNP = P$ então *Fator Menor* $\in P$ o que implica que fatoração $\in P$!!! Por outras palavras, *Fatoração* $\notin P \Rightarrow NP \cap coNP \neq P$. FATORAÇÃO é Difícil? Provavelmente nós não podemos utilizar a noção de $NP-$hard para provar que *Fatoração* é um problema difícil. Por quê? Se *FATORAÇÃO* é $NP-$hard então $NP = coNP$.

Teorema 1.8.2
1. *$A \leq_P B$ se e somente se $\widetilde{A} \leq_P \widetilde{B}$.*
2. *A é NP-hard se e somente se $\widetilde{A}$ é coNP-hard.*
3. *A é completo se e somente se $\widetilde{A}$ é cocompleto.*
4. *Se A é completo então $A \in coNP$ se e somente se $NP = coNP$.*

1.9 Memória

Em um mundo em que o recurso mais escasso é espaço haveria algum diferença entre um computador determinístico e um não determinístico?

Por outras palavras, quando memoria é o nosso recurso escasso, o que nós podemos resolver nesse mundo?

Surpreendentemente quando a nossa preocupação é memória a lacuna entre encontrar uma solução e verificar uma solução é muito mais estreita do que a que nós acreditamos existir quando tempo é a nossa preocupação. Ainda mais bizarro é o fato de que, em um mundo limitado pela memória ao invés do tempo, não existe muita diferença entre provar que alguma coisa existe e provar que esta não existe. Provas da existência e da não existência são igualmente fáceis de verificar.

Atualmente um notebook típico possui 10^{10} bits de memória, ou 10^{12} se nós incluirmos o disco rígido. Ou seja o número de estados possíveis em que o computador pode estar é $2^{10^{12}}$! Esse número é enorme, mas finito.

Onde nós mantemos o programa? Mais importante, onde nós colocamos a entrada?

Nós consideramos que o programa e a entrada não fazem parte da nossa memória de trabalho. O programa e a entrada residem em uma memória específica a qual nós podemos apenas ler, mas não modificar.

Qual é o montante mínimo de memória de trabalho que nosso computador deve possuir? Bem se a entrada é de tamanho n nosso computador deve possuir pelo menos $log\ n$ bits de memória, uma vez que nós precisamos de $log\ n$ bits para especificar qual bit da entrada de n bits nós queremos.

Nós agora estamos em uma posição que nos permite definir classes de complexidade limitadas em espaço, consistindo de problemas que podem ser resolvidos com uma dada quantia de memória.

Definição 1.9.1 *$Space(f(n))$ é classe de problemas para os quais existem algoritmos que resolvem instâncias de tamanho n tendo acesso somente de leitura à entrada e $O(f(n))$ bits de memória.*

Duas classe de interesse são:

$$L = Space(lg\ n) \tag{1.17}$$

$$PSpace = Space(poli(n)) = \bigcup_{k>0} Space(n^k) \tag{1.18}$$

Considere o seguinte problema:

Problema Palíndromo
Entrada: Uma string w de tamanho n.
Pergunta: w é um palíndromo?

O algoritmo 2 resolve o problema Palíndromo

Algorithm 2 Palíndromo

```
PALINDROMO(w, n)
i = 1
j = n
while (i < j)
    if (w_i ≠ w_j)
        return FALSE
    else i = i + 1
         j = j − 1
return TRUE
```

O algoritmo utiliza $\Theta(lg\, n)$ bits de memória. Portanto, o problema Palíndromo pertence à classe L.

Vamos agora considerar o seguinte problema:

Problema Retorno à origem
Entrada: Um autômato celular e um estado inicial.
Pergunta: O estado original é parte de uma órbita periódica? Isto é, existe um t tal que o sistema retorna ao estado inicial depois de t passos?

Exemplo 1.9.1 Considere a instância do problema *Retorno à origem*, na qual o autômato celular é definido pela regra 30 introduzida por Stephen Wolfram em 1983 [19].

[19]: Wolfram (1983), *Statistical mechanics of cellular automata*

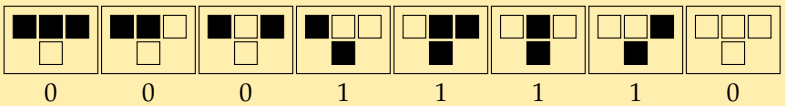


Figura 1.31: Autômato celular - Regra 30

E o estado inicial é:

Esta é uma instância "sim" ou uma instância "não"? Nós podemos resolver este problema executando o autômato celular, e verificando em cada ponto se o estado atual é idêntico ao estado inicial, esta execução é mostrada na figura 1.32. Note que existem $2^{poli(n)}$ estados possíveis, e a dinâmica do autômato celular pode envolver um ciclo que leva um tempo exponencial para ser observado. No entanto, esta execução só demanda $poli(n)$ bits para manter a trilha do estado corrente, assim o problema está em $PSpace$.

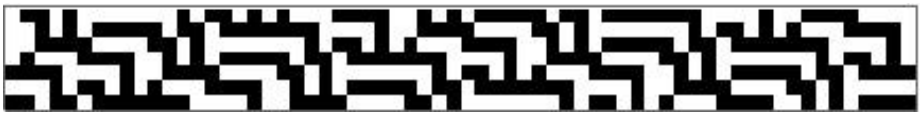


Figura 1.32: Execução do autômato celular

Como as classes L e $PSpace$ se relacionam com as classes de complexidade limitadas em tempo?

$$Space(f(n) \subseteq Time(2^{O(f(n))}). \tag{1.19}$$

Em particular, $2^{O(lg\, n)} = poli(n)$. Por exemplo, $2^{clg\, n} = n^c$. Portanto, um programa que utiliza $O(lg\, n)$ bits de memória possui um número polinomial de estados e executa em tempo polinomial

$$L \subseteq P.$$

Por outro lado,

$$Time(f(n)) \subseteq Space(f(n)).$$

Destacando-se que $P \subseteq PSpace$.

Agora suponha que um problema pode ser resolvido não deterministicamente em tempo t. Assumindo que $t = O(f(n))$ então

$$Ntime(f(n)) \subseteq Time(2^{O(f(n))}).$$

f

Nós podemos simular um algoritmo não determinístico gerando todos os 2^t possíveis certificados e executando o algoritmo do Rei Arthur sobre cada um desses certificados.Portanto, o tempo total da simulação determinística é 2^t x Tempo que o Rei Arthur leva para verificar um certificado. Mas se o certificado tem tamanho t o Rei Arthur leva um tempo $O(t)$ para verificá-lo. Quanto de memória o algoritmo determinístico utiliza?

O algoritmo determinístico utiliza $t + O(t)$ bits de memória.

Logo nós podemos concluir que

$$NTime(f(n) \subseteq Space(f(n)),$$

em particular

$$NP \subseteq PSpace. \tag{1.20}$$

Juntando tudo nós temos:

$$L \subseteq P \subseteq NP \subseteq PSpace \subseteq ExpTime \tag{1.21}$$

Todas estas classes de complexidade são diferentes? Nós sabemos que

$$L \subset PSpace$$

Portanto, pelo menos um das inclusões em 1.21 deve ser estrita. Na verdade acredita-se que

$$L \subset P \subset NP \subset PSpace \tag{1.22}$$

No entanto, isso ainda não foi provado.

Algo está faltando. Como nós definimos versões não determinísticas das classes L e $PSpace$? Que tipo de certificado o Rei Arthur precisa para que ele possa encontrar o caminho através do espaço estado?

1.9.1 Mostre-me o Caminho

O Rei Arthur esteve observando Merlin, enquanto ele resolvia uma instância do quebra-cabeça dos Blocos Deslizantes (nós definiremos este problema mais a frente). Depois de algum tempo Merlin resolve o problema, ele dá um sorriso de triunfo. O Rei não pode recordar a sequência de movimentos, mas Merlin provou que existe uma solução - ou seja, existe um caminho , através do espaço de estados possíveis, do estado inicial para o estado final.

Esta é a essência da computação não determinística quando memória é limitada. Vamos imaginar que Merlin afirma que uma determinada instância é uma instância "sim." Ele pode provar esta afirmação para o Rei fornecendo um certificado. Quando o Rei verifica este certificado, ele segue um caminho no espaço de estados, até alcançar um estado no qual ele concorda com Merlin e retorna "sim."

O único problema com este tipo de certificado é que este pode ser muito grande para o Rei armazená-lo em sua memória. Afinal de contas o Rei tem somente m bits de memória, então o tamanho de seu espaço de estados, e por conseguinte o tamanho possível do caminho, é 2^m. Assim, ao invés de dar ao Rei o certificado todo de uma única vez, Merlin toma o Rei pela mão, e lhe mostra um passo de cada vez. Da mesma forma que ocorreu quando o Rei observou Merlin resolvendo o problema dos Blocos Deslizantes, o Rei necessita apenas manter a trilha do estado atual, e garantir que cada passo individual é legal. Não existe necessidade do Rei recordar todos os estados que vieram antes.

De modo a fazer isso mais concreto, considere o problema *Particionamento de Inteiros:*

Problema Particionamento de Inteiros
Entrada: Uma lista $S = \{x_1, \cdots, x_l\}$ de inteiros positivos.
Pergunta: Existe um particionamento balanceado, isto é um subconjunto $A \subseteq \{1, \cdots, l\}$ tal que $\sum_{i \in A} x_i = \sum_{i \notin A} x_i$?

Para prova r que a resposta é "sim", Merlin pode dar um certificado consistindo de l bits, dizendo à Arthur para adicionar ou subtrair cada x_i. O Rei pode verificar este certificado com o algoritmo3.

Algorithm 3 Verificando Certificado

```
VERIFICANDO-CERTIFICADO-PARTICIONAMENTO(S, w, l)
r = 0
for i = 1 to l
    if (w_i == 1)
        r = r + x_i
    else r = r − x_i
if r == 0
    return TRUE
```

Em cada passo, tudo o que o Rei necessita manter em memória são os valores atuais de i e r, assim existe um estado para cada combinação (i, r). Estes estados formam um grafo dirigido G no qual cada aresta corresponde a um passo único da computação. No algoritmo 3 cada estado tem duas arestas saindo, correspondendo a diferentes ações que o algoritmo que executar baseado no próximo bit do certificado.

Baseado neste exemplo, nós podemos definir as classes $NPSpace$ da seguinte forma:

Definição 1.9.2 *$NSpace(f(n))$ com a seguinte propriedade. Existe um programa Π, que verifica um certificado, o qual tem acesso somente de leitura à entrada x, que pode consultar o próximo bit do certificado, e que utiliza $O(f(n))$ bits de memória. Assuma que G é o grafo dirigido descrevendo o espaço de estados de Π quando dado a entrada x. Então x é uma instância "sim" se e somente se existir um caminho em G do estado inicial de Π para o estado "sim."*

Especificamente, nós podemos definir as versões não determinísticas de L e $PSpace$:

$$NL = NSpace(log\, n) \tag{1.23}$$

$$NPSpace = NSpace(poli(n)) = \bigcup_{k>0} NSpace(n^k) \tag{1.24}$$

Por que definir estas classes? Nós definimos classes de complexidade para podermos expressar ideias fundamentais a respeito de como problemas podem ser resolvidos. Nós definimos NP porque para muitos problemas, soluções são fáceis de verificar , mesmo se elas são difíceis de serem obtidas. Da mesma maneira, as classes NL e $NPSpace$ capturam o fato de que para alguns problemas, suas soluções são muito longas, mas nós podemos verificá-las passo a passo com uma quantia de memória pequena.

Existe uma outra forma de definir a classe $NPSpace$, na qual nós consideramos programas não determinísticos como àqueles da seção ??, em que nós temos uma instrução mágica "goto ambas" que divide a computação em dois caminhos possíveis.

Definição 1.9.3 *$NSpace(f(n))$ é a classe dos problemas para os quais existem algoritmos não determinísticos que utilizam $O(f(n))$ bits de memória, tal que a entrada é uma instância "sim" se e somente se existir um caminho na computação que retorna "sim".*

Toda essa discussão à respeito de caminhos traz à tona um problema em Teoria dos Grafos que casa perfeitamente com a ideia apresentada na definição 1.9.2, o problema da Alcançabilidade*:*

Problema Alcançabilidade
Entrada: Um grafo dirigido G e dois vértices s e t.
Pergunta: Existe um caminho de s para t em G?

Alcançabilidade $\in NL$.

O certificado é simplesmente um caminho de s para t. Dado que nós temos acesso somente de leitura à G, o qual é representado na forma de sua matriz de adjacências, o Rei Arthur pode seguir o caminho e verificar se cada passo segue uma aresta válida. Se G possui n vértices, o caminho pode ter no máximo n arestas, mas o Rei necessita somente $O(lg\ n)$ bits de memória para saber onde ele está. Talvez, com sorte, você se recorde das aulas de Teoria dos Grafos que o problema da *Alcançabilidade* pode ser resolvido em tempo polinomial no tamanho do grafo G. Portanto, Alcançabilidade $\in P$. Uma vez que qualquer problema pertencente à classe NL corresponde a um problema *Alcançabilidade* em um grafo de tamanho $2^{O(lg\ n)}$, nós temos que

$$NL \subseteq P.$$

Qual é lacuna entre computação determinística e não determinística? Quando tempo é a nossa preocupação nós acreditamos que ela é enorme. No entanto, quando memória é a nossa preocupação, a lacuna é muito menor. Nós provaremos agora o seguinte resultado

Teorema 1.9.1 *Savitch [16] provou que para qualquer $f(n) \geq log\, n$,*

$$NPSpace(f(n)) \subseteq Space(f(n)^2).$$

[16]: Savitch (1970), *Relationships Between Nondeterministic and Deterministic Tape Complexities*

Demonstração. (Esquemática) Nós Necessitamos simular um algoritmo não determinístico que utiliza $f(n)$bits de memória de uma forma determinística utilizando apenas $f(n)^2$ bits de memória. Uma abordagem ingênua consiste em tentar todos os ramos da computação não determinística de uma forma sistemática um a um. Esta simulação necessita manter uma trilha de qual ramo está atualmente sendo explorado, de modo que nós possamos seguir o próximo ramo posteriormente. Mas um ramo que utiliza $f(n)$ espaço pode executar por $2^{O(f(n))}$ passos, e cada passo pode ser uma escolha não determinística. Explorar os ramos sequencialmente demandaria armazenar todas as escolhas utilizadas sobre um ramo particular para que nós possamos encontrar o próximo ramo. Portanto, esta abordagem pode utilizar $2^{O(f(n))}$ bits de memória.

Ideia

Confúcio: Uma jornada de mil milhas começa com um só passo.

Savitch: Uma jornada de mil milhas deve ter um passo que te levará a 500 milhas de casa e que estará a 500 milhas do seu destino.

Ao invés disso nós consideramos uma abordagem diferente a partir do problema mais geral. Nós temos dois estados do nosso computador, e_1 e e_2, e um número t e nós devemos testar se o algoritmo não determinístico pode ir de e_1 para e_2 em no máximo t passos. Nós chamaremos este problema de alcançabilidade em que e_1 é o estado inicial e e_2 é o estado "sim", e t é o número máximo de passos que o algoritmo não determinístico pode utilizar para determinar se a instância é uma instância "sim."

Nosso objetivo é apresentar um algoritmo recursivo determinístico para o problema da alcançabilidade. Este funciona pesquisando por uma configuração intermediária e_m, e recursivamente testa se (1) e_m pode ser alcançado a partir de $e_1$em no máximo $\frac{t}{2}$ passos, e (2) e_2 pode ser alcançado a partir de e_m em no máximo $\frac{t}{2}$ passos. Reutilizando espaço para cada uma dessas chamadas recursivas nos possibilita uma grande economia de memória.

O algoritmo necessita de espaço suficiente para armazenar a pilha de chamadas recursivas. Cada nível da recursão utiliza $O(f(n))$ bits para armazenar um estado. A profundidade da recursão é $lg\, t$, onde t é o tempo máximo que o algoritmo não determinístico pode utilizar em qualquer ramo da computação. Nós temos que $t = 2^{O(f(n))}$,logo $lg\, t = O(f(n))$. Assim a simulação utiliza $O(f(n)^2)$ bits de memória. □

Por outras palavras, resolver problemas deterministicamente ao invés de não deterministicamente requer expandir a nossa memória quadraticamente, ao invés de exponencialmente. Em particular, uma vez que o

Algorithm 4 Savitch

```
SAVITCH(i, j, t)
if (n == 1)
    if (i == j ∨ A_ij == 1)
        return SIM
    else return NÃO
for k ← 1 to t
    if (SAVITCH(i, k, t/2) ∧ SAVITCH(k, j, t/2))
        return SIM
return NÃO
```

quadrado de polinômio é outro polinômio de grau maior, nós temos o seguinte resultado:

Corolário 1.9.2

$$PSpace = NPSpace.$$

$$\mathsf{PSPACE} = \mathsf{NPSPACE}$$

E isto é radicalmente diferente da nossa crença que $\mathsf{P} \neq \mathsf{NP}$.

1.9.2 Jogos

Durante uma viagem entendiante para o reino Tão Tão Distante, Merlin e Rei Arthur resolveram passar o tempo jogando o seguinte jogo: Merlin diz o nome da capital c de um reino. O Rei Arthur deve então dizer o nome da capital de um outro reino c', ainda não mencionada, cuja primeira letra é a última letra de c. Merlin e Arthur alternam-se até que um dos dois não consiga continuar. Merlin possui uma estratégia vencedora?

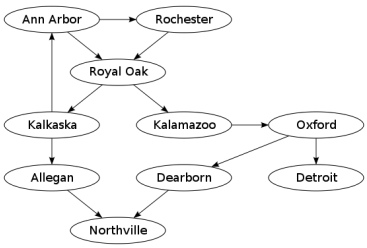


Figura 1.33: Grafo do Jogo Geografia.

Exemplo 1.9.2 Kalkaska ⟶ Ann Arbor ⟶ Royal Oak ⟶Kalamazoo ⟶ Oxford ⟶ Detroit

Nós podemos definir este jogo da seguinte forma:

Jogo Geografia
Entrada: Um grafo dirigido $G = (V, E)$. Dois jogadores se alternam escolhendo, se possível, uma aresta deixando o nó atual e indo para um nó não visitado.
Pergunta: O primeiro jogador possui uma estratégia vencedora? Ou seja, o primeiro jogador pode escolher um vértice s que lhe garante a vitória?

Teorema 1.9.3 O jogo *Geografia* é $PSpace$–completo [14].

[14]: Lichtenstein et al. (1980), *GO Is Polynomial-Space Hard*

Demonstração. Nós temos que provar que *Geografia* $\in PSpace$ e $\forall B \in PSpace \leq_K$ *Geografia*.

Para provar que Geografia $\in PSpace$ nós apresentamos o algoritmo recursivo 5 que utiliza espaço polinomial.

Algorithm 5 Geografia

```
GEOGRAFIA(G, jogador, u)
L ← o conjunto de todos os vertices v tal que (u, v) ∈ E e
    v ainda não foi marcado
if jogador == MERLIN
    if L == ∅
        return FALSE
    else    for cada v ∈ L
                G' ← G com v marcado
                if GEOGRAFIA(G', ARTHUR, v)
                    return TRUE
                return FALSE
else if L == ∅                                  // Arthur
        return TRUE
    else for cada v ∈ L
            G' ← G com v marcado
            if GEOGRAFIA(G', MERLIN, v)
                return FALSE
        return TRUE
```

Agora nós mostraremos que o jogo *GEOGRAFIA* é $PSpace$–hard. Nós faremos isto reduzindo o problema *JOGO SAT* ao jogo *GEOGRAFIA*, ou seja *JOGO SAT* $\leq_K$ *GEOGRAFIA*.

Nós faremos isso mostrando a redução de uma instância $\phi = (x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3)$ de *JOGO SAT* para uma instância do jogo GEOGRAFIA. Nós começamos com a tranqueira escolha, mostrada na Figura 1.34.

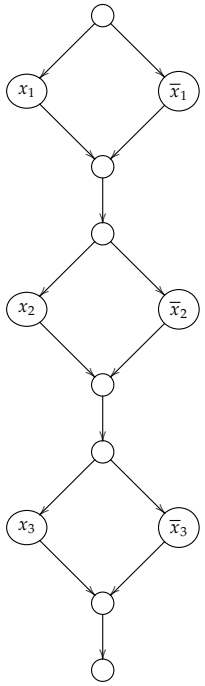


Figura 1.34: Tranqueira escolha - GEOGRAFIA

Esta é projetada para emular o procedimento de escolha dos valores lógicos das variáveis em uma instância do problema SAT QUANTIFICADO. Cada estrutura em um losangolo corresponde a uma variável quantificada. Arthur e Merlin alternam-se decidindo o caminho em cada nó que bifurca. Merlin joga primeiro, ele escolha se a variável x_1 é ajustada para verdadeiro ou falso indo para a esquerda ou para à direita, respectivamente. Arthur então deve escolher um valor para x_2. Estas atribuições alternadas continuam dessa forma até atingir o último nó do lado esquerdo. Depois que ambos os jogadores tenham caminhado pelos nós da cadeia do lado esquerdo. É então a vez de Merlin jogar, porque nós assumimos que o último jogador a jogar é Merlin, neste ponto Merlin não tem nenhuma escolha a não ser seguir o caminho para o lado direito do grafo, lado este que representa a tranqueira restrição.

A tranqueira restrição é mostrada na Figura 1.35. Já a Figura 1.36 mostra a grafo completo do jogo Geografia.

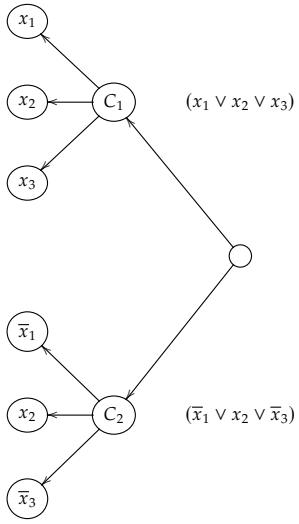


Figura 1.35: Tranqueira restrição - GEOGRAFIA

Merlin começa, ele escolhe ajustar x_1 para verdadeiro, o que no jogo Geografia significa que ele foi para o vértice rotulado x_1. No jogo quando Merlin visita nó nós pintamos ele de vermelho, e quando Arthur visita um nó nós pintamos ele de azul, as cores são apenas para indicar que

um vértice já foi visitado.Recorde-se que em Jogo Sat Merlin ganha se a fórmula é verdadeira e o Rei Arthur ganha se a fórmula é falsa. Suponha que após fixar o valor de todas as variáveis nós estamos com o grafo mostrado na Figura 1.37.

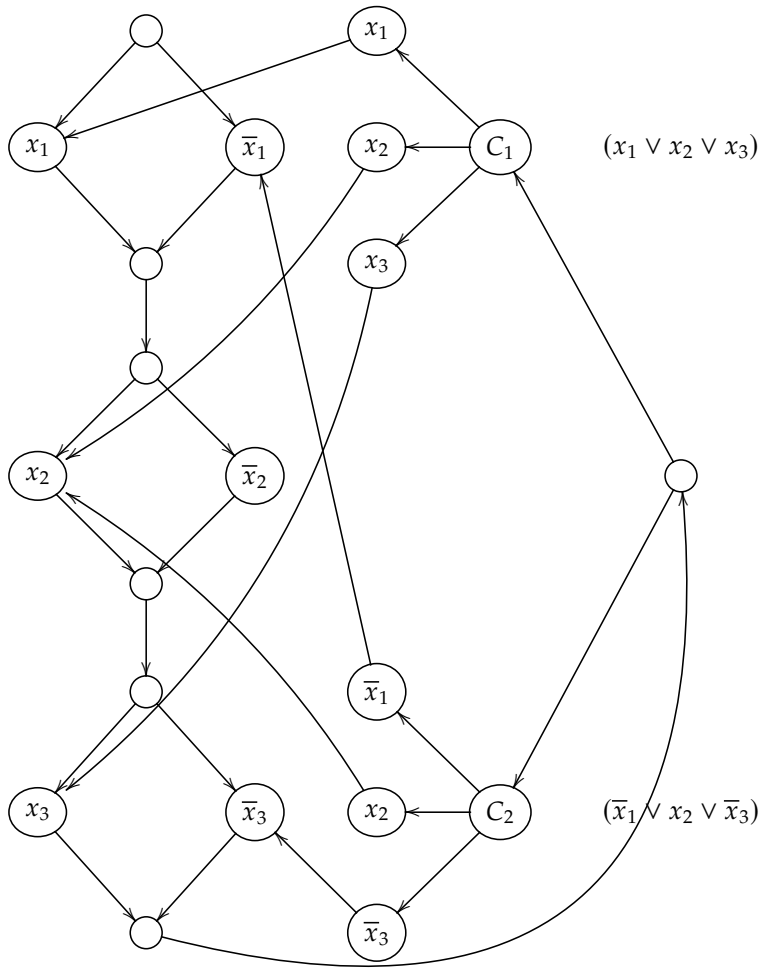


Figura 1.36: Exemplo redução Jogo Sat à Geografia

O que o Rei deve fazer agora? Ele deve escolher uma cláusula que não tenha sido satisfeita, ou seja ele deve ir para um nó que representa uma cláusula tal que Merlin não consiga ir para nenhum outro vértice. Se nós observarmos o grafo nós podemos notar que isto ocorrerá se nenhuma variável tiver sido ajustada para o um valor que satisfaz a fórmula. Dessa forma o lado direito do grafo garante que o Rei vence se a fórmula é falsa e perde se esta é verdadeira. □

Todo jogo com dois jogadores podem ser entendido como uma batalha entre Merlin e Arthur, onde Merlin deseja provar que ele tem uma estratégia vencedora e o Rei Arthur aposta que ele não tem. No entanto, para muitos jogos não é óbvio como nós podemos codificar relacionamentos lógicos arbitrários na dinâmica do jogo. Vamos considerar um dos jogos mais populares que existe.

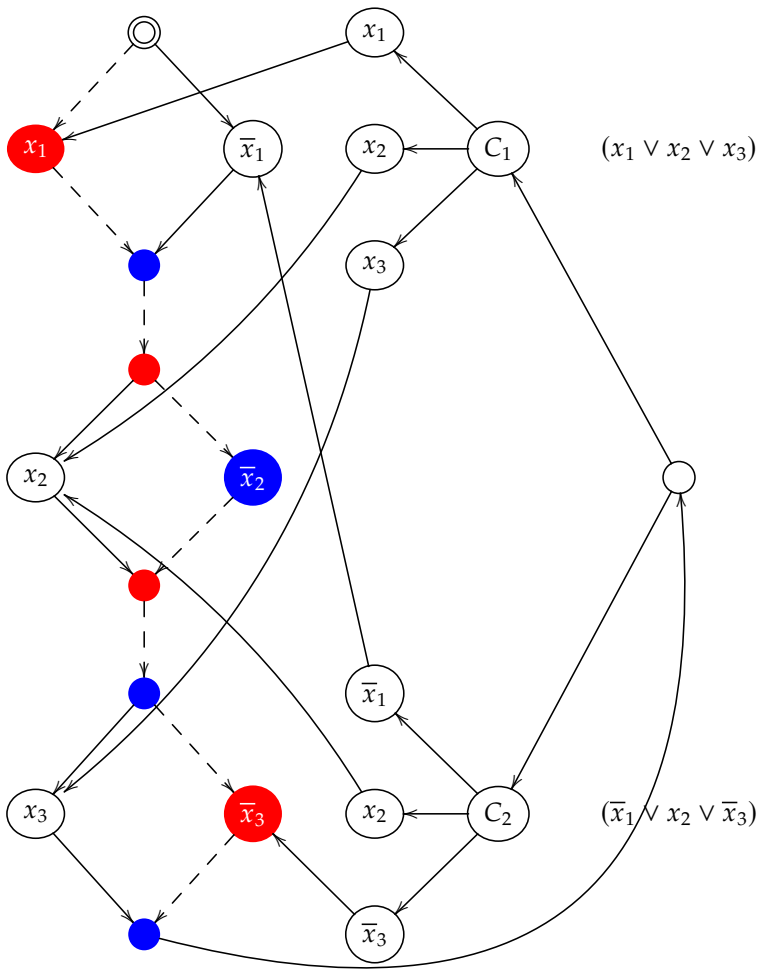


Figura 1.37: Exemplo de execução do Jogo Geografia no grafo de uma instância de Jogo Sat.

1.9.3 Jogo de Damas

O rei Arthur é um grande admirador do jogo de Damas, que é um jogo de tabuleiro muito elegante, inventado na China a mais de 2500 anos atrás. As regras são muito simples. as estratégias, por outro lado, são muito profundas e elaboradas. De modo a entretê-lo Sir Lancelot criou a seguinte variante do jogo, na qual todo movimento deve capturar uma peça. Vamos chamar esta variante de Damas Sempre Capturando. Este jogo pode ser jogado em um tabuleiro quadrado de qualquer dimensão, ou seja em um tabuleiro de dimensões $n \times n$, com o correspondente número de peças pretas e brancas.

Jogo de Damas Sempre Capturando[a]
Entrada: Uma configuração do tabuleiro do jogo de damas geral, ou seja jogado em um tabuleiro $n \times n$.
Pergunta: O rei Arthur, que é o próximo a jogar, possui uma estratégia vencedora?

[a] Esta seção segue partes do artigo [5].

[5]: Bosboom et al. (2018), *Losing at Checkers is Hard*

Teorema 1.9.4 *Damas Sempre Capturando é $PSPACE-$completo.*

Demonstração. A prova consiste de duas partes: a primeira parte provará que o problema pertence à classe $PSPACE$, a segunda parte mostrará que este jogo é $PSPACE-$hard. Para este jogo a pertinência na classe $PSPACE$ é direta, porque em Damas Sempre Capturando cada movimento estritamente decresce o número de peças, assim o número de movimentos é limitado pelo número inicial de peças. Para provar que este é $PSPACE-$hard, nós reduziremos um problema conhecido ser $PSPACE-$hard a este.

Como foi mencionado anteriormente Schaefer [17] mostrou que decidir o vencedor de Jogo Sat é $PSPACE-$completo. Dado uma instância ϕ de Jogo Sat, nós construiremos tranqueiras correspondentes aos conectivos lógicos e a as variáveis existentes na fórmula, nós também mostraremos como estas tranqueiras podem ser conectadas de modo a representar a fórmula lógica ϕ.

[17]: Schaefer (1978), *On the Complexity of Some Two-Person Perfect-Information Games*

Fio. Um fio é simplesmente uma linha de peças vermelhas que se alternam com espaços em branco. Uma peça preta pode atravessar o comprimento inteiro de um fio em um único movimento. Como toda peça é uma dama, esta pode pousar em qualquer quadrado vazio alongo do seu comprimento do fio.

Shift. Em muitas tranqueiras, a paridade das peças de chegam e saem são diferentes. Para manipular, uma possível mudança de direção, nós podemos empregar a tranqueira *Shift*, como mostrado na figura 1.38a.

Variável. Quando o jogo começa os jogadores alternam-se escolhendo ativar ou desativar a tranqueira variável, mostrada na Figura 1.38b. Se as Pretas movem-se primeiro com relação a uma dada variável, elas devem continuar capturando as peças vermelhas ao longo do fio, o que corresponde a ajustar a variável para o valor lógico verdadeiro. Se as Vermelhas movem-se primeiro, elas podem capturar a peça preta, correspondendo a ajustar a variável para falso. Uma vez que uma variável é ajustada esta nunca mais muda.

Fanout, Choice, And, Or. Estas tranqueiras são todas baseadas em uma construção similar. Triplas de peças vermelhas impedem que as peças pretas de capturarem, e pares de peças pretas impedem as peças vermelhas de capturarem. Quando uma peça preta chega via um fio, esta captura a peça do meio de alguma tripla(s), permitindo que outras peças pretas deixem a tranqueira ao longo dos fios de saída. A tranqueira Choice também tem dois fios de saída, mas somente uma peça preta pode deixar a tranqueira, assim as Pretas devem escolher qual fio ativar. Na tranqueira And, ambas peças pretas devem chegar para quebrar suas respectivas triplas, enquanto que na tranqueira Or, a chegada de uma peça preta ao longo de qualquer um dos dois fios já possibilita que a peça preta deixe a tranqueira. Mesmo se ambas as entradas são ativadas, o fio de saída é ativado quando a primeira peça preta deixa a tranqueira.

Em Jogo Sat, Merlin vence se a fórmula é verdadeira e Arthur ganha se a fórmula é falsa. Já no jogo Damas Sempre Capturando, Merlin vence se o rei Arthur não pode capturar quando é a sua vez de jogar, e perde se ele não pode capturar. Como estas duas formas, aparentemente, diferentes

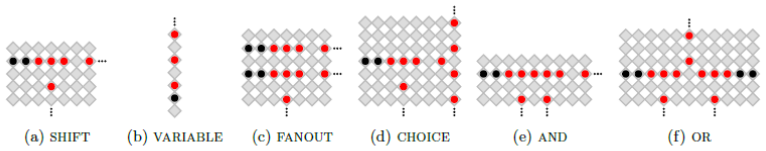


Figura 1.38: Tranqueiras para a redução de Jogo Sat à *Damas Sempre Capturando*

de definir o vencedor podem ser conciliadas? A ideia consiste em permitir que Merlin tenha mais movimentos de captura de peças se e somente se a fórmula for verdadeira, e Arthur tenha mais movimentos de captura se e somente se a fórmula for falsa. Para fazer isso nós teremos que Arthur possui uma coleção F_A de movimentos livres, cada movimento é formado por duas peças vermelhas adjacentes a uma única peça preta. Dessa forma Arthur poderá realizar $|F_A|$ movimentos. Nós ajustaremos o tamanho de F_A para ser igual ao número de tranqueiras no jogo. Por outro lado, Merlin possui um conjunto grande de tranqueiras Shift □

1.9.4 GO

Go é um jogo de tabuleiro muito elegante, inventado na China a mais de 2500 anos atrás. As regras são muito simples. as estratégias, por outro lado, são muito profundas e elaboradas.

Dois jogadores alternam-se colocando uma pedra da sua cor no tabuleiro, e os jogadores também podem passar a vez. Se um grupo conectado de pedras é completamente cercado por pedras de outra cor como mostrado na figura ??, elas são consideradas aprisionadas e são removidas do tabuleiro. O jogo termina quando ambos os jogadores passam a vez consecutivamente. Neste ponto o escore do jogador é o tamanho do seu território, isto é, o número de espaços vazios cercado pelas suas pedras, menos o número de pedras que foram aprisionadas durante o jogo. O jogador com maior escore vence.

Existe apenas mais uma regra. A figura ?? mostra uma situação comum chamada um *ko*, a palavra japonesa para eternidade. As Pretas acabaram de tomar uma das pedras Brancas. As Brancas poderiam retalhar jogando a pedra novamente, e capturando a pedra Preta. Mas então as Pretas poderiam jogar como eles fizeram antes, e o jogo prosseguiria dessa forma indefinidamente. Para prevenir tal laço infinito, a regra ko proíbe que as Brancas respondam imediatamente tomando a pedra Preta, Ao invés, elas tem de jogar em algum outro lugar no tabuleiro. Em sua forma básica, a regra *ko* estabelece que nenhum movimento pode retornar o tabuleiro para o estado anterior.

Qual é a complexidade computacional de Go? Tradicionalmente, este é jogado em um tabuleiro de 19 x 19. Existem finitamente muitas posições, assim nós poderíamos construir uma tabela que contivesse o movimento correto para toda posição. A tabela demandaria somente $O(1)$ bits de memória, assim Go é, desse ponto de vista, bastante fácil. Infelizmente, quando nós dizemos $O(1)$ isto significa $3^{19^2} \approx 10^{172}$.

Figura 1.39: Ladder - Jogo Go

Uma vez que este tipo de abordagem não é viável, vamos abordar o jogo de Go para tabuleiros de tamanho arbitrários. Isto resulta no seguinte problema:

Problema Go
Entrada: Uma posição em tabuleiro n x n
Pergunta: As Pretas tem uma estratégia vencedora nesta posição?

Nós mostraremos que

$$\textit{Sat Quantificado} \leq_K \textit{Go}.$$

Isto implica que Go é $PSpace$-hard, isto é qualquer problema em $PSpace$ pode ser reduzido à Go. Se Go estivesse em $PSpace$, então este seria $PSpace$-completo. No entanto, nós veremos que este é mais difícil que isto.

Nós nos concentraremos em um tipo especial de jogada chamada ladder, mostrada na figura 1.39. O ladder começa quando as Pretas jogam em 1 e ameaçam capturar a pedra Branca. As Brancas podem sobreviver jogando em 2, 4, 6 e assim sucessivamente, mas as Pretas mantêm a pressão com movimentos em 3, 5, e 7. Durante todo o processo as Pretas mantêm as Brancas em atari, uma ameaça imediata de captura.

Uma vez que o ladder começa, o destino da pedra Branca depende do que o ladder atinge. Se este encontra uma pedra preta ou a borda do tabuleiro, então o grupo inteiro de pedras Brancas é perdido. Mas se este encontra uma pedra branca, este reforço permite as Brancas escaparem como mostrado na figura ??.

Mas suponha que o ladder das Pretas ataca um grupo grande de pedras brancas. Suponha que as Pretas falham em manter as Brancas em atari, as Brancas terão tempo de fazer um movimento crucial que elimina a ameaça das Pretas e assegura as pedras brancas. Um exemplo de tal situação é mostrada na figura ??. Se o ladder é grande o bastante o destino inteiro do jogo dependerá do que ocorrer com o ladder.

Quem sabe o que aparecerá aqui.

1.10 Questão Fundamental: P x NP

> "A evidência a favor da hipótese de Cook é tão forte, e as consequências de sua falha são tão grotescas, que seu status pode ser comparada a aquela das leis físicas, ao invés de uma conjectura matemática ordinária"
> Volker Strassen

A questão se $P = NP$ é a questão central da teoria da computação. Esta é amplamente reconhecida como um dos problemas mais profundas ainda não resolvido de toda matemática.

A questão $P \times NP$ é a respeito da natureza do processo de resolução de problemas. Uma vez que P é classe dos problemas onde nós podemos encontrar soluções eficientemente, e NP é a classe dos problemas onde nós podemos verificá-las, nós podemos expressar isso como:

É mais difícil encontrar soluções do que verificá-las?

Nossa intuição diz que sim. É difícil encontrar uma agulha em um palheiro, ou afirmar que uma existe, mas é muito fácil furar o dedo em uma agulha. Mas provar esta intuição revelou-se uma tarefa muito difícil.

Se $P = NP$ nós seriamos forçados a revisar o que nós pensamos saber, não só a respeito de computação, mas a respeito de verdade, criatividade e inteligência.

O que acontece se $P = NP$?

A forma mais corriqueira de expressar a questão $P = NP$ é perguntar se *Sat - Coloração de Grafo* ou *Clique* ou *Jantar Real* - pode ser resolvido em tempo polinomial. Do ponto de vista de um projetista de chip de computador ou um organizador de festas, é bastante frustrantes que estes problemas pareçam ser exponencialmente difíceis. Talvez nós simplesmente não fomos espertos o bastante. Talvez exista alguma ideia matemática que coloque um desses problemas, e portanto todos eles, dentro da classe P.

Se $P = NP$ as consequências seriam muito maiores que meramente a capacidade de planejar festas melhores ou uma forma mais fácil de azulejar banheiros. Nós mostraremos que o cenário da complexidade computacional seria profundamente alterado.

Relembre que a classe $coNP$ é a classe dos problemas, tais como como problema Não Hamiltoniano, onde existe um certificado que nós podemos verificar em tempo polinomial quando a resposta é "Não". Mostre que se $P = NP$ então $P = coNP$.

Além de NP e $coNP$, existe uma torre infinita de classes de complexidade, da qual P e NP são apenas os dois primeiros níveis. Os problemas em NP correspondem a propriedades da forma $\exists w : B(x, w)$, onde w é o certificado e B está em P. Similarmente, problemas em $coNP$ correspondem a propriedades da forma $\forall w : B(x, w)$. Assim NP ou $coNP$ podem ser vistos como P com um único $\exists$ ou $\forall$em frente deste.

Nós podemos definir classes de complexidade mais altas com números crescentes de quantificadores. Considere o seguinte problema:

Problema Menor Circuito Booleano
Entrada: Um circuito Booleano C que computa uma função f_c de sua entrada.
Pergunta: C é menor circuito que computa f_c ?

Nós podemos expressara a afirmação que C é o menor circuito que computa f_c da seguinte forma:

$$\forall C^* < C : \exists x : f_{c^*}(x) \neq f_c(x).$$

Nós podemos pensar nessa fórmula como um jogo com dois movimentos. Primeiro, Arthur escolhe um circuito C^* menor que C, e afirma que este computa a mesma função que C. E Então Merlin responde com uma entrada x tal que $f_{c^*}(x) \neq f_c(x)$. A declaração é verdadeira se Merlin possui uma estratégia vitoriosa - se ele pode demonstrar que todo circuito possível C^* falha sobre alguma entrada.

Uma vez que *Menor Circuito Booleano* pode ser expresso com dois quantificadores, um $\exists$ dentro de um $\forall$, este problema está na seguinte classe de complexidade:

Definição 1.10.1 *Classe $\Pi_2 P$ é a classe das propriedades A da forma*

$$A(x) = \forall y : \exists z : B(x, y, z),$$

onde B está em P, e onde $|y|$ e $|z|$ são polinomiais em $|x|$.

Neste caso, y e z são strings que codificam circuitos Booleanos e entradas, respectivamente, e cada uma destas strings é mais curta que a descrição do circuito C.

Definição 1.10.2 *Continuando dessa forma, nós podemos definir classes com k camadas de quantificadores, alternando-se entre $\exists$ e $\forall$. A classe $\Sigma_k P$ é a classe das propriedades A da forma*

$$A(x) = \exists y_1 : \forall y_2 : \exists y_3 : \ldots : Q y_k : B(x, y_1, \ldots, y_k),$$

onde B está em P, e onde $|y_i|$ é polinomial em $|x|$, para todo i, e $Q = \exists$ ou $\forall$ se k é ímpar ou par, respectivamente. Similarmente, a classe $\Pi_k P$ é classe das propriedades de A da forma

$$A(x) = \forall y_1 : \exists y_2 : \forall y_3 : \ldots : Q y_k : B(x, y_1, \ldots, y_k),$$

onde B está em P, e onde $|y_i|$ é polinomial em $|x|$, para todo i, e $Q = \forall$ ou $\exists$ se k é ímpar ou par, respectivamente.

A notação $\Sigma_k P$ espelha o fato que o quantificador mais externo $\exists y_1$ é um Or , ou uma soma, sobre todos os possíveis y_1. Similarmente, $\Pi_k P$ espelha o fato de que o quantificador mais externo é um $\forall y_1$, ou um produto, sobre todos os possíveis y_1.

Problemas nestas classes correspondem a jogos com dois jogadores e terminam em k movimentos. Por exemplo, se um problema de xadrez afirma que as Brancas podem dar mate em k movimentos, isto significa que existe um movimento para as Brancas, tal que para todos as respostas das Pretas, existe um movimento para as Brancas, e assim sucessivamente até que as Brancas vençam. Dado a posição inicial e a sequência de

movimentos que os jogadores fazem, nós podemos verificar em tempo polinomial se as Brancas deram cheque mate em seu oponente. Assim se nós estamos jogando xadrez generalizado sobre um tabuleiro $n \times n$, este tipo de afirmação está em $\Sigma_k P$.

Aumentando o número de quantificadores nós podemos fazer essas classes somente mais amplas - nós podemos sempre adicionar um quantificador sobre uma variável artificial, dentro ou fora de todos os quantificadores existentes. Portanto, para todo $k \geq 0$ nós temos

$$\Sigma_k \subseteq \Sigma_{k+1}; \Sigma_k \subseteq \Pi_{k+1}, \Pi_k \subseteq \Sigma_{k+1}, \Pi_k \subseteq \Pi_{k+1}.$$

Nós podemos pensar em cada $\exists$ como uma camada de não determinismo, esta camada pergunta se existe um certificado que faz a declaração dentro do quantificador verdadeira. Em analogia à N em NP, nós podemos escrever

$$\Sigma_k P = N\Pi_{k-1} P.$$

Em particular, uma vez que $\Sigma_0 P = \Pi_0 P = P$ nós temos

$$\Sigma_1 P = NP \text{ e } \Pi_1 P = coNP.$$

De uma forma mais geral

$$\Pi_k P = co - \Sigma_k P,$$

uma vez que a negação de um $\forall$ é um $\exists$ e vice-versa. Por exemplo, a declaração que C é o menor circuito possível que computa f_c pode ser expresso como

$$\begin{aligned} & \overline{\forall C^* < C : \exists x : f_{C^*}(x) \neq f_C(x)} \\ = \quad & \exists C^* < C : \overline{\exists x : f_{C^*}(x) \neq f_C(x)} \\ = \quad & \exists C^* < C : \forall x : f_{C^*}(x) = f_C(x), \end{aligned}$$

dessa forma o problema Não É Menor Circuito Booleano está em $\Sigma_2 P$. Similarmente a afirmação que as Brancas não podem dar mate em k movimentos pertence à classe $\Pi_k P$.

Estas classes de complexidade são conhecidas, coletivamente, como a hierarquia polinomial. Tomando sua união sobre todos os valores de k resulta na classe PH, que consiste de problemas que podem ser descritos com qualquer número de quantificadores:

$$PH = \bigcup_{k \geq 0} \Sigma_k P = \bigcup_{k \geq 0} \Pi_k P.$$

Tal como nós acreditamos que $P \neq NP$, e que $NP \neq coNP$, nós também acreditamos que as classes Σ_k e Π_k são todas distintas - elas formam uma

hierarquia infinita de classes de complexidade crescente cada vez mais poderosas.

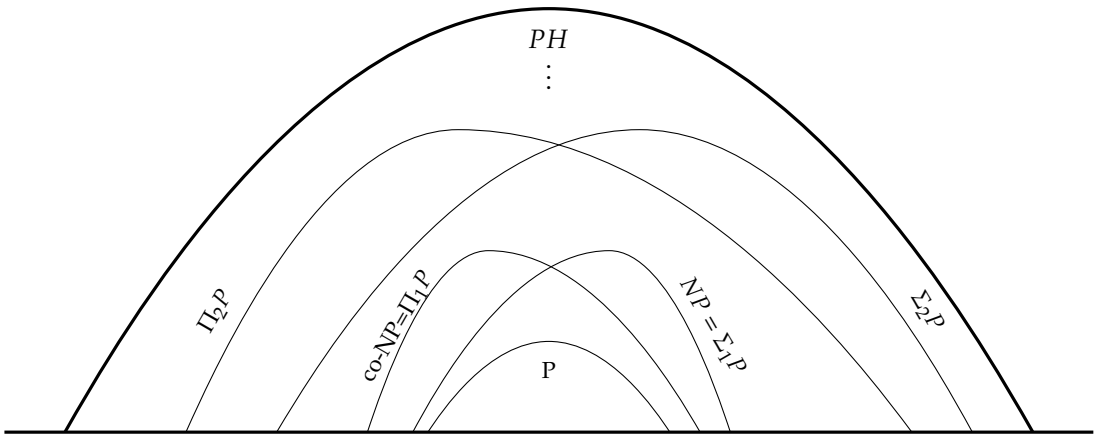


Figura 1.40: Hierarquia Polinomial

Isto é, nós acreditamos que cada vez que nós acrescentamos um quantificador, ou uma camada de não determinismo, nós obtemos uma espécie de problema mais profundo. Enquanto que problemas em NP demandam certificados de tamanho polinomial, problemas em $\Sigma_k P$ e $\Pi_k P$ demandam estratégias vencedoras com k movimentos de profundidade, as quais são objetos muito maiores. Mesmo o problema Menor Circuito Booleano demanda que Merlin tenha, para qualquer circuito menor C^*,uma entrada x sobre a qual C^* obtenha a resposta errada. Uma vez que existem um número exponencial de tais circuitos, a descrição de tal estratégia vencedora demandará um número exponencial de bits para ser descrita.

Mas eis que para nossa surpresa e espanto, se $P = NP$ então esta torre infinita de classes de complexidade colapsa para P!!!! Como isto é possível? Note que $P = NP$, e o mesmo que dizer que se $B(x, y)$ está em P, então $A(x) = \exists y : B(x, y)$ também está em P.

Dessa forma o quantificador existencial pode ser absorvido. Mas uma vez que $P = NP$, também implica que $P = coNP$, então o quantificador universal também pode ser absorvido.

Dessa forma o quantificador mais interno foi absorvido, e nós reduzimos o número de quantificadores. Nós podemos proceder dessa forma até que todos os quantificadores tenham sido absorvidos, restando apenas uma propriedade de tempo polinomial que a entrada x possui ou não possui. E a a torre de Babel de classes de problemas ruiu. O que nós devemos considerar extremamente surpreendente, e funciona como um argumento a favor da hipótese que $P \neq NP$.

Um erro comum é acreditar que a questão $P \times NP$ é a respeito de tempo polinomial. Na realidade esta é uma pergunta a respeito da potência do não determinismo, e se encontrar soluções são mais difíceis que verificá-las. Se $P = NP$ então $Tempo(f(n)) = NTempo(f(n))$ para qualquer função $f(n)$ que cresça pelo menos polinomialmente.

Recorde-se que Exp é a classe dos problemas que podem ser resolvidos em tempo exponencial, e que $NExp$ é a classe dos problemas que podem ter uma solução ou certificado verificado em tempo exponencial. Então

em analogia a questão $P \neq NP$, nós podemos perguntar se $Exp \neq NExp$, e assim sucessivamente a medida que nós subimos na hierarquia.

No entanto, estas questões não são independentes uma da outra. Pode-se provar que:

Se $P = NP$, então $Exp = NExp$, $ExpExp = NExpExp$, e assim sucessivamente.

A prova é muito simples. Lembre-se que uma vez que a complexidade de um problema é medida como uma função do tamanho da entrada, problemas difíceis tornam-se fáceis se nós expressamos a entrada de uma forma ineficiente. Preenchendo a entrada para fazê-la mais longa, nós podemos transladar uma função em $NExp$ para uma em NP. Nós podemos então resolvê-la deterministicamente em tempo polinomial como uma função de seu novo tamanho preenchido - que é exponencial como uma função do seu tamanho original.

Por exemplo, suponha que um problema A está em $NExp$, de modo que certificados possam ser verificados em tempo $t(n) = 2^{O(n^c)}$ para alguma constante c. Agora preencha a entrada, fazendo-a ter $t(n)$, adicionando $t(n) - n$ zeros. O problema resultante A' está em NP, uma vez que o certificado pode ser verificado em tempo $t(n)$, linear no tamanho da entrada. Mas se $P = NP$, então A' pode ser resolvido de forma determinística em tempo polinomial em $t(n) = 2^{O(n^c)}$. Assim A está em Exp, e $Exp = NExp$.

1.10.1 A Extinção da Magia

A consequência mais profunda de $P = NP$ é que:

> a pesquisa por conhecimento, com toda a sua carga de frustração e alegria, seria muito mais fácil do que nós acreditamos

Encontrar provas de conjecturas matemáticas, ou dizer se uma prova existe, parece ser uma tarefa muito difícil. Por outro lado, verificar uma prova não é, desde que esta esteja escrita em uma linguagem formal cuidadosa. Realmente, nós inventamos sistemas formais como a geometria Euclidiana, ou a teoria dos conjuntos de Zermelo-Fraenkel, precisamente para reduzir provas a uma série de passos simples, como, por exemplo, modus ponens, "Se A e $(A \Rightarrow B)$ então B." Isto torna fácil verificar que cada linha da prova segue da linha anterior, e nós podemos verificar a prova inteira em tempo polinomial como uma função do seu tamanho. Dessa forma o seguinte problema está em P:

Problema VERIFICANDO UMA PROVA
Entrada: Uma declaração S e uma prova P.
Pergunta: P é uma prova válida de S?

Isto implica que o seguinte problema está em NP:

Problema PROVA CURTA
Entrada: Uma declaração S e um inteiro n dado em unário.
Pergunta: S tem uma prova de tamanho n ou menor?

Note que nós damos o tamanho da prova em unário, uma vez que o tempo de VERIFICANDO UMA PROVA é polinomial como uma função de n. Note também que PROVA CURTA é $NP-$completo, uma vez que S poderia ser a declaração que uma dada fórmula *Sat* é satisfatível.

Agora suponha que $P = NP$. Você pode considerar o seu problema matemático favorito ainda não resolvido, por exemplo, a hipótese de Riemann, a conjectura de Goldbach, e utilizar seu algoritmo de tempo polinomial para Prova Curta para pesquisar por soluções menores que um determinado tamanho, digamos, um bilhão de símbolos. O ponto crucial é que nenhuma prova construída pelo homem será maior que um bilhão de símbolos, mesmo quando nós fazemos isto de uma forma formal. Assim, se tal prova não existe, nós não temos esperança de encontrá-la.

Esta questão foi levantada antes mesmo que as classes P e NP fossem definidas. Em 1956 o logicista Kurt Gödel escreveu uma carta para John von Neumann[15]. Turing havia mostrado, no artigo seminal,que o *Entscheidungsproblem*, o problema de decidir se um determinado declaração matemática possui uma prova, é indecidível, isto é, esta questão não pode ser resolvida por qualquer programa [18]. Em sua carta Gödel considera uma versão limitada desse problema, onde nós perguntamos se existe uma prova de tamanho n ou menor.

[15]: Lipton et al. (2013), *People, Problems, and Proofs - Essays from Gödel's Lost Letter: 2010*

[18]: Turing (1936), *On Computable Numbers, with an Application to the Entscheidungsproblem*

Pode *Prova Curta* realmente ser fácil? Matemáticos gostam de acreditar que eles necessitam utilizar todas as ferramentas disponíveis para encontrar o caminho que conduz a uma prova curta. Mas se $P = NP$, então encontrar provas não é mais difícil do que verificá-las, e existe um algoritmo de tempo polinomial que faz o trabalho árduo de verificação.

definidas por recursão geral:

A consequência de $P = NP$ não seria limitada à matemática. Cientistas em vários campos diferentes do conhecimento batalham com o seguinte problema:

Problema TEORIA ELEGANTE
Entrada: Um conjunto E de dados experimentais e um inteiro n dado em unário.
Pergunta: Existe uma teoria T de tamanho nou menor que explica E?

E poderia ser, por exemplo, dados astronômicos, T poderia ser um modelo matemático do movimento planetário, e T poderia explicar E com uma determinada precisão. Uma teoria elegante é uma cujo tamanho n, definido como o número de símbolos que esta utiliza em uma linguagem formal, é bem pequena, tal como a lei da gravitação de Newton.

Vamos assumir que nós podemos computar, em tempo polinomial

1.10.2 Limites Superiores são Fáceis, Limites Inferiores são Difíceis

A estratégia direta para provar que $P \neq NP$ é provar que existe um problema que pertence à NP mas não está em P. Como nós podemos fazer isso?

Um dos temas recorrentes na ciência da computação é que limites inferiores sobre a complexidade de um problema são muito mais difíceis de provar que limites superiores. Para provar que um problema está em P tudo o que nós temos que fazer é mostrar um algoritmo de tempo polinomial que o resolve. Mas para provar que este não está em P, nós temos que raciocinar à respeito de todos os algoritmos de tempo polinomial possíveis, e provar que cada um deles falha de alguma forma.

Nós poderíamos conceber uma forma de provar que um determinado problema A não pode ser resolvido em tempo polinomial pela técnica da programação dinâmica ou por um algoritmo guloso. Mesmo isso seria muito desafiador, uma vez que existem muitas formas pela qual programação dinâmica pode ser organizada, e muitos tipos de escolhas que um algoritmo guloso poderia fazer. Mas mesmo se nós pudéssemos fazer isso, ainda assim isto não provaria que A está fora de P. Depois de tudo, a lista de estratégias é incrivelmente longa.

Nós podemos provar alguns limites inferiores mostrando que um problema está fora de uma certa classe de complexidade. No entanto, nós podemos mostrar que nenhuma dessas técnicas é poderosa o bastante para provar que $P \neq NP$. De fato, nós descreveremos "metateoremas" negativos que eliminam várias abordagens para a questão $P \times NP$. Atualmente existem poucas ideias que parecem ter esperança de resolver a questão.

(..... Aqui entra um texto a respeito do limite inferior sobre ordenação .....)

Se você considera uma fórmula 3-*Sat* como uma caixa preta na qual nós inserimos atribuições lógicas, e e que simplesmente responde "satisfeita" ou "insatisfeita" por alguma razão que nós desconhecemos, então nós não temos nenhuma escolha a não ser tentar todas as 2^n atribuições até encontrar uma que funcione.

No entanto, este ponto de vista é muito pessimista. Nós podemos analisar a fórmula e entender como as cláusulas interagem, pelo menos em um nível local, e utilizar este conhecimento para guiar nossa pesquisa. Dessa forma, existem algoritmos que resolvem 3-*Sat* em tempo $O(n^c)$ para uma constante $c < 2$. Estes algoritmos demandam tempo exponencial, mas eles são exponencialmente mais rápidos que uma pesquisa ingênua. Assim enquanto que nós podemos considerar os problemas $NP-$completos como sendo uma pesquisa em um palheiro, na realidade nós não estamos totalmente sem ajuda, afinal o palheiro pode ser muito arrumado, e bons algoritmos podem tirar partido dessa arrumação.

1.10.3 Diagonalização e a Hierarquia de Tempo

Provar que um problema particular não pode ser resolvido em tempo polinomial parece ser um problema muito desafiador. No entanto, nós podemos construir problemas artificiais que estão fora da classe P, de modo que qualquer algoritmo dará a resposta errada pelo menos para um caso.

Nós utilizamos esta abordagem para provar que tempo polinomial é menos poderoso que tempo exponencial, isto é, $P \subset ExpTime$. Mais geralmente, nós mostraremos que $Time(g(n)) \subset Time(f(n))$, quando $f(n)$ é grande o bastante quando comparado à $g(n)$.

Vamos começar escolhendo uma função de complexidade de tempo $f(n)$. Agora nós construiremos um problema que captura todos os problemas que podem ser resolvidos em tempo $f(n)$. A ideia é similar a aquela utiliza para construir o nosso primeiro problema $NP-$completo Existência de Certificado, recorde que este problema pergunta s existe uma entrada que faz um determinado programa Π retornar "sim" . No caso atual, nós perguntamos como Π responderá a uma determinada entrada x de tamanho n,se este programa é permito executar por no máximo $f(n)$ passos.

Problema Prever$(\Pi,\ x)$
Entrada: Um programa Π *e uma entrada* x.
Pergunta: Se Π *para dentro de* $f(|n|)$ *passos quando recebe* x *como entrada, então retorne sua saída* $\Pi(x)$. *Caso contrário retorne "não sei."*

Note que a função f é fixa, ao invés de ser parte da entrada. Observe também que o problema *Prever* prediz o comportamento de Π por precisamente $f(|x|)$ passos ou menos, como oposto a algum tempo constante $f(|x|)$. Para isto estar bem definido, nós temos de escolher uma linguagem especifica, assim nós sabemos o que conta como um passo único. Vamos assumir que isto foi feito.

É claro, que nos podemos resolver *Prever* em $f(|x|)$ passos, simulando Π e vendo o que este faz. Mas é concebível, a priori, que exista uma outra forma de resolver *Prever*, talvez por algum atalho que permita ver o futuro e predizer a saída de Π sem realmente executá-lo. Nós mostraremos que isto é impossível, explicitando um paradoxo onde um programa tenta prever a sua própria saída.

Primeiro nós modificamos o problema *Prever* aplicando um programa a seu próprio código fonte, e *"invertendo"* o resultado.

Problema Prever42(Π)
Entrada: Um programa Π.
Pergunta: Se Π para dentro de $f(|\Pi|)$ passos quando recebe o seu próprio código fonte como entrada, retorne a negação da sua saída, $\overline{\Pi(\Pi)}$. Caso contrário, responda "não sei."

1.11 Provas Interativas

Uma das motivações para a classe NP é capturar a ideia de "provabilidade eficiente". Se um problema $A \in NP$ e x é uma instância "sim" de A então existe um certificado curto para este fato o qual pode ser verificado em tempo polinomial. Pela definição da classe NP todas as provas são corretas e Merlin sempre pode convencer o rei Arthur de que ele não está mentindo. O que acontece se nós introduzimos aleatoriedade e decrescemos a exigência de que a prova seja sempre correta. Neste contexto a prova será uma conversa entre Merlin (o provador todo poderoso) e o rei Arthur (o verificador polinomial), onde este último executa escolhas probabilísticas. A Figura 1.41 representa a interação entre Merlin e o Rei.

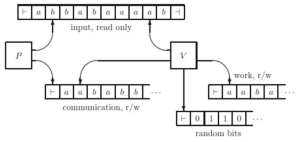


Figura 1.41: Interação entre Merlin e o rei Arthur no protocolo IP

[3]: Arora et al. (2009), *Computational Complexity - A Modern Approach*

Antes de prosseguirmos é interessante considerar um exemplo lúdico tomado emprestado de [3] "Merlin possui um meia vermelha e uma amarela, mas seu amigo Arthur, que é incapaz de distinguir cores, não acredita que as meias tenham cores diferentes. Como Merlin pode convencê–lo que este é realmente o caso?

Aqui está uma forma de fazer isso. Merlin dá ambas as meias para Arthur, e lhe diz qual meia é amarela e qual é vermelha, e Arthur segura a meia vermelha em sua mão direita e a meia amarela na mão esquerda. Então Merlin virá de costas para Arthur e ele arremessa uma moeda. Se a moeda cair "cara" então Arthur mantêm as meias como elas estão, e caso contrário ele troca as meias de mão. Ele então pergunta a Merlin para adivinhar se ele trocou as meias ou não. É claro que Merlin pode fazer isso facilmente vendo se a meia vermelha ainda está na mão direita de Arthur ou não. Mas se as meias forem idênticas então ela não seria capaz de adivinhar a resposta com probabilidade melhor que 1/2. Assim se Marla responde corretamente em todas as, digamos, 100 repetições deste jogo, então Arthur pode realmente ser convencido que as meias tem cores diferentes.

Como observado por Arora e Barak o fato de que Arthur é incapaz de distinguir cores torna-o menos poderoso que o provador. Em protocolos de provas interativas o verificador - sendo de tempo polinomial - também o torna menos poderoso que o provador.

Outro exemplo. Considere dois grafos G_1 e G_2 ambos com n vértices. Se nós permitimos um conversa entre Merlin e Arthur será que o mago conseguirá convencer o rei, com grande probabilidade, que os dois grafos não são isomórficos?

Note que o problema do isomorfismo em grafos pertence à classe NP, mas nós não sabemos se este pertence a classe P (ou a classe $co-NP$) nem

sabemos que este é $NP-$completo. Agora considere o seguinte protocolo para o diálogo entre o rei Arthur e o mago Merlin.

```
for M ← 1 to k
    O rei escolhe aleatoriamente i ∈ 1, 2
    faz H ser uma permutação aleatória de G_i
    mostra H para Merlin. E pergunta se foi G_1 ou G_2 que gerou H
    Merlin responde j
    if j == i
        Arthur diz mentiroso e para
    Arthur aceita
```

Nesse protocolo Merlin tenta convencer o rei que os grafos não são isomórficos. Para que o rei fique convencido é necessário que Merlin (o provador) sempre acerte qual grafo resultou em H. Suponha que G_1 e G_2 não são isomórficos. Então H somente a um dos dois grafos G_1 ou G_2, e uma vez que Merlin é um mago todo poderoso, sempre responderá corretamente a pergunta do verificador Arthur. Por outro lado, se G_1 e G_2 são isomórficos então qualquer um dos dois grafos pode ter originado H. Portanto, não existe forma de Merlin saber qual dos dois grafos foi escolhido pelo rei para gerar H. Tudo que Merlin pode fazer é "chutar" um valor para i, e torcer para acertar. A probabilidade de Merlin acertar $\frac{1}{2}$. Assim a probabilidade de Merlin enganar o rei é 2^{-k} que é muito pequena se k é grande.

Definição 1.11.1 *Um problema A admite uma prova interativa se e somente se existe uma interação entre um verificador probabilístico de tempo polinomial, que nós chamamos de rei Arthur, e um provador todo poderoso, que nós chamamos de Merlin, tal que:*

- *Se x é uma instância "sim" de A então a probabilidade, de acordo com as escolhas aleatórias do rei Arthur, que o rei aceite esta interação como uma prova é pelo menos $2/3$.*
- *Se x é uma instância "não" de A então não importa o que Merlin faça a probabilidade, novamente acordo com as escolhas aleatória do rei, é que este engane o rei é no máximo $1/3$.*

Nós podemos agora definir a classe de complexidade IP.

Definição 1.11.2 *O conjunto de problemas que admitem provas interativas formam a classe de complexidade IP.*

O número de interações entre o provador e o verificador pode depender do tamanho da entrada, mas uma vez que nós demandamos que processo inteiro demande tempo polinomial, o número de interações deve ser limitado a um número polinomial com relação ao tamanho da entrada.

Vamos considerar agora um exemplo de um problema que pertence à classe IP.

Problema #SAT
Entrada: Uma fórmula Booleana $\phi(x_1, \ldots, x_n)$, escrita em FNC e um inteiro positivo k.
Pergunta: ϕ possui exatamente k atribuições que a satisfazem?

Quais são as implicações do fato de que o problema #3SAT pertencer à classe IP? Em primeiro lugar nós devemos notar que 3SAT é um tipo especial de problema #Sat e 3SAT é NP−completo portanto NP ⊆ IP.

Considere agora a situação em que nós temos um caso especial de #3Sat na qual $k = 0$. Este problema consiste em decidir se uma fórmula ϕ é insatisfazível. Uma vez que o problema da insatisfazibilidade de uma fórmula booleana escrita em FNC é coNP−completo nós também temos que $coNP \subseteq IP$. Isto não é coincidência uma vez que nós podemos entender intuitivamente a classe IP como uma generalização da classe NP obtida permitindo-se randomização.

Contudo isto não é a história toda, uma vez que #3Sat é completo para uma classe de problemas chamados de $\#P$, que nós apresentaremos na seção ??, nós temos que $\#P \subseteq IP$. Mas a história ainda não está completa, pode-se provar que $IP = PSpace$, a classe dos problemas em que instâncias "sim" podem ser verificadas em espaço polinomial, e acredita-se que #P ⊆ PSPACE.

A seguir nós apresentamos um ideia extremamente poderosa chamada de aritmetização que será utilizada para provar que #SAT pertence à classe IP, mas também é crucial para provar que $IP = PSpace$.

A ideia da aritmetização remonta a Boole .... A ideia consiste em:

1. Associar os inteiros 1 e 0 com os valores lógicos verdadeiro e falso, respectivamente.
2. Reescrever uma expressão lógica $x \wedge y$ como xy.
3. Reescrever uma expressão $\overline{x}$ como $1 - x$.

Devido a regra de De Morgan nós temos que $x_1 \vee x_2$ é equivalente a $\overline{\overline{x_1} \wedge \overline{x_2}}$, portanto $x_1 \vee x_2$ pode ser reescrito como $(1 - ((1 - x_1)(1 - x_2))$, ou seja $x_1 + x_2 - x_1x_2$.

Como um exemplo considere a seguinte fórmula lógica

$$\phi = (\overline{x}_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \overline{x}_2 \vee \overline{x}_3), \tag{1.25}$$

esta será convertida no seguinte polinômio multivariável

$$\begin{aligned} P_\phi(x_1, x_2, x_3) &= (1 - (x_1(1 - x_2)(1 - x_3))(1 - ((1 - x_1)x_2x_3)) \\ P_\phi(x_1, x_2, x_3) &= -x_2^2x_3^2x_1^2 + x_2x_3^2x_1^2 + x_2^2x_3x_1^2 - x_2x_3x_1^2 + x_2^2x_3^2x_1 - x_2x_3^2x_1 + x_2x_1 - x_2^2x_3x_1 + x_3x_1 - x_1 + 1 \end{aligned}$$

Para uma fórmula lógica 3FNC ϕ função de n variáveis lógicas $x_1, \ldots, x_n$ por $P_\phi(x_1, \ldots, x_n)$ nós denotamos o polinômio multivariável de grau d obtido de acordo com as regras de reescrita definidas anteriormente.

Note que o número de atribuições que satisfazem uma fórmula 3FNC ϕ é dado pelo seguinte somatório

$$k = \sum_{x_1 \in \{0,1\}} \sum_{x_2 \in \{0,1\}} \cdots \sum_{x_n \in \{0,1\}} P_\phi(x_1, \ldots, x_n). \tag{1.26}$$

Por $P_i(x_1, \ldots, x_i)$, para $1 \leq i \leq n$, nós denotamos o seguinte polinômio

$$P_i(x_1, \ldots, x_i) = \sum_{x_{i+1} \in \{0,1\}} \cdots \sum_{x_n \in \{0,1\}} P_\phi(x_1, \ldots, x_n). \tag{1.27}$$

Nós temos as seguintes propriedades:

1. $P_0(x_1, \ldots, x_i)$ é igual ao número de atribuições que satisfaz a fórmula ϕ.
2. $P_n(x_1, \ldots, x_i) = P_\phi(x_1, \ldots, x_n)$.
3. Para $1 \leq i \leq n, P_{i-1}(x_1, \ldots, x_{i-1}) = P_i(x_1, \ldots, x_{i-1}, 0) + P_i(x_1, \ldots, x_{i-1}, 1)$.

Suponha agora que o rei queira saber se existem exatamente 6 atribuições que satisfazem ϕ. O rei poderia avaliar o somatório 1.26 de modo a determinar se o seu valor é igual à 6. Infelizmente isso implicaria em avaliar 2^n termos. Dessa forma o rei, mais uma vez, recorre ao velho mago Merlin.

O diálogo entre Arthur e Merlin poderia transcorrer da seguinte forma:

- Merlin: Eu afirmo que portanto ϕ é uma instância "sim" de #Sat com $k = 6$.

5 Arthur: Por que eu deveria acreditar nisso?
3) Merlin: Porque eu afirmo que $P_1(0) + P_1(1) = 6$, e $P_1(x_1) = 2 + 2x_1$.
4) Arthur: Se $P_1(x_1) = 2 + 2x_1$ então fazendo $x_1 = 7$ eu teria 16. Você pode provar isso?
5) Merlin: Claro! Nós temos que $P_1(7) = P_2(7, 0) + P_2(7, 1)$, onde $P_2(x_1, x_2) = 1 + x_1$.
6) Arthur: Eu não estou convencido. Se $P_2(x_1, x_2) = 1 + x_1$ então fazendo $x_1 = 7$ e $x_2 = 5$ o resultado seria 8. Você pode provar isso?
7) Merlin: Perfeitamente, meu rei. Nós temos que $P_2(7, 5) = P_3(7, 5, 0) + P(7, 5, 1)$, onde $P_3(x_1, x_2, x_3) = P_\phi = (1 - (1 - x_1)(1 - x_2)(1 - x_3))(1 - ((1 - x_1)x_2x_3)$. E isso sua majestade pode verificar por conta própria.
8) Arthur: Talvez você tenha tido apenas muita sorte. Vamos continuar conversando.

A ideia do diálogo é que Merlin pode convencer o rei de que ϕ é uma instância sim se ele puder convencê-lo que $P_1(0) + P_1(1)$ é igual ao valor 6. Agora o rei precisa ser convencido que $P_1(x_1)$ é igual ao polinômio declarado pelo mago. O rei será convencido disso se Merlin puder provar que $P_1(7)$ é igual à 16. Para fazer isso Merlin afirma que $P_2(x_1, x_2) = 1 + x_1$. O rei Arthur será convencido disso se Merlin puder provar que $P_2(7, 5)$ for igual à 8. Para provar esta última declaração Merlin clama que $P_2(7, 5) = P_3(7, 5, 0) + P_3(7, 5, 1)$, sendo que $P_3(x_1, x_2, x_3)$ é a tradução direta da fórmula ϕ, tradução esta que o rei pode fazer sem a ajuda de Merlin. Portanto, o rei pode verificar, por conta própria, os valores de $P_3(7, 5, 0)$ e $P_3(7, 5, 1)$.

Se Merlin não está mentindo ele sempre pode convencer o rei Arthur. Tudo o que ele tem que fazer é sempre entregar os polinômios corretos, o que ele pode fazer.

O que acontece se Merlin tentar enganar o rei, ou seja afirmar que ϕ é uma instância "sim" de #Sat quando na verdade esta é uma instância "não"?

Para evitar que o rei imediatamente descubra o engodo Merlin tem que declarar que o valor de $P_1(0) + P_1(1)$ é igual a 6, e enviar para o rei um polinômio *falso,* que nós chamaremos de $\tilde{P}_1(x_1)$, que satisfaça isso. Agora nós chegamos a um momento crucial da prova.

Quando o rei escolhe aleatoriamente um valor c para a variável x_1, é improvável que o valor de $\tilde{P}_1(c)$, seja igual ao valor do polinômio verdadeiro $P_1(x_1)$ no ponto c. Isto deve-se ao fato que um polinômio univariável de grau no máximo d pode ter no máximo d raízes. Portanto, quaisquer dois polinômios univariáveis, não idênticos, de grau máximo d podem concordar em no máximo d valores.

Nós temos que o grau do polinômio $P_1(.)$ é no máximo m, o número de cláusulas de ϕ, se Merlin enviar um polinômio de grau maior que m o rei descobrirá imediatamente que ele está mentindo. Como Merlin está tentando encanar o rei nós sabemos que os polinômios $P_1(x_1)$ e $\tilde{P}_1(x_1)$ não são idênticos, logo eles só podem concordar em no máximo m pontos. O rei escolheu, aleatoriamente, um valor para x_1 que é maior que 2^m, logo a probabilidade de que o valor escolhido seja um dos m valores nos quais os dois polinômios coincidem é no máximo $m/2^m$, que é menor que m^{-2} para $m \geq 10$. Portanto,

$$\Pr[P_1(c) = \tilde{P}_1(c)] < \frac{1}{m^2}. \tag{1.28}$$

Merlim clama que $\tilde{P}_1(c)$ é o valor correto, e que este valor é igual a $P_2(c, 0) + P_2(c, 1)$. Note que$P_2(c, x)$ é um polinômio em x_2 de grau máximo m. O rei agora escolhe um valor aleatório d para a variável x_2. E pede para Merlin provar que o valor correto de $P_2(c, d)$ é àquele que ele obtêm avaliando $\widetilde{P_2}(c, d)$. Merlim agora afirma que $\widetilde{P_2}(c, d)$ é o valor correto porque este é igual a $\widetilde{P_3}(c, d, 0) + \widetilde{P_3}(c, d, 1)$. Uma vez que $P_3(x_1, x_2, x_3)$ é igual à $P_\phi(x_1, x_2, x_3)$ o rei pode verificar diretamente se os valores $\widetilde{P_3}(c, d, 0)$ e $\widetilde{P_3}(c, d, 1)$ são os valores corretos.

O rei será enganado por Merlin se e somente se $P_\phi(c, d, 0)$ e $P_\phi(c, d, 1)$ forem iguais aos valores $\widetilde{P_3}(c, d, 0)$ e $\widetilde{P_3}(c, d, 1)$, respectivamente. A probabilidade disso ocorrer é menor que $1/m^2$.

Para terminar a prova que #Sat $\in IP$ nós temos que mostrar que o rei Arthur executa em tempo polinomial probabilístico, mas isso é direto do protocolo.

1.12 Exercícios

1. Um triângulo em um grafo não dirigido é um clique de tamanho três. Mostre que o seguinte problema pertence à classe P

Problema TRIÂNGULO
Entrada: Um grafo não dirigido G.
Pergunta: Existe um triângulo em G?

2. Considere o seguinte problema:

   Problema EXPONENCIAÇÃO MODULAR
   Entrada: Inteiros x, y, w e p, representados em decimais, cada um tendo n dígitos.
   Pergunta: $x^y \, mod \, p$ é igual à w mod p?

   Prove que esse problema pertence à classe P. (Note que o algoritmo óbvio não executa em tempo polinomial. Assuma que y é uma potência de 2.)
3. Suponha que exista um oráculo em uma próxima à você. O oráculo lhe dirá, pelo preço de um dracma por pergunta, se um grafo possui um ciclo Hamiltoniano ou não. Mostre que perguntando-lhe uma série de questões, talvez envolvendo uma versão modificada do grafo original, você pode encontrar um ciclo Hamiltoniano gastando um número de dracmas que cresce polinomialmente como uma função do número de vértices.
4. Novamente você tem que consultar o oráculo de Delfos. Mas dessa vez você deve determinar uma atribuição que satisfaça uma fórmula proposicional ϕ, escrita em FNC, se tal atribuição existir. Você só pode gastar um número de dracmas que cresce polinomialmente como uma função do tamanho da fórmula. Dica: Descreva como fixar uma variável resulta em uma fórmula menor nas variáveis restantes.
5. Suponha que o mesmo oráculo possa resolver o problema de decisão $k-$COLORAÇÃO DE GRAFOS. Mostre que perguntando um número polinomial de questões, você pode encontrar uma $k-$coloração se ela existir. Dica: Como nós podemos modificar o grafo para indicar que dois vértices são da mesma cor, ou que eles são de cores diferentes?
6. Escreva um programa não determinístico para 3-ESAT em sua linguagem de programação favorita aumentada com a instrução "goto ambas".
7. Mostre que se um programa não determinístico executa somente $O(lg\ n)$ instruções "goto ambas" sobre cada ramo da computação, então o problema de este programa resolve está em P.
8. Considere o seguinte problema de otimização

   Problema MÁXIMO CONJUNTO INDEPENDENTE
   Entrada: Um grafo não dirigido G
   Saída: Um conjunto independente S de máximo tamanho do grafo G

   Prove que se o problema de decisão CONJUNTO INDEPENDENTE pertencer à classe P então o problema de otimização MÁXIMO CONJUNTO INDEPENDENTE também pertence à P.
9. Para cada uma das duas questões abaixo, decida se a resposta é (i) "Sim", (ii) "Não", ou (iii) "Desconhecido, porque isso resolveria a questão se $P = NP$." Dê uma breve explicação para sua resposta.
   Considere o seguinte problema:

Problema ESCALONAMENTO DE INTERVALOS
Entrada: Uma coleção de intervalos sobre uma linha do tempo, e um inteiro k.
Pergunta: A coleção contem um subconjunto de intervalos que não se sobrepõem de tamanho pelo menos k?

a) Questão: É o caso que ESCALONAMENTO DE INTERVALOS $\leq_K$ *COBERTURA POR VÉRTICE*?
b) Questão: É o caso que *CONJUNTO INDEPENDENTE*$\leq_K$ *ESCALONAMENTO DE INTERVALOS?*

10. Um supermercado está tentando analisar o comportamento de seus clientes. Para isso o supermercado mantêm uma matriz bidimensional A, onde as linhas representam os clientes e as colunas correspondem aos produtos que são vendidos. A entrada $A[i, j]$ especifica a quantidade do produto j comprada pelo cliente i. Abaixo nós temos um exemplo minúsculo da matriz A.

	Detergente	Cerveja	Barbeador	Ração para Cachorro
Ana	2	0	0	3
Guilherme	2	3	1	0
Marina	0	6	0	7

Uma das coisas que o supermercado pode querer fazer com esses dados é a seguinte. Vamos dizer que um subconjunto S dos clientes é diverso se nenhum par de clientes em S comprou o mesmo produto (isto é, para cada produto, no máximo um dos clientes em S o comprou.) Um conjunto diverso de clientes pode ser utilizado, por exemplo, como foco de uma pesquisa de mercado.
Nós podemos agora definir o seguinte problema:

Problema SUBCONJUNTO DIVERSO
Entrada: Uma matriz A de dimensões $m \times n$ como definida acima, e um inteiro positivo k.
Pergunta: Existe um conjunto de pelo menos k clientes que é diverso?

Mostre que *SUBCONJUNTO DIVERSO* é $NP-$completo.

11. Considere um conjunto $A = \{a_1, \ldots, a_n\}$e uma coleção $B_1, B_2, \ldots, B_m$ de subconjuntos de A.
Nós dizemos que um conjunto $H \subseteq A$ é um conjunto sucesso para a coleção $B_1, B_2, \ldots, B_m$ se H contem pelo menos um elemento de cada B_i - isto é, se $H \cap B_i$ não é vazia para cada i. Nós podemos agora definir o seguinte problema:

Problema CONJUNTO SUCESSO
Entrada: Um conjunto $A = \{a_1, \ldots, a_n\}$, uma coleção $B_1, B_2, \ldots, B_m$ de subconjuntos de A, *e um inteiro positivo k.*
Pergunta: Existe um conjunto sucesso H de tamanho no máximo k?

Mostre que *CONJUNTO SUCESSO* é $NP-$completo.

12. Dado um grafo $G = (V, E)$ e um subconjunto $S \subseteq V$, mostre que as seguintes declarações são equivalentes:

a) S é um conjunto independente,

b) $V - S$ é uma cobertura por vértice,
c) S é um clique no grafo complementar $\bar{G} = (V, \bar{E})$, onde dois vértices são adjacentes se e somente se eles não são adjacentes em G.

13. Nós dizemos que dois grafos não dirigidos G e H são isomórficos se os nós de G podem ser reordenados de modo a que este seja idêntico ao grafo H. Mostre que o seguinte problema pertence à classe NP

Problema ISOMORFISMO
Entrada: Dois grafos não dirigidos G e H.
Pergunta: Os dois grafos são isomórficos?

14. Assuma que ϕ é uma fórmula FNC em que toda cláusula tem tamanho 3. Uma $\neq$atribuição as variáveis de ϕ é uma onde cada cláusula contem dois literais com valores lógicos diferentes. Por outras palavras, uma $\neq$atribuição satisfaz ϕ sem atribuir o valor lógico verdadeiro aos três literais da cláusula.

a) Mostre que o inverso de uma $\neq$atribuição à ϕ também é uma $\neq -$atribuição.
b) Prove que o seguinte problema pertence à classe NP

Problema $\neq$ATRIBUIÇÃO
Entrada: Dada um fórmula FNC ϕ.
Pergunta: Existe $\neq$ATRIBUIÇÃO *que satisfaz a fórmula?*

15. Um corte em um grafo não dirigido é uma separação dos vértices de V em dois subconjuntos disjuntos S e T. O tamanho do corte é o número de arestas que tem uma extremidade em S e outra extremidade em T. Mostre que o seguinte problema pertence à classe NP

Problema CORTEMAX
Entrada: Um grafo não dirigido G e um inteiro k.
Pergunta: Existe um corte de tamanho k em G?

16. Considere o seguinte texto extraído de uma lista de discussão de um curso de Teoria da Computação:

> "Pessoal, estou estudando a algum tempo para ver se para a prova de sexta da tempo de assimilar todas as ideias que aprendemos. E hoje estava aqui pensando com meus botões e cheguei a um dilema, será que alguém pode me ajudar?
>
> Primeiro nós vimos em aula que um problema X é completo com relação a uma classe de complexidade se todo problema dessa classe pode ser reduzido à X.
>
> Aí nós vimos que o problema *VALOR-CIRCUITO* é um problema completo com relação à classe P. Beleza, lembram da prova com a tabela de computação e tal ...
>
> Depois vimos que *SAT* é $NP-$completo.
>
> Porém antes disso nós tínhamos brincado um pouco com reduções e descobrimos que era possível reduzir *VALOR-CIRCUITO* para *CIRCUITO-SAT* (sem problemas) e *CIRCUITO-SAT* para *SAT*. Além disso descobrimos que reduções são transitivas.

O meu problema:

Se eu consigo reduzir qualquer problema pertencente à P para *Valor-Circuito* então *Valor-Circuito* é um problema $P-$completo. Mas aí eu posso reduzir todos os problemas para *Sat* (basta reduzir primeiro para *Valor-Circuito* e depois pra *Circuito-Sat* e depois pra *Sat*). Então qualquer problema pertencente a classe P pode ser reduzido pra *Sat*, e então *Sat* é $P-$completo. E se *Sat* é $P-$completo então $P = NP$!!! Ou eu sou um gênio (o que não chega nem perto da verdade) e provei isso, ou eu estou tão confuso que não estou entendendo onde errei! O que eu não entendi? Tem alguma regra escondida na transitividade das reduções que eu não lembrei? Ou outra coisa?

Alguém me ajuda porque estou bem confuso agora ...

Valeu!"

O que há de errado com o raciocínio do aluno?

17. O problema *CC* é definido como:

    Problema CC
    Entrada: Um conjunto X de elementos, uma coleção de F de subconjuntos de X, e um inteiro k.
    Pergunta: Existe uma subcoleção de F de tamanho menor ou igual ao inteiro k que inclua todos os elementos de X.

    Mostre que este problema é $NP-$completo. Utilize como ponto de partida o problema da *Cobertura por Vértice*.

18. O problema *Sat-Duplo* é definido como:

    Problema Sat-Duplo
    Entrada: Uma fórmula lógica ϕ escrita em FNC.
    Pergunta: Existem duas atribuições que satisfazem ϕ?

    Mostre que problema *Sat-Duplo* é $NP-$completo.

19. Suponha que você tenha acesso a um oráculo, o qual pode resolver o problema de decisão Clique. Mostre que, perguntando-lhe um número polinomial de questões, você pode encontrar um clique de tamanho máximo em um grafo não dirigido G.

20. Um leilão combinatorial é um mecanismo específico desenvolvido pelos economistas para vender uma coleção de itens para um grupo de potenciais compradores. Por exemplo, suponha que existam n itens a venda, os quais são rotulados de $I_1, \ldots, I_n$. Cada item é indivisível e pode ser vendido a apenas uma pessoa. Agora, m pessoas diferentes colocam seus lances: O $i-$ésimo lance especifica um subconjunto de itens S_i, e um preço de oferta x_i que o comprador está disposto a pagar pelos itens do conjunto S_i. Nós representaremos este lance como um par (S_i, x_i).
    Um leiloeiro agora abre os envelopes que contêm os m lances; ele escolhe aceitar alguns desses lances e rejeitar outros. Cada pessoa cujo lance i é aceito adquire todos os itens do correspondente conjunto S_i. Assim a regra é que nenhum par de lances aceitos pode especificar conjuntos que contêm um item comum, uma vez que isto implicaria que o mesmo item foi vendido para duas pessoas diferentes.
    O leiloeiro obtém a soma dos preços dos lances aceitos. O objetivo

do leiloeiro é obter a maior quantia possível.

Problema VENCEDORES_LEILÃO_COMBINATORIAL
Entrada: Um conjunto de itens $I_1, \ldots, I_n$*, um conjunto de lances* $(S_1, x_1), \ldots, (S_m, x_m)$*, e um número real positivo* B*.*
Pergunta: Existe uma coleção de lances que o leiloeiro pode aceitar de modo a obter uma quantia maior ou igual a B*?*

Prove que esse problema é $NP-$completo.

21. Dado um grafo não dirigido G, seu número cromático é o menor número de cores que nós necessitamos para colori-lo; isto é, o menor k tal que o grafo pode ser pintado com k cores, de modo que vértices adjacentes tenham cores diferentes. Considere o seguinte problema:

    Problema NÚMERO CROMÁTICO
    Entrada: Um grafo não dirigido G *e um inteiro* k*.*
    Pergunta: O número cromático de G *é* k*?*

    Você acredita que este pertence à classe NP? Por quê?

22. Vamos considerar um caso especial de QSat em que a fórmula Booleana subjacente não tem nenhuma variável negada. Especificamente, $\Phi(x_1, \cdots, x_n)$ é uma fórmula Booleana da forma

$$C_1 \wedge C_2 \wedge \cdots \wedge C_k, \tag{1.29}$$

    onde cada cláusula C_i é uma disjunção de três termos. Nós dizemos que Φ é monótona se cada termo em cada cláusula consiste de uma variável não negada - isto é, cada termo é igual à x_i. Considere o seguinte problema:

    Problema QSAT MONÓTONO
    Entrada: Uma fórmula QSat $\Phi = \exists x_1 \forall x_2 \cdots \exists x_{n-2} \forall x_{n-1} \exists x_n \, \phi(x_1, \cdots, x_n)$ *monótona.*
    Pergunta: Φ *é verdade?*

    Faça uma das seguintes duas coisas: Prove que o problema *QSAT MONÓTONO* é $PSpace-$completo; ou apresente um algoritmo de tempo polinomial para esse problema. (Note o algoritmo deve executar em tempo polinomial.)

23. Uma sequência de parênteses é bem formada se os parênteses combinam e estão alinhados. Por exemplo, as sequências (( )) ou ((( ))) ( ) são bem formadas, já a sequência ( )) (( ) não é bem formada.

    Problema PARÊNTESES BEM FORMADOS
    Entrada: Uma string de parênteses s*.*
    Pergunta: A string s *é bem formada?*

    Prove que este problema pertence à classe L.

24. Mostre que

    a) Qualquer problema *PSpace-hard* também é *NP-hard*
    b) Se toda linguagem *NP-hard* também é *PSpace-hard*, então $PSpace = NP$.

25. Demonstre que $PSpace$ é fechado sob união, intersecção, complemento e fecho de Kleene.

26. O jogo da velha generalizado é jogado em uma malha $n \times n$ com a primeira jogada feita pela jogador X, os jogadores X e O fazem movimentos alternados. O jogador que primeiro coloca sua marca

em cinco células consecutivas em uma linha, coluna ou diagonal vence. O jogo termina em empate se nenhum jogador vencer até todas as células estarem ocupadas. Considere o seguinte problema:

> *Problema* Jogo da Velha Generalizado
> *Entrada: Uma configuração do jogo no qual o próximo movimento pertence ao jogador X.*
> *Pergunta: O jogador X possui uma estratégia vitoriosa?*

Prove que este problema pertence à classe $PSpace$.

27. O problema do Clique tamanho $m/2$ é definido como:

> *Problema* Clique de tamanho $m/2$
> *Entrada: Um grafo não dirigido G, com m vértices.*
> *Pergunta: G possui um clique de tamanho $m/2$?*

Mostre que esse problema é $NP-$completo.

28. Considere o seguinte diálogo entre Aquiles e a Tartaruga
Aquiles: Eureka! Eureka! Encontrei uma prova que $NP \neq coNP$.
Tartaruga: Parabéns Aquiles! Com isso você resolveu o problema se $P = NP$.
Aquiles: Resolvi? Você poderia me explicar como eu resolvi esse problema?
Tartaruga: _____
Você deve provar que se $NP \neq coNP$ então $P \neq NP$.

29. Indique se as afirmações a seguir são verdadeiras ou falsas, e justifique a sua resposta:
    a) Se $X_1 \leq_k X_2$ e $X_1 \in P$, então $X_2 \in P$.
    b) Se $X_1 \leq_k X_2$ e $X_2 \in P$, então $X_1 \in P$.
    c) Se $X_1 \in NP$, então Sat $\leq_k X_1$.

30. Nós demonstramos que Programação Linear Inteira 0-1 é um problema $NP-$hard. Agora você deve reduzir o problema Cobertura por Vértice ao problema Programação Linear Inteira 0-1. Mostre que a sua redução é correta.

31. Considere o seguinte problema:

> *Problema* Dois Ciclos Hamiltonianos
> *Entrada: Entrada um grafo dirigido $G = \{V, E\}$.*
> *Pergunta: G possui dois ciclos Hamiltonianos diferentes?*

Mostre que este problema é $NP-$completo.

32. Outra variante de Sat é Horn-Sat, onde cada cláusula contem no máximo uma variável positiva. Mostre que Horn-Sat está em P. Observação: uma cláusula como $(\overline{x} \vee \overline{y} \vee z)$ é equivalente a implicação $(x \wedge y) \rightarrow z$. Projete um algoritmo que começa com as cláusulas unitárias e determina o menor conjunto possível de variáveis que tem que ser verdadeiras.

33. Considere o problema Maioria-3Sat, no qual cada cláusula contem três literais e pelo menos dois deles devem ser verdadeiros para que esta esteja satisfeita. Mostre que este problema está em P reduzindo-o à 2Sat, ou mostre que este é $NP-$completo reduzindo 3Sat a este problema.

34. Suponha que você está ajudando a organizar uma colônia de férias. A colônia deseja ter pelo menos um educador físico especializado

em cada um dos n esportes que serão praticados pelo usuários da colônia (futebol, vólei, natação, etc). A colônia recebeu o curriculum de m potencias candidatos. Para cada um dos n esportes, existe um conjunto dos m candidatos qualificados para àquele esporte. A verba para contratação de pessoal permite que sejam contratados no máximo k educadores físicos, onde $k < m$. É possível contratar pelo menos um especialista em cada um dos k esportes? Mostre que este problema é $NP-$completo.

35. Você contratado como consultor de uma empresa que possui um sistema de tempo real em que processos assíncronos utilizam vários recursos compartilhados. Assim o sistema possui um conjunto de n processos e um conjunto de m recursos. Em qualquer instante, cada processo especifica um conjunto de recursos que este necessita utilizar. Cada recurso pode ser requisitado por muitos processos ao mesmo tempo; mas este só pode ser utilizado por um único processo de cada vez. Seu trabalho consiste em alocar recursos aos processos que os demandaram. Se um processo obtém todos os recursos que este demanda, nós dizemos que o processo está ativo; caso contrário este está bloqueado. Você quer determinar uma alocação de modo que o maior número possível de processos estejam ativos. Nós podemos agora estabelecer o seguinte problema:

    Problema Alocação de Recursos
    Entrada: Um conjunto de processos, um conjunto de recursos, uma família de conjuntos de recursos demandados por cada processo, e um inteiro k.
    Pergunta: É possível alocar recursos aos processos de modo que pelo menos k processos serão ativados?

    a) O problema Alocação de Recursos é $NP-$completo ou ele pode ser resolvido em tempo polinomial?
    b) Para o caso especial no qual $k = 2$, o problema Alocação de Recursos é polinomial ou $NP-$completo?
    c) Para o caso especial quando existem dois tipos de recursos, por exemplo, roteadores e servidos de banco de dados, e cada processo demanda no máximo um recurso de cada tipo, o problema pode ser resolvido em tempo polinomial ou é $NP-$completo?
    d) Para o caso em que cada recurso pode ser demandado por no máximo dois processos; o problema pode ser resolvido em tempo polinomial ou é $NP-$completo?

36. Um caso especial do problema Sat é àquele em que a fórmula normal conjuntiva só possui literais positivos, ou seja todo literal é uma variável proposicional. A satisfabilidade da fórmula, nesse caso, é trivial, basta ajustar todas as variáveis para o valor lógico verdadeiro, por exemplo,

$$\phi = (x_1 \vee x_2) \wedge (x_1 \vee x_3) \wedge (x_2 \vee x_3),$$

    é satisfeita fazendo x_1, x_2 e x_3 assumirem o valor verdadeiro. Mas a fórmula ϕ também é satisfeita fazendo x_1e x_2 verdadeiros e x_3 falso. Nós podemos então definir o seguinte problema

Problema SAT NÃO TÃO TRIVIAL
Entrada: Uma fórmula lógica ϕ positiva, ou seja sem literais negativos, escrita em FNC, e um inteiro k.
Pergunta: É possível satisfazer ϕ fazendo k variáveis assumirem o valor lógico verdadeiro?

Mostre que este problema é $NP-$completo.

37. Prove que o seguinte problema é $NP-$completo.

Problema ÁRVORE GERADORA COM DETERMINADO NÚMERO DE FOLHAS
Entrada: Um grafo não dirigido $G = (V, E)$ e um inteiro k.
Pergunta: O grafo G possui uma árvore geradora com no máximo k folhas?

38. Você foi contratado como consultor da empresa MidiaSocial S/A. A empresa irá iniciar uma agressiva campanha publicitária, e planeja enviar convites para um estada em um spa luxuoso para alguns influenciadores digitais. A rede de influenciadores pode ser modelada como um grafo não dirigido, de modo que cada vértice representa um influenciador e uma aresta entre dois vértices significa que os dois influenciadores se seguem mutualmente. Infelizmente restrições orçamentárias permitem que apenas k influenciadores possam ser convidados. A empresa deseja que cada influenciador não convidado conheça pelo menos um influenciador convidado, de modo que este possa receber boas informações a respeito da empresa. Prove que o seguinte problema é $NP-$completo.

Problema INFLUENCIADORES
Entrada: Um grafo não dirigido $G = (V, E)$ e um inteiro positivo k.
Pergunta: É possível convidar k influenciadores de modo que para todo influenciador não convidado este conheça pelo menos um que foi convidado?

39. Considere o seguinte problema:

Problema Empacotamento de Conjuntos
Entrada: Uma coleção F de $T_1, \ldots, T_m$ conjuntos, onde $T_i \subseteq \{1, \ldots, n\}$,para $1 \leq i \leq m$, e um inteiro positivo $k \leq m$.
Pergunta: É possível escolher k conjuntos de F que sejam disjuntos dois a dois?

Dê uma redução direta de Empacotamento de Conjunto para o problema Sat.

40. O rei Arthur foi ferido em uma batalha ele precisa retornar imediatamente para ilha de Avalon onde os poderes mágicos dos druidas o ajudarão a se recuperar. Infelizmente o campo de batalha fica distante da ilha, e várias estradas estão repletas de soldados inimigos, o que implica em um risco considerável de viajar por estas estradas. Dessa forma cada estrada possui dois valores, um representando o tamanho da estrada em milhas e o outro representando o número de soldados inimigos patrulhando a estrada. O que nós leva ao seguinte problema:

Problema CAMINHO PARA AVALON
Entrada: Um grafo não dirigido $G = (V, E)$, onde cada aresta $e \in E$ possui dois inteiros associados, d_e e r_e, onde o primeiro representa o tamanho da aresta, em milhas, e o segundo o número de soldados na aresta. Dois vértices s e t e dois inteiros positivos D e R.
Pergunta: É possível o rei ir de s para t percorrendo no máximo D milhas e encontrando no máximo R soldados ao longo do caminho?

Mostre que o problema CAMINHO PARA AVALON É $NP-$completo.

41. Nós vimos em sala de aula que o problema BIPARTIDO pode ser resolvido em tempo polinomial

Problema BIPARTIDO
Entrada: Um grafo não dirigido $G = (V, E)$.
Pergunta: É possível particionar os vértices de G em dois subconjuntos disjuntos S e T tal que para toda aresta $(u, v) \in E$ nós temos que $u \in S$ e $v \in T$ ou $v \in S$ e $u \in T$?

Utilize este fato para mostrar que o problema BIPARTIDO_EM_CLIQUES, definido abaixo, também pertence à classe P.

Problema BIPARTIDO_EM_CLIQUES
Entrada: Um grafo não dirigido $G = (V, E)$.
Pergunta: É possível particionar os vértices de G em dois subconjuntos disjuntos S e T tal que ambos os subconjuntos são cliques?

42. Considere o seguinte problema

Problema CAMINHOS SEM VÉRTICES COMUNS
Entrada: Um grafo dirigido $G = (V, E)$, e k pares de vértices $(f_1, d_1), \ldots, (f_k, d_k)$.
Pergunta: Existem k caminhos $C_1, \ldots, C_k$, onde C_i é um caminho do vértice f_i para o vértice d_i, tal que nenhum vértice apareça em dois caminhos?

Mostre que o problema *CAMINHOS SEM VÉRTICES COMUNS* é $NP-$completo.

43. Merlin pediu demissão do cargo de mago da corte, mudou-se para Delfos e começou a trabalhar como oráculo. Ele cobra um dracma por pergunta feita e, como é tradição entre os oráculos, só responde "sim" ou "não".
O rei Arthur sabe que um determinado grafo planar $G = (V, E)$ pode ser pintado com três cores, infelizmente ele não sabe como pintá-lo. Ele então tomou a atitude drástica de viajar até Delfos e consultar o oráculo. Como Arthur poderá determinar, com a ajuda do oráculo Merlin, uma coloração apropriada de G com três cores, se a única pergunta que ele pode fazer ao oráculo é se um grafo *planar* é $3-$colorível, gastando uma quantia de dracmas que é polinomial no tamanho do grafo?
Observação: Uma face em um grafo planar é uma região limitada por arestas. Nós dizemos que um grafo planar está triangularizado se todas as suas faces têm três vértices. Como nós podemos pintar um grafo planar triangularizado? Como nós podemos triangularizar um grafo planar?

Apêndices

Bibliografia

Aqui estão as referências em ordem alfabética.

[1] K. Appel e W. Haken. "Every planar map is four colorable. Part I: Discharging". Em: *Illinois J. Math.* 21.3 (set. de 1977), pp. 429–490. DOI: 10.1215/ijm/1256049011 (ver p. 30).

[2] K. Appel, W. Haken e J. Koch. "Every planar map is four colorable. Part II: Reducibility". Em: *Illinois J. Math.* 21.3 (set. de 1977), pp. 491–567. DOI: 10.1215/ijm/1256049012 (ver p. 30).

[3] Sanjeev Arora e Boaz Barak. *Computational Complexity - A Modern Approach*. Cambridge University Press, 2009 (ver p. 68).

[4] Bengt Aspvall, Michael F. Plass e Robert Endre Tarjan. "A Linear-Time Algorithm for Testing the Truth of Certain Quantified Boolean Formulas". Em: *Inf. Process. Lett.* 8.3 (1979), pp. 121–123. DOI: 10.1016/0020-0190(79)90002-4 (ver p. 19).

[5] Jeffrey Bosboom et al. "Losing at Checkers is Hard". Em: *CoRR* abs/1806.05657 (2018) (ver p. 56).

[6] Stephen A. Cook. "The Complexity of Theorem-Proving Procedures". Em: *Proceedings of the 3rd Annual ACM Symposium on Theory of Computing, May 3-5, 1971, Shaker Heights, Ohio, USA*. Ed. por Michael A. Harrison, Ranan B. Banerji e Jeffrey D. Ullman. ACM, 1971, pp. 151–158. DOI: 10.1145/800157.805047 (ver p. 14).

[7] Shimon Even, Alon Itai e Adi Shamir. "On the Complexity of Timetable and Multicommodity Flow Problems". Em: *SIAM J. Comput.* 5.4 (1976), pp. 691–703. DOI: 10.1137/0205048 (ver p. 19).

[8] M. R. Garey e D. S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness (Series of Books in the Mathematical Sciences)*. First Edition. W. H. Freeman, 1979 (ver p. 37).

[9] M. R. Garey, David S. Johnson e Larry J. Stockmeyer. "Some Simplified NP-Complete Graph Problems". Em: *Theor. Comput. Sci.* 1.3 (1976), pp. 237–267. DOI: 10.1016/0304-3975(76)90059-1 (ver p. 41).

[10] Robert A. Hearn. "Games, puzzles, and computation". Tese de dout. Massachusetts Institute of Technology, Cambridge, MA, USA, 2006 (ver p. 31).

[11] Richard M. Karp. "Reducibility Among Combinatorial Problems". Em: *Proceedings of a symposium on the Complexity of Computer Computations, held March 20-22, 1972, at the IBM Thomas J. Watson Research Center, Yorktown Heights, New York, USA*. Ed. por Raymond E. Miller e James W. Thatcher. The IBM Research Symposia Series. Plenum Press, New York, 1972, pp. 85–103. DOI: 10.1007/978-1-4684-2001-2_9 (ver pp. 20, 22, 24).

[12] L.G. Khachiyan. "Polynomial algorithms in linear programming". Em: *USSR Computational Mathematics and Mathematical Physics* 20.1 (1980), pp. 53–72. DOI: https://doi.org/10.1016/0041-5553(80)90061-0 (ver p. 39).

[13] Leonid A. Levin. "Universal Sequential Search Problems". Em: *Problems of Information Transmission* 9.3 (1973) (ver p. 14).

[14] David Lichtenstein e Michael Sipser. "GO Is Polynomial-Space Hard". Em: *J. ACM* 27.2 (1980), pp. 393–401. DOI: 10.1145/322186.322201 (ver p. 53).

[15] Richard J. Lipton e Kenneth W. Regan. *People, Problems, and Proofs - Essays from Gödel's Lost Letter: 2010*. Springer, 2013 (ver p. 65).

[16] Walter J. Savitch. "Relationships Between Nondeterministic and Deterministic Tape Complexities". Em: *J. Comput. Syst. Sci.* 4.2 (1970), pp. 177–192. DOI: 10.1016/S0022-0000(70)80006-X (ver p. 52).

[17] Thomas J. Schaefer. "On the Complexity of Some Two-Person Perfect-Information Games". Em: *J. Comput. Syst. Sci.* 16.2 (1978), pp. 185–225. DOI: 10.1016/0022-0000(78)90045-4 (ver p. 57).

[18] Alan M. Turing. "On Computable Numbers, with an Application to the Entscheidungsproblem". Em: *Proceedings of the London Mathematical Society* 2.42 (1936), pp. 230–265 (ver p. 65).

[19] Stephen Wolfram. "Statistical mechanics of cellular automata". Em: *Rev. Mod. Phys.* 55 (3 jul. de 1983), pp. 601–644. DOI: 10.1103/RevModPhys.55.601 (ver p. 47).

Letras Gregas com Pronúncia

Character	Name	Character	Name
α	alpha *AL-fuh*	ν	nu *NEW*
β	beta *BAY-tuh*	ξ, Ξ	xi *KSIGH*
γ, Γ	gamma *GAM-muh*	o	omicron *OM-uh-CRON*
δ, Δ	delta *DEL-tuh*	π, Π	pi *PIE*
ϵ	epsilon *EP-suh-lon*	ρ	rho *ROW*
ζ	zeta *ZAY-tuh*	σ, Σ	sigma *SIG-muh*
η	eta *AY-tuh*	τ	tau *TOW (as in cow)*
θ, Θ	theta *THAY-tuh*	υ, Υ	upsilon *OOP-suh-LON*
ι	iota *eye-OH-tuh*	ϕ, Φ	phi *FEE, or FI (as in hi)*
κ	kappa *KAP-uh*	χ	chi *KI (as in hi)*
λ, Λ	lambda *LAM-duh*	ψ, Ψ	psi *SIGH, or PSIGH*
μ	mu *MEW*	ω, Ω	omega *oh-MAY-guh*

Capitals shown are the ones that differ from Roman capitals.

Alphabetical Index

$NTempo(f(n))$, 8

Caminho Hamiltoniano Dirigido, 35
Classe NP, 6
Classe $\mathcal{NP}$, 7
Classe P, 4
Cobertura por Vértice, 24

Frank Nelson Cole, 6

goto ambas, 8

Marin Mersenne, 6
matching, 9
Matching Bipartido Perfeito, 9

problema do caminho Hamiltoniano, 35
problema do *Ciclo Hamiltoniano*, 36

Problema Percorrendo o Reino, 35
Problema Protegendo a Fortaleza, 24
préfacio, iii

Redução, 9
robustez da classe P, 5

Tempo(f(n)), 4